# Random Robust Secret Sharing with Perfect Privacy and its Applications

Mohammad Hassan Ameri[1], Jeremiah Blocki[1]

[1]Purdue University, Department of Computer Science
West Lafayette, IN, USA
{mameriek, jblocki}@purdue.edu

**Abstract.** Secret Sharing schemes allows a dealer to distribute $n$ shares $s_1,...,s_n$ of a secret $s$ so that any $t$ shares suffice to reconstruct the secret, while any $t-1$ shares reveal no information about $s$. In fact, schemes such as Shamir Secret Sharing satisfy a stronger guarantee called $(t-1)$-perfect privacy, meaning that for any subset $S \subseteq [n]$ with $|S| \leq t-1$, the joint distribution $(s_i)_{i \in S}$ is uniformly distributed over its domain. This strong guarantee is essential for applications such as fuzzy password-authenticated key exchange (fPAKE) and conditional encryption — a recent cryptographic primitive introduced to enable secure personalized password typo correction. Unfortunately, Shamir secret sharing is not robust: corrupted shares can prevent correct reconstruction or cause reconstruction of an incorrect secret. Existing robust secret sharing schemes address this issue but necessarily sacrifice perfect privacy. We introduce and construct *Random Robust Secret Sharing with Perfect Privacy* (RRSS), a new notion that preserves $(t-1)$-perfect privacy while providing robustness against random share corruptions. In our schemes, the secret is recovered with high probability even if an arbitrary subset of up to $n-t$ shares is independently corrupted at random. We demonstrate the utility of RRSS through two applications. First, we present the first practically efficient fPAKE construction that tolerates Hamming errors. Second, we obtain the first efficient conditional encryption scheme for arbitrary Hamming distances, improving upon prior work that achieved efficiency only for constant distances. We implement both constructions and empirically demonstrate their practicality.

**Keywords:** Conditional Encryption, Fuzzy Password Authenticated Key Exchange, Robust Secret Sharing

## 1 Introduction

Secret sharing schemes [26] are foundational cryptographic primitive which allows a dealer disperse a secret among $n$ parties so that any subset of $t$ parties can collaborate to recover the secret, whilst any subset of *at most* $t-1$ parties will fail to recover the secret. More precisely, in a $t$ out of $n$ secret sharing scheme, the dealer will distribute $n$ shares $\left([[s]]_1,...,[[s]]_n\right)$ of the secret $s$ among $n$ players such that any subset of $t$ parties can collaborate to reconstruct the secret $s$, while any combination of at most $t-1$ parties cannot learn anything about the secret in an information theoretic sense. Some secret sharing schemes (including Shamir's [26]) satisfy an even strong notion called $(t-1)$-perfect privacy which intuitively ensures that for any subset $S \subseteq [n]$ of at most $|S| \leq t-1$ shares, the joint distribution $\left([[s]]_i\right)_{i \in S}$ is uniform. This condition is

sufficient, but not always necessary, to ensure that any subset of $t-1$ parties cannot learn anything about the secret. The stronger $(t-1)$-perfect privacy requirement can be useful as it additionally ensures that, if $n-t+1$ shares are randomly corrupted, then it is impossible to distinguish corrupted shares from correct shares.

In some applications (e.g., secure multi-party computation [3] and secure cloud storage [13], secure message transmission [15, 19, 20], distributed algorithm [9]) it is useful to ensure that the Secret Sharing scheme share recovery algorithm is *robust* to a few incorrect shares [8, 24, 27]. This ensures that it is possible to identify and remove incorrect shares if some parties shares were corrupted either maliciously or randomly. Intuitively, if we receive $n$ shares and at most $n-t$ of these shares were corrupted then we would like to be able to (1) identify all corrupted shares, and (2) recover the secret using $t$ of the renaming uncorrupted shared. There has been a line of work on designing and improving robust secrete sharing schemes (e.g., [3, 4, 9, 11, 12, 14, 17, 21]. Each of these constructions utilize an information theoretic MAC which is embedded in individual shares and then used to identify dishonest parties who report an incorrect share. They all use a pairwise MAC between all pairs of parties so that each honest party $i$ can independently (in)validate the share from any other party $j$. This is a useful property in some MPC applications as it allows each honest party to independently identify parties that behaved dishonestly and recover the correct secret. However, this property also runs directly counter to the stronger guarantee of $(t-1)$-perfect privacy. In particular, a subset $([[s]]_i,[[s]]_j)$ of just $2 \leq t$ valid shares can already be trivially distinguished from a uniformly random string simply by validating the MAC.

There are some applications where the stronger guarantee of perfect privacy is required. As an example we consider the (relaxed) Fuzzy Password-based Authenticated Key Exchange (fPAKE) protocol from [5, 16]. Intuitively, an (exact) PAKE protocol allows Alice and Bob to securely agree on a session key $K$ as long as they share a (possibly low-entropy) password $pwd$ — if the passwords don't match then Alice and Bob should not learn anything about the other party's password. By contrast, a fPAKE allows Alice an Bob to agree on the session key $K$ as long as their passwords are close under some distance metric e.g., even if their passwords don't exactly match they should be able to exchange a key as long as the hamming distance between the two passwords is below a threshold $\ell$. Two concrete constructions of fPAKE for Hamming distance are presented in [16] and [5]. Intuitively, these protocols work by having one party (Alice) generate $n$ shares $([[s]]_1,...,[[s]]_n)$ of a secret session key $K$ where $n$ is the number of characters in the password. Then for each individual character Alice and Bob run a special PAKE protocol to exchange each secret share $[[s]]_i$. If the passwords agree on character $i$ then Bob will receive the correct share $[[s]]_i$ otherwise Bob will receive a randomly corrupted share $[[s']]_i$. The key idea is that Bob will obtain enough shares to recover the session key $K$ if and only if the Hamming Distance between the passwords is at most $n-t$. However, the above description is overly simplified and has a problem. If the secret sharing scheme is not robust then Bob will not be able to identify the corrupted shares and recover the secret key $K$. It is tempting to simply fix the problem by using a robust secret sharing scheme. However, this introduces a major security issue. Suppose for example, that Bob's password only matches Alice's password on the first character so that Bob receives the uncorrupted share $[[s]]_i$ and randomly corrupted shares $[[s']]_j$ for all $j > 1$. Bob can

use the MACs embedded in $[[s]]_1$ to conclude that the first character matches while each the remaining characters are incorrect. Thus, even though the passwords do not match Bob learns a lot of information about Alice's password — over time a malicious Bob would be able to quickly guess each individual character of Alice's password.

Dupont et al. [16] address this issue by constructing a new secret sharing scheme achieving $t_2$-robustness and $t_1$-perfect privacy with $t_1 < t_2 - 1$. Intuitively, $t_2$-robustness means that if we are given $n$ shares and at most $n - t_2$ of those shares are corrupt then it is possible to identify the corrupted shares and recover the secret. $t_1$-perfect privacy means that for any subset $S \subseteq [n]$ of $|S| \leq t_1$ shares that the joint distribution $(s_i)_{i \in S}$ is uniformly random. However, the secret sharing scheme of Dupont et al. [16] has a gap between the two parameters $t_2$ and $t_1$ i.e., $t_1 < t_2 - 1$. In the (relaxed) $\mathsf{fPAKE}$ protocol that Bob will only be able to recover the session key $K$ the Hamming Distance between the two passwords is at most $n - t_2$ so that at least $t_2$ characters match. However, the security guarantee only prevents Alice/Bob from learning useful information about the other's password if the Hamming Distance is at least $n - t_1 \geq n - t_2 + 2$. Because of this gap, one cannot truly consider the protocol of Dupont et al. [16] (or [5]) to be an $\mathsf{fPAKE}$, since it only achieves a relaxed/weakened security notion. [5] reduces the size of the gap between $t_2$ and $t_1$ but there is still a gap. Ideally we would like to have a secret sharing scheme that is $t_2$-robust and $t_1$-perfectly private with no gap i.e., $t_2 = t_1 + 1$.

Conditional Encryption [1] provides yet another example where the stronger guarantee of $(t-1)$-perfect privacy is critical. Conditional Encryption is a primitive introduced by Ameri and Blocki [1] to enable secure personalized password typo correction. Intuitively, a conditional encryption scheme is a public key encryption scheme augmented by the addition of a new conditional encryption algorithm $\mathsf{CEnc}$. The conditional encryption algorithm takes as input the public key $pk$, a ciphertext $c_1$ encrypting an unknown message $m_1$, a control message $m_2$ and a payload message $m_3$. The output if a conditional ciphertext $c$ with the guarantee that (1) if the unknown message $m_1$ is related to the control message $m_2$ (i.e., $P(m_1, m_2) = 1$) then $c$ will decrypt to the payload message $m_3$, (2) unknown message $m_1$ is not related to the control message $m_2$ (i.e., $P(m_1, m_2) = 0$) then the conditional ciphertext will not leak any information the control message or the payload message — even if the adversary knows the secret decryption key $sk$. Ameri and Blocki [1] constructed a conditional encryption scheme for the Hamming Distance predicate $P_{\mathtt{Ham} \leq \ell}(m_1, m_2)$ which is 1 if any only if the hamming distance between $m_1$ and $m_2$ is at most $\ell$. Their construction utilized Pallier Public Key Encryption and Shamir Secret Sharing. Intuitively, the conditional encryption algorithm will generate $n$ shares of a fresh symmetric key $K$ and work character by character to generate conditional encryptions of each share. The conditional encryption algorithm additionally encrypts the payload message $m_3$ using $K$. As long as the Hamming Distance is at most $\ell$ the decryption algorithm will recover enough shares to recover $K$ and decrypt $m_3$. The security argument crucially relies on the observation that Shamir Secret Sharing achieves $(t-1)$-perfect privacy. However, because Shamir Secret Sharing is not robust there is not an efficient way to identify remove the $n - t$ corrupted shares. Ameri and Blocki [1] addressed the challenge by simply brute-forcing over all possible $\binom{n}{n-t}$ subsets to find the correct key $K$. Thus, their construct is only efficient when the Hamming distance $\ell = n - t$ is a small constant, e.g., their primary implementation uses $\ell = 2$. Once again if we had a robust secret sharing

scheme with perfect privacy we would be able to greatly improve the construction of Ameri and Blocki [1] to obtain an efficient scheme for arbitrary Hamming Distances.

A key observation underlying both applications is that the robustness requirement is fundamentally a correctness concern, whereas perfect privacy is a security concern. In the settings of interest—namely fuzzy PAKE and conditional encryption—security must hold against fully malicious adversaries, but correctness is only required when both parties follow the protocol honestly.

Crucially, even in honest executions, it is entirely expected that some shares will be corrupted. In fPAKE, this occurs whenever the two parties' passwords differ at a particular position: the corresponding share $s_i$ obtained by the receiving party is effectively corrupted. Similarly, in conditional encryption for Hamming distance, character mismatches induce incorrect shares during decryption. However, in both cases these corrupted shares are not adversarially chosen. Instead, conditioned on honest protocol execution, the resulting corruptions are distributed uniformly at random over the share domain, arising from fresh randomness internal to the protocol rather than from malicious behavior. This distinction is critical. Existing notions of robust secret sharing are designed to tolerate arbitrary (maliciously crafted) share corruption, which necessitates heavy machinery such as pairwise MACs and inevitably conflicts with $(t-1)$-perfect privacy. Prior work [5, 16] focused on reducing the gap between $t_2$ (robustness) and $t_1$ (perfect privacy). By contrast, we ask whether or not the gap can be eliminated entirely.

> "*Can we eliminate the privacy/robustness gap by relaxing the robustness guarantee?*"

More specifically, we ask whether or not it is possible to design a secret sharing scheme with $(t-1)$-perfect privacy while maintaining robustness against up to $n-t$ randomly corrupted shares. We answer this question in the affirmative by introducing the new notion of *Random Robust Secret Sharing* (RRSS), and providing an efficient construction for any value of $t$ and $n$. We also ask whether or not such a relaxed robustness constraint is sufficient for our applications: fPAKE and Conditional Encryption. Once again we answer this question in the affirmative.

### 1.1 Our Contributions

The contributions of the paper are outlined as follows.

**1.1.1 Introducing Random Robust Secret Sharing** We introduce *Random Robust Secret Sharing* (RRSS) and formalize its security guarantees. An RRSS scheme $\Pi_{\mathsf{RRSS}}$ consists of four efficient algorithms $\Pi_{\mathsf{RRSS}} = (\mathsf{Setup}, \mathsf{ShareGen}, \mathsf{SecretRecover}, \mathsf{Identify})$. The setup algorithm $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n, t)$ takes as input the security parameter $\lambda$ along with $n$ and $t$ and generates public parameters $\mathsf{pp}$ which can be published. In our construction $\mathsf{Setup}(1^\lambda)$ can actually be deterministic, eliminating the need for trusted step. To share a secret $s$, the dealer computes a vector of $n$ shares $\boldsymbol{s} = ([[s]]_1, \ldots, [[s]]_n) \leftarrow \mathsf{ShareGen}(s, \mathsf{pp})$. The $\mathsf{Identify}$ algorithm is used to identify $t$ uncorrupted shares which can then be used to recover the secret.

A bit more formally, we say that $\Pi_{\mathsf{RRSS}}$ is an $(n,t,\epsilon(\lambda))$-RRSS scheme (see Definition 1) if it is secure against random share corruption and satisfies three properties: $(t-1)$-perfect privacy and $t$-random robustness. $(t-1)$-perfect privacy requires that the joint distribution of any subset of at most $t-1$ shares is uniform and independent of the secret. The $t$-random robustness property states that for any $S \subseteq [n]$ of size $|S| = n-t$ we have

$$\Pr_{\boldsymbol{s}'}\left[\mathsf{SecretRecover}\left(\boldsymbol{s}',\mathsf{pp}\right) = s \;\middle|\; [[s']]_i = [[s]]_i \quad \forall i \notin S\right] \geq 1-\epsilon(\lambda)$$

where the randomness is taken over the uniformly random selection of $[[s']]_i$ for each $i \in S$. This means that if up to $n-t$ shares are randomly corrupted we will still be able to recover the secret with high probability — we will pick $\epsilon(\lambda)$ negligible. The first step for the $\mathsf{SecretRecover}$ procedure is to use $\mathsf{Identify}$ to find $t$ uncorrupted shares. The $\mathsf{Identify}$ algorithm ensures that for any $S \subseteq [n]$ of size $|S| = n-t$ we have

$$\Pr_{\boldsymbol{s}'}\left[|U| \geq t \wedge U \subseteq [n] \setminus S \;\middle|\; U \leftarrow \mathsf{Identify}\left(\boldsymbol{s}',\mathsf{pp}\right) \wedge [[s']]_i = [[s]]_i \quad \forall i \notin S\right] \geq 1-\epsilon(\lambda)$$

where the randomness is taken over the uniformly random selection of $[[s']]_i$ for each $i \in S$. In particular, if $n-t$ shares are randomly corrupted we will still be able to correctly and efficiently identify the $t$ remaining shares that are correct.

As an additional (optional) property we say that the RRSS scheme is secure against malicious dealers if for all strings $[[s']]_1,\ldots,[[s']]_n$ and all $S \subseteq [n]$ of size $|S| > n-t$ we have

$$\Pr_{\boldsymbol{s}''}\left[\mathsf{Identify}\left(\boldsymbol{s}'',\mathsf{pp}\right) = \perp \;\middle|\; \boldsymbol{s}''[i] = [[s']]_i \quad \forall i \notin S\right] \geq 1-\epsilon(\lambda)$$

where the randomness is taken over the uniformly random selection of $\boldsymbol{s}'_i$ for each $i \in S$. In particular, this means that if more than $n-t$ shares are randomly corrupted, then $\mathsf{Identify}$ will output $\perp$ — even if the initial shares $[[s']]_1,\ldots,[[s']]_n$ were generated by a malicious dealer. See Definition 1 for a formal description of $(n,t,\epsilon(\lambda))$-RRSS scheme.

**1.1.2 Construction of RRSS with formal security proof** We show how to construct a $(n,t,\epsilon(\lambda))$-random robust secret sharing which provides $(t-1)$-perfect privacy and $t$-robustness against random share corruption. Our construction completely eliminates the gap between the perfect privacy parameter $t_1 = t-1$ and the robustness parameter $t_2 = t_1 + 1 = t$ from prior constructions of Robust Secret Sharing with Perfect Privacy [5,16]. In particular, either $t$ shares remain uncorrupted and we can fully recover the secret or more than $n-t$ shares are randomly corrupted, in which case the remaining shares are uniformly random and leak no information about the secret *or about which shares were corrupted*. As our primary building block, our construction utilizes Shamir Secret Sharing. In the setup phase we pick $3n$ arbitrary non-zero field elements $a_1,\ldots,a_{3n} \neq 0$ which are included in the public parameters $\mathsf{pp}$. The $\mathsf{ShareGen}$ generation algorithm will generate $n$ shares $([[s]]_1,\ldots,[[s]]_n)$ where each individual share $[[s]]_i = ([[s]]_i^0,\ldots,[[s]]_i^{6n})$ consists of a vector of $6n+1$ Shamir Secret Shares. Here, the shares $([[s]]_1^0,\ldots,[[s]]_n^0)$ are generated using a regular $(n,t)$-shamir secret

sharing scheme to generate $n$ shares of the underlying secret $s$. For $j \geq 1$ the shares $[[s]]_1^{2j-1}$,$[[s]]_1^{2j}$,...,$[[s]]_i^{2j-1}$,$[[s]]_i^{2j}$,...,$[[s]]_n^{2j-1}$,$[[s]]_n^{2j}$ are generated using a $(2n,2t)$-shamir secret sharing to generate $2n$ shares of the public value $a_j \neq 0$ which is fixed a priori. Intuitively, the first part $[[s]]_i^0$ of each share $[[s]]_i$ is used to recover the secret. The remaining parts ($[[s]]_i^1$,...,$[[s]]_i^{6n}$) are used to help efficiently identify and eliminate incorrect shares using Gaussian Elimination. We prove that if $n-t$ shares are randomly corrupted while the remaining $t$ shares are correct then with high probability the solution to our linear system of equations will correctly reveal the location of at least $t$ valid shares. Once we have identified a set $S$ of $t$ valid shares we can use the regular polynomial interpolation to recover the secret using the secrets $\{[[s]]_i^0\}_{i \in S}$. Our construction is also secure against malicious dealers i.e., if more than $n-t$ shares are randomly corrupted then (whp) the identification algorithm will recognize this and output $\perp$ and this holds even if the initial shares were generated by a malicious dealer. See Section 3 for the definition of RRSS, a full description of our construction and security proofs.

#### 1.1.3 Application 1: Efficient fPAKE Construction for Hamming Distance

As the first application of our RRSS, we use it to construct an efficient Fuzzy Password Authenticated Key Exchange (fPAKE) protocol [5,6,16] for Hamming Distance. A PAKE protocol allows two parties who share a (low-entropy) password to securely exchange a cryptographic key — key exchange is successful if and only if the passwords match exactly. In an fPAKE protocol, the key exchange process should be successful, if the passwords are "sufficiently close". If the key-exchange is not successful, then participants should not learn anything about the password of the other party other than that it was not "sufficiently close" to his/her own. We consider two passwords to be "sufficiently close" if Hamming Distance between the two passwords is at most $n-t$. As we noted the fPAKE protocols from [5,16] only achieve a relaxed security notion i.e., success is only guaranteed if the Hamming Distance is at most $n-t_2$ while security is only guaranteed if the Hamming Distance is at most $n-t_1 > n-t_2+1$. The gap arises due to the use of Robust Secret Sharing.

Equipped with our new notion of random robust secret sharing we can eliminate the gap entirely. Effectively, we can simply take the protocol from [5] and replace the Robust Secret Sharing scheme with a Random Robust Secret Sharing scheme. In particular, if the Hamming Distance between the two passwords is at most $n-t$ then key exchange will succeed while if the Hamming Distance is greater than $n-t$ then neither party learns anything about the password of the other party (other than that the Hamming Distance is greater than $n-t$). We also prove that our construction UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$ for fPAKE. We follow the definition from [5] though we only consider the special case where there is no privacy/robustness gap i.e., $t_2 = t_1+1$.

Fomichev et al. [18] implemented the (relaxed) fPAKE protocol from [5] as part of the Fast-ZIP protocol, which leverages physical context (e.g., ambient audio) to enable secure device pairing without user interaction. We modify this implementation by replacing RSS with our RRSS construction, thereby upgrading the protocol to achieve full fPAKE security. To the best of our knowledge, this is the first practical implementation of an fPAKE protocol satisfying the full (non-relaxed) ideal functionality.

We evaluate the performance of our modified protocol in terms of execution time, communication overhead, and overall delay. For fingerprints of length $n = 36$, the

sender (resp. receiver) execution time is 0.2498 (resp. 0.0447) seconds, with overall delay 0.2580 (resp. 0.2943) seconds and communication overhead 34.20318 (resp. 1.8281) KB when parameters are selected to achieve $\lambda = 128$-bit security.

For comparison, the relaxed fPAKE implementation of [18] achieves sender (resp. receiver) execution time 0.0123 (resp. 0.0485) seconds, overall delay 0.0186 (resp. 0.0605) seconds, and communication overhead 3.3008 (resp. 1.82) KB. While our construction incurs higher overhead due to the larger RRSS share size and linear-algebra-based identification step, the performance remains practical and enables stronger security guarantees.

See Section 4 for a detailed construction, and a brief overview of the UC security definition and proof. See Appendix B.2 for the full proof.

**1.1.4 Application 2: Conditional Encryption for Arbitrary Hamming Distance** As the second application of our RRSS, we use it to provide an *efficient* and secure conditional encryption scheme for Hamming Distance with arbitrarily large distance parameters $\ell$. In particular, our conditional decryption algorithm runs in time $O(n^\omega)$ i.e., the cost to solve a system of linear equations of size $O(n) \times O(n)$. By contrast, the conditional decryption algorithm of [1] runs in time $\Omega\left(\binom{n}{\ell}\right)$ which is exponential if $\ell$ is large and is only practical for very small hamming distance parameters e.g., $\ell \in \{1,2,3,4\}$. We obtain our more efficient conditional encryption construction by modifying the construction of [1] to replace the $t$ out of $n$ Shamir Secret Sharing scheme with a Random Robust Secret Sharing Scheme. Because each RRSS secret share in our constructions consists of $O(n)$ Shamir shares, the length of a conditional ciphertext scales with $O(n^2)$. This is longer than the $O(n)$ term from [1], but still manageable. Empirical analysis shows that for typical values of $n$ the final size of a conditional ciphertext size is comparable to [1]. This is due to efficient share packing optimization where we show how to pack multiple Shamir Shares into a single Pallier ciphertext.

We implement our conditional encryption construction and evaluate its performance empirically. Our results demonstrate that the scheme remains practically efficient even for larger values of $\ell$. For passwords of length at most $n = 32$, the ciphertext size is 48.08 KB and the conditional decryption time is 1.208 s. By contrast, the construction of [1] achieves ciphertext size 24.08 KB and decryption time 17.3837 s for Hamming distance threshold $\ell = 8$. Moreover, its running time grows combinatorially in $\ell$, becoming impractical as $n$ and $\ell$ increase, and is competitive only when $\ell \leq 2$.

See Section 5 for an overview of the construction and security proofs. Full technical details are provided in Appendix D.

*1.1.4.1 Implementations/Replicability:* To support replicability, we plan to release all source code as an open-source artifact. For the review process, the implementation is temporarily available via an anonymous repository at `https://github.com/mhassanameri/RRSS`. The repository includes complete implementations of RRSS, our fPAKE protocol, and our Conditional Encryption scheme for arbitrary Hamming distance.

## 2 Preliminaries

In this section, we review the notations and cryptographic primitives which will be used in the rest of the paper.

Given a set $S$ we use $s \in_R S$ to denote a uniformly random sample from $S$. Given a randomized algorithm $\mathcal{A}$ we use the notation $y = \mathcal{A}(x;r)$ to denote the deterministic output $y$ when we fix the random coins $r$ and we use $y \leftarrow \mathcal{A}(x)$ to denote the randomized output when the random coins $r$ are sampled uniformly at random.

Let $\Sigma$ denote an alphabet (e.g., ASCII or unicode). Given a string $w \in \Sigma^*$ we use $\|w\|$ to denote the length of $w$ and for $i \leq \|w\|$ we use $w[i]$ to denote the $i$th character of $w$. We let $\mathcal{M}_n = \Sigma^{\leq n}$ denote the set of all strings $w$ with length $\|w\| \leq n$ and $\Sigma^n$ denotes the set of all strings of length $n$.

Given an integer $n \geq 1$ we use $[n] = \{1,...,n\}$. Given a vector $\boldsymbol{s}$ of length $n$ and a subset $S \subseteq [n]$ we use $\boldsymbol{s}_S = \left(\boldsymbol{s}[i]\right)_{i \in S}$ to denote the subvector of length $|S|$ corresponding to the indices in $S$. Let $L = \langle l_1,...,l_{|L|} \rangle$ be list of $|L|$ elements. We also define the operation $L' = \texttt{Append}(L,l)$ which adds $l$ to the list and we have $L' = \langle l_1,...,l_{|L|},l \rangle$. We note that $l_i$ can be an element in $\mathbb{Z}_{N^2}, \Sigma^n, \mathcal{M}_n$ or $\mathcal{PWD}$, etc.

### 2.1 Hamming Distance and Close Passwords

Given two strings $w_1, w_2 \in \Sigma^n$ we use $\texttt{Ham}(w_1,w_2) = \left|\{i | w[i] \neq w[j]\}\right|$ to denote the hamming distance between them. Note that if $w_1 = w_2$ then $\texttt{Ham}(w_1,w_2) = 0$. In this paper we use Hamming Distance to determine if two passwords $\texttt{pwd}_1, \texttt{pwd}_2$ are close e.g., we define a predicate $P_t(\texttt{pwd}_1, \texttt{pwd}_2) = 1$ if $\texttt{Ham}(\texttt{pwd}_1, \texttt{pwd}_2) \leq t$ for some fixed threshold $t \leq n$.

It will be convenient to assume that all passwords have the same length. While user passwords are generally short, thy do not all have the exact same length. Thus, we define the 1 to 1 function $\texttt{Pad}: \Sigma^{\leq n-1} \rightarrow \Sigma^n$ and consider $\mathcal{PWD} = \Sigma^n$ to be the set of all possible length $\leq n-1$ passwords after padding. In practice, it would be reasonable to select $n = 30$ as essentially all user passwords are shorter than this (e.g., over 99.9% of leaked RockYou passwords were less than 30 characters.). The symbol "$\|$" will be used for concatenation. Thus, $y = x_1 \| x_2$ is concatenation of $x_1$ and $x_2$. Given two strings $\texttt{pwd}_1, \texttt{pwd}_2 \in \Sigma^{\leq n-1}$ we will slightly abuse notation and write $\texttt{Ham}(\texttt{pwd}_1, \texttt{pwd}_2)$ instead of $\texttt{Ham}(\texttt{Pad}(\texttt{pwd}_1), \texttt{Pad}(\texttt{pwd}_2))$.

### 2.2 Shamir Secret Sharing

Let $\mathbb{F}$ be a finite field of size $|\mathbb{F}| \geq n+1$. The Shamir Secret Sharing Scheme [26] over a field $\mathbb{F}$ consists of three algorithms $(\mathsf{Setup}, \mathsf{ShareGen}, \mathsf{SecretRecover})$. The algorithm $\mathsf{Setup}(n,t)$ takes $n$ and input, picks $n$ distinct points $x_1,...,x_n \in \mathbb{F} \setminus \{0\}$ from the finite field $\mathbb{F}$ and outputs public parameters $\mathsf{pp} = ((n,t), x_1,...,x_n)$. The share generation algorithm $\mathsf{ShareGen}(s, \mathsf{pp})$ takes as input a secret $s \in \mathbb{F}$ along with the public parameters $\mathsf{pp}$ and (1) sets $a_0 = s$, (2) picks coefficients $a_1,...,a_{t-1} \in_R \mathbb{F}$ uniformly at random, (3) defines the polynomial $p(x) = \sum_{i=0}^{t-1} a_i x^i$, (4) computes $[[s]]_i = p(x_i)$ for all $1 \leq i \leq n$, and (5) outputs the shares $[[s]]_i,...,[[s]]_n$.

For any subset $S \subseteq [n]$ of $|S| = t$ shares we can use polynomial interpolation to recover the secret $s$ given all of the shares $[[s]]_j$ for each $j \in S$. In particular, the algorithm $\mathsf{SecretRecover}(S, [[s]]_S)$ will first use polynomial interpolation to recover the polynomial $p(x)$ and then output $s = p(0)$. Thus, $t$ shares suffice to recover the secret while for any subset of $|S| < t$ shares the vector $[[s]]_S$ is uniformly random over $\mathbb{F}^{|S|}$.

In a bit more detail, for any $S \subseteq [n]$ of size $|S| = t$ we can interpolate the polynomial $p(x) = \sum_{i \in S} [[s]]_i L_{i,S}(x)$ where $L_{i,S}(x) = \prod_{j \in S \setminus \{i\}} \left(\frac{x - x_i}{x_j - x_i}\right)$ denotes the Lagrange interpolating polynomial which can be computed based on the public parameters $\mathsf{pp}$. We can also more directly compute $s = p(0) = \sum_{i \in S} [[s]]_i L_{i,S}(0)$ where $L_{i,S}(0) = \prod_{j \in S \setminus \{i\}} \left(\frac{x_i}{x_i - x_j}\right)$. In particular, the secret $s$ can be expressed as a linear sum over the shares $[[s]]_j$ with coefficients $L_{j,S}(0)$ that can be calculated entirely from the public parameters.

## 3 Random Robust Secret Sharing

In this section, we will introduce the novel cryptographic notion of $t$ out of $n$ Random Robust Secret sharing with $t-1$-perfect privacy. Intuitively, $t-1$-perfect privacy means that any subset of $t-1$ shares are uniformly distributed. Random robustness means that we can (whp) *efficiently* recover the secret event up to $n-t$ of the given $n$ shares were *randomly* corrupted. By only requiring robustness against random corruptions we are able to overcome prior barriers and eliminate the robustness/privacy gap inherent in prior work [5,16]. We begin by defining Random Robust Secret Sharing in Section 3.1 and then in Section 3.2 we provide a construction. Finally, we prove that our construction is satisfies $t$-random robustness and $t-1$-perfect privacy in Section 3.3.

### 3.1 Definition of Random Robust Secret Sharing

We begin by formally defining a Random Robust Secret Sharing (RRSS) scheme $\Pi_{\mathsf{RRSS}} = (\mathsf{Setup}, \mathsf{ShareGen}, \mathsf{SecretRecover}, \mathsf{Identify})$. Intuitively, the new algorithm $\mathsf{Identify}$ is used to identify which shares were/were not randomly corrupted before we run $\mathsf{SecretRecover}$.

Before giving the formal definition, it will be helpful to introduce some notation. Given a set $\mathbb{S}$, a vector $\boldsymbol{c} = (c_1, \ldots, c_n) \in \mathbb{S}^n$ of dimension $n$ over $\mathbb{S}$ and a subset $S \subseteq [n]$ of indices, it will be useful to define

$$\mathcal{D}(\mathbb{S}, \boldsymbol{c}, S) \doteq \left\{ \boldsymbol{c}' \in \mathbb{S}^n \ : \ \boldsymbol{c}'_S = \boldsymbol{c}_S \right\},$$

as the set of all vectors $\boldsymbol{c}' = (c'_1, \ldots, c'_n) \in \mathbb{S}^n$ which are consistent with $\boldsymbol{c}$ over the indices in $S$ i.e., such that $c'_i = c_i$ for all $i \in S$. Intuitively, if $\boldsymbol{c}$ denotes a vector of $n$ shares then sampling $\boldsymbol{c}'$ uniformly at random from $\mathcal{D}(\mathbb{S}_\lambda, c, S)$ is equivalent to replacing $c_i$ with a uniformly random sample $c'_i \in_R \mathbb{S}_\lambda$ for each $i \notin S$. $t$-random robustness tells us that if $|S| \geq t$ then (whp) we should still be able identify $t$ uncorrupted shares and recover the secret.

**Definition 1.** *Let $\lambda \in \mathbb{N}$ be the security parameter, $p$ a polynomial, $\mathbb{D}_\lambda = \{0,1\}^\lambda$ be the $\lambda$-bit secret space, $n,t \in \mathbb{N}$ with $t < n$, $PP$ set of all public parameters and $\mathbb{S}_\lambda$ the space of all possible shares. An $\big(n,t,\epsilon(\lambda)\big)$-Random Robust secret sharing scheme $\Pi_{RRSS}$ contains four probabilistic polynomial time algorithms: setup $\mathsf{Setup}: 1^\lambda \times \mathbb{N} \times \mathbb{N} \to PP$, share generation $\mathsf{ShareGen}: \mathbb{D}_\lambda \times PP \to \mathbb{S}_\lambda^n$, reconstruction $\mathsf{SecretRecover}: 2^{[n]} \times \mathbb{S}_\lambda^t \times PP \to \mathbb{D}_\lambda$ and valid share identifier $\mathsf{Identify}: \mathbb{S}_\lambda^n \times PP \to 2^{[n]}$ with the following properties:*

- *Correctness: For any secret $s \in \mathbb{D}_\lambda$, any $\mathsf{pp}$ in the support of $\mathsf{Setup}(1^\lambda,n,t)$, any $\boldsymbol{c} = ([[c]]_1,...,[[c]]_n)$ in the support of $\mathsf{ShareGen}(s,\mathsf{pp})$, and any subset $S \subseteq [n]$ with $|S| \geq t$ we have $\mathsf{SecretRecover}(S,\boldsymbol{c}_S,\mathsf{pp}) = s$. Intuitively, this property says that we can always recover the original secret if we are given $t$ shares.*
- *$(t-1)$-Perfect Privacy: for all secrets $s \in \mathbb{D}_\lambda$, all $\mathsf{pp}$ in the support of $\mathsf{Setup}(1^\lambda,n,t)$, all $S \subseteq [n]$ with $|S| \leq t-1$ and all $c' \in \mathbb{S}_\lambda^{|S|}$ we have $\Pr[\boldsymbol{c}_S = c'] = |\mathbb{S}_\lambda|^{-|S|}$ where $\boldsymbol{c} = ([[c]]_1,...,[[c]]_n) \leftarrow \mathsf{ShareGen}(s,\mathsf{pp})$ and the randomness is taken over the random coins of $\mathsf{ShareGen}$. Intuitively, for any subset $S \subseteq [n]$ of $|S| \leq t-1$ shares the joint distribution over those shares is uniformly random over $\mathbb{S}_\lambda^{|S|}$.*
- *$t$-Random Robustness: for any $s \in \mathbb{D}_\lambda$, $S \subseteq [n]$ with $|S| \geq t$, any $\mathsf{pp}$ in the support of $\mathsf{Setup}(1^\lambda,n,t)$, and any $c = ([[c]]_1,...,[[c]]_n)$ in the support of $\mathsf{ShareGen}(s,\mathsf{pp})$ we have*

$$\Pr_{\boldsymbol{c}' \in_R \mathcal{D}(\mathbb{S}_\lambda,\boldsymbol{c},S)}\left[S' = \mathsf{Identify}\Big(\boldsymbol{c}',\mathsf{pp}\Big) \subseteq S \wedge |S'| \geq t\right] \geq 1-\epsilon(\lambda)\ ,$$

*where the randomness is only taken over the uniformly random choice of $[[c']]_i$ for each $i \notin S$. Note that we must have $\boldsymbol{c}'_i = \boldsymbol{c}_i$ for all $i \in S$. Intuitively, this property tells us that if at most $n-t$ shares are randomly corrupted while the remaining shares are correct then (whp) $\mathsf{Identify}$ will correctly identify $t$ uncorrupted shares without missclassifying any corrupted shares.*

Given a vector $\boldsymbol{c}' = ([[c']]_1,...,[[c']]_n)$ of (possibly corrupted) shares it will often be convenient to write $\mathsf{SecretRecover}(\boldsymbol{c}',\mathsf{pp})$ to be the algorithm that runs $S' = \mathsf{Identify}\big(\boldsymbol{c}',\mathsf{pp}\big)$ and then outputs $\mathsf{SecretRecover}(S',\boldsymbol{c}',\mathsf{pp})$ if $|S'| \geq t$; otherwise $\mathsf{SecretRecover}(\boldsymbol{c}',\mathsf{pp})$ outputs $\perp$.

**3.1.1 Security Against Malicious Dealers** It will also be useful to define an additional (optional) security guarantee against malicious dealers which ensures that if the $i$'th share is randomly corrupted, then the index $i$ will not be output by $\mathsf{Identify}$ except with negligible probability — even if the attacker picked the initial shares maliciously with the goal of tricking the $\mathsf{Identify}$ algorithm. Similarly, if at least $n-t+1$ shares are uniformly random (and the remaining shares are fixed a priori by the malicious dealer) then (whp) the algorithm $\mathsf{Identify}$ will recognize this and output $\perp$. More formally, we say that $\mathsf{Identify}$ achieves $(t-1,\epsilon(\lambda))$-Security against malicious dealers if for any $\boldsymbol{c}' = (c'_1,...,c'_n) \in \mathbb{S}_\lambda^n$, any $\mathsf{pp}$ in the support of $\mathsf{Setup}(1^\lambda,n,t)$, and any $S \subseteq [n]$ with $|S| \geq t$ we have

$$\Pr_{\boldsymbol{c}'' \in_R \mathcal{D}(\mathbb{S}_\lambda,\boldsymbol{c}',S)}\left[\mathsf{Identify}\Big(\boldsymbol{c}'',\mathsf{pp}\Big) \neq \perp \wedge \mathsf{Identify}\Big(\boldsymbol{c}'',\mathsf{pp}\Big) \not\subseteq S\right] \leq \epsilon(\lambda)\ .$$

where the randomness is only taken over the uniformly random choice of $[[c'']]_i$ for each $i \notin S$. Note that we must have $\boldsymbol{c'}_i = \boldsymbol{c}_i$ for all $i \in S$. Similarly, for any $S \subseteq [n]$ with $|S| \leq t-1$ we have

$$\Pr_{\boldsymbol{c''} \in_R \mathcal{D}(\mathbb{S}_\lambda, \boldsymbol{c'}, S)} \left[ \mathsf{Identify}\left(\boldsymbol{c''}, \mathsf{pp}\right) = \perp \right] \geq 1 - \epsilon(\lambda) \ .$$

While this final (optional) property is not essential for our applications it is a useful guarantee, and the construction we give in Section 3.2 will satisfy this property.

### 3.2 Construction of Random Robust Secret Sharing

We now describe our $\left(n, t, 2^{-n\lambda_1}\right)$-Random Robust Secrete Sharing $\Pi_{RRSS} = (\mathsf{ShareGen}, \mathsf{SecretRecover}, \mathsf{Identify})$. We reference our construction as **Construction 1** in Appendix E. However, **Construction 1** is described completely in the text below. We first provide a high level description.

**3.2.1 High level intuition of Construction 1.** Intuitively, we use Shamir secret sharing as the primary building block. In the setup phase, we setup the parameters of our Shamir secret sharing and select a set of $3n$ non-zero values $(a_1, ..., a_{3n}) \in \left(\mathbb{F}_{2^{\lambda_1}} \setminus \{0\}\right)^{3n}$ which are part of our public parameter $\mathsf{pp}$. Each individual share $[[z]]_i$ of our secret $s$ will consist of a vector of $6n+1$ Shamir Secret shares i.e., $[[z]]_i = \left([[s]]_i, [[a]]^1_{2i-1}, ..., [[a]]^{3n}_{2i-1}, [[a]]^1_{2i}, ..., [[a]]^{3n}_{2i}\right)$. The first component $[[s]]_i$ is secret share of $s$ generated using the regular $t$ out of $n$ Shamir Secret Sharing procedure i.e., $([[s]]_1, ..., [[s]]_n) \leftarrow \Pi^{\lambda}_{SSS,n,t}.\mathsf{ShareGen}_{2^\lambda}(n, t, s)$. This is the only component of $[[z]]_i$ that is directly related to the secret $s$. The remaining Shamir Secret Shares are simply used to identify shares that have been randomly corrupted. In particular, for each $j \leq 3n$ the shares $[[a]]^j_{2i-1}$ and $[[a]]^j_{2i}$ are both Shamir Secrets of the public value $a_j$ generated using a $2t$ out of $2n$ secret sharing scheme i.e., $[[a]]^j_1, ..., [[a]]^j_{2n} \leftarrow \Pi^{\lambda_1}_{SSS,2n,2t}.\mathsf{ShareGen}_{2^{\lambda_1}}\left(2n, 2t, a_j\right)$ for all $j \in [3n]$.

The shares $[[a]]^j_i$ are useful to identify a set $S \subseteq [n]$ of at least $|S| \geq t$ uncorrupted indices using Gaussian Elimination. To provide some intuition assume that we are given shares $[[z']]_1, ..., [[z']]_n$ and that $t$ of these shares are correct while $n-t$ have been randomly corrupted. Let $S \subseteq [n]$ with $|S| = t$ denote the (currently unknown) set of valid share indexes such that $[[z']]_i = [[z']]_i$ for all $i \in S$. Given a subset $S \subseteq [n]$ of the shares of our secret $s$ it will be convenient to write $\hat{S} = \bigcup_{i \in S} \{2i-1, 2i\}$ to denote the corresponding subset of secret shares for each public valid $a_j$. Applying Lagrange interpolation we have $a_j = \sum_{i \in S} \left( L_{2i-1,\hat{S}}(0)[[a]]^j_{2i-1} + L_{2i,\hat{S}}(0)[[a]]^j_{2i} \right)$ for each $j \leq 3n$ [1]. Observe that each term $L_{2i-1,\hat{S}}(0)$ and $L_{2i,\hat{S}}(0)$ is non-zero and entirely independent of $j$ and of the secret $s$. However, we cannot directly compute $L_{2i-1,\hat{S}}(0)$ since it does depend on the subset $S \subseteq [n]$ of correct shares which is currently unknown. Instead we introduce $2n$ variables $x_i$ for each $i \leq 2n$ and define an (over-constrained) system of $3n$ linear equations where for each $j \leq 3n$ we have

[1] An astute reader will note that we should

$$a_j = \sum_{i=1}^{2n} x_i [[a']]_i^j \ .$$

Intuitively, if $S \subseteq [n]$ is a set of $t$ uncorrupted indices then the system has a valid solution by setting $x_{2i-1} = L_{2i-1,\hat{S}}(0)$ and $x_{2i} = L_{2i,\hat{S}}(0)$ for each $i \in S$ and setting $x_{2i-1} = x_{2i} = 0$ for each $i \notin S$. Thus, our approach is to solve the system of $3n$ linear equations for our $2n$ unknown variables. Once we solve for $x_1,...,x_{2n}$ we build our set $S \subseteq [n]$ of uncorrupted shares by setting $S = \{i : x_{2i} > 0 \vee x_{2i-1} \geq 0\}$. Because the system is over-constrained ($3n$ equations vs $2n$ variables) we can argue that (whp) we must have $x_{2i-1} = x_{2i} = 0$ for all shares $i \in [n] \setminus S$ that were randomly corrupted. Similarly, we can argue that (whp) we will have $\left|\{i : x_i \neq 0\}\right| \geq 2t-1$ for every feasible solution. Thus, if there are $t$ valid shares we will (whp) find a set $S$ containing $|S| \geq t$ shares and if there are fewer than $t$ valid shares there will be no feasible solution (whp).

#### 3.2.2 Full Description

- **Setup** $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n, t)$: Given the security parameter $\lambda$ we select $\lambda_1 \geq \left\lceil \frac{\lambda}{n} \right\rceil$ [2]. Let $\Pi^{\lambda}_{SSS,n,t} = (\mathsf{ShareGen}_{2^\lambda}, \mathsf{SecretRecover}_{2^\lambda})$ (resp. $\Pi^{\lambda_1}_{SSS,2n,2t} = (\mathsf{ShareGen}_{2^{\lambda_1}}, \mathsf{SecretRecover}_{2^{\lambda_1}})$) be a secure $(n,t)$ (resp. $(2n,2t)$ Shamir secret sharing over filed $\mathbb{F}_{2^\lambda}$ (resp. $\mathbb{F}_{2^{\lambda_1}}$). Let $\boldsymbol{a} = (a_1,...,a_{3n}) \in_R \left(\mathbb{F}_{2^{\lambda_1}} \setminus \{0\}\right)^{3n}$ be $3n$ non-all-zero vector whose elements are chosen from the small size finite field $\mathbb{F}_{2^{\lambda_1}}$. Return the public parameter $\mathsf{pp} = (\boldsymbol{a}, n, t, \Pi^{\lambda}_{SSS,n,t}, \Pi^{\lambda_1}_{SSS,2n,2t})$.
- **Share Generation** $\left([[z]]_1,...,[[z]]_n\right) = \mathsf{ShareGen}(s, \mathsf{pp})$: This algorithm takes as input the secret $s$, the threshold value $t$ and the number of shares $n$ and generates $n$ shares $[[z]]_i$ for all $i \in [n]$ as follows:
  1. Parse: $\boldsymbol{a} = (a_1,...,a_{3n}) \in \mathsf{pp}$.
  2. $([[s]]_1,...,[[s]]_n) \leftarrow \Pi^{\lambda}_{SSS,n,t}.\mathsf{ShareGen}_{2^\lambda}(n,t,s)$
  3. $\forall j \in [3n]$: compute $[[a]]_1^j,...,[[a]]_{2n}^j \leftarrow \Pi^{\lambda_1}_{SSS,2n,2t}.\mathsf{ShareGen}_{2^{\lambda_1}}\left(2n, 2t, a_j\right)$.
  4. $\forall i \in [n]$: Set $[[z]]_i = \left([[s]]_i, [[a]]_{2i-1}^1,...,[[a]]_{2i-1}^{3n}, [[a]]_{2i}^1,...,[[a]]_{2i}^{3n}\right)$.
  5. Return $\left(([[z]]_1...,[[z]]_n)\right)$
- **Recovery from Valid Shares:** $s \leftarrow \mathsf{SecretRecover}(S, \boldsymbol{z}_S)$: This algorithm takes as input a set $S \subseteq [n]$ of size $|S| \geq t$ and $|S|$ shares $\boldsymbol{z}_S$ and recovers the secret $s$ as follows:
  1. Parse $\boldsymbol{z}_S$ to recover $[[z]]_i$ for each $i \in S$.
  2. For each $i \in S$ extract $[[s]]_i$ from $[[z]]_i = \left([[s]]_i, [[a]]_{2i-1}^1,...,[[a]]_{2i-1}^{3n}, [[a]]_{2i}^1,...,[[a]]_{2i}^{3n}\right)$. Let $\boldsymbol{s}_S = \left([[s]]_i\right)_{i \in S}$.
  3. Return $s = \Pi^{\lambda}_{SSS,n,t}.\mathsf{SecretRecover}_{2^\lambda}(S, \boldsymbol{s}_S)$.
- **Valid Share Identification** $S \leftarrow \mathsf{Identify}\left(([[z']]_1...,[[z']]_n), \mathsf{pp}\right)$: This algorithm takes the input shares $\left([[z']]_1...,[[z']]_n\right)$ and if it contains at most $n-t$ randomly corrupted shares, it returns a set $S \subseteq [n]$ identifying the indexes of at least $t$ valid shares (whp). The share identification algorithms is described in the following steps:

[2] For our implementation we used $\lambda_1 = 8$

1. Parse: $\boldsymbol{a}=(a_1,...,a_{3n})\in \mathsf{pp}$
2. for all $i\in[n]$ parse: $[[z']]_i=\left([[s']]_i,[[a']]^1_{2i-1},...,[[a']]^{3n}_{2i-1},[[a']]^1_{2i},...,[[a']]^{3n}_{2i}\right)$
3. Form the system of linear equations $(\boldsymbol{a})^T=AX^T$ in which $A=\begin{bmatrix} a_{1,1} & ... & a_{1,2n} \\ ... & a_{i,j} & ... \\ a_{3n,1} & ... & a_{3n,2n} \end{bmatrix}$ where $a_{i,j}=[[a']]^i_j$ and $X=(x_1,...,x_{2n})$ is a $2n$ unknown variable vector.
4. Solve $\boldsymbol{a}=AX$ to find the values $x_1,...,x_{2n}$ and set $S=\{i\in[n]: x_{2i-1}\neq 0\vee x_{2i}\neq 0\}$. If there are no solutions then return $\perp$.
5. If $|S|<t$, then return $\perp$; otherwise, return $S$ as the set of valid shares.

– **Valid Share Identification and Secret Reconstruction** $\{s,\perp\}:=\mathsf{SecretRecover}\left(([[z']]_1,...,[[z']]_n),\mathsf{pp}\right)$:

This algorithm takes the input shares $\boldsymbol{z'}=([[z']]_1,...,[[z']]_n)$ and if we have at least $t$ valid shares, it returns the secret $s$; otherwise, it returns $\perp$. The reconstruction secret is described as follows.

1. Run $S\leftarrow\mathsf{Identify}\left(([[z']]_1,...,[[z']]_n),\mathsf{pp}\right)$ to identify the set of valid shares.
2. If $S=\perp$ then return $\perp$.
3. If $|S|>t$ discard the last $|S|-t$ items from $S$ to ensure that $|S|=t$.
4. For each $i\in S$ extract $[[s']]_i$ from $[[z']]_i=\left([[s']]_i,[[a']]^1_{2i-1},...,[[a']]^{3n}_{2i-1},[[a']]^1_{2i},...,[[a']]^{3n}_{2i}\right)$. Let $\boldsymbol{s'}_S=\left([[s']]_i\right)_{i\in S}$.
5. Return $s=\Pi^{\lambda}_{SSS,n,t}.\mathsf{SecretRecover}_{2^\lambda}\left(S,\boldsymbol{s'}_S\right)$.

*3.2.2.1 Share Size and Running Time:* Each share $[[z]]_i$ requires $\lambda+6n\lambda_1$ bits to encode since there are $6n$ field elements from $\mathbb{F}_{2^{\lambda_1}}$ and a single field element from $\mathbb{F}_{2^\lambda}$. Thus, the total amount of bits to encode all $n$ shares is $n\lambda+6n^2\lambda_1$. From a computational efficiency standpoint the most complex operation is solving the system of linear equations in $\mathsf{Identify}$. In particular, the time complexity is dominated by solving system of linear equations of size $O(n)\times O(n)$, i.e., $O(n^\omega)$ where $\omega\leq 2.3716$ [28].

We now make a couple of remarks about **Construction 1**.

*Remark 1.* We note that **Construction 1** is not secure against *adversarial corruptions*. For example, an attacker could take the share $[[z]]_i=\left([[s]]_i,[[a]]^1_{2i-1},...,[[a]]^{3n}_{2i-1},[[a]]^1_{2i},...,[[a]]^{3n}_{2i}\right)$ and simply modify the value of $[[s]]_i$ to obtain $[[z']]_i=\left([[s']]_i,[[a]]^1_{2i-1},...,[[a]]^{3n}_{2i-1},[[a]]^1_{2i},...,[[a]]^{3n}_{2i}\right)$. Because this first component is never used to identify (in)valid shares the modification will go undetected, and will likely result in the recovery of an incorrect secret $s'\neq s$. However, this is not a problem in our context because a random corruption of $[[z]]_i$ will randomly modify each individual component $[[a]]^j_{2i-1}$ and $[[a]]^j_{2i}$ for each $j\leq 3n$.

*Remark 2.* It is natural to wonder why we use $2t$ out of $2n$ Shamir secret sharing instead of $t$ out of $n$ secret sharing which seems to be more natural and efficient. The reason is because we made the value $a_j$ public for each $j\leq 3n$. Effectively $a_j$ becomes a "bonus share" making it possible for the attacker to interpolate the Shamir polynomial with one fewer share. Suppose that we used $t$ out of $n$ secret sharing and let $p_j(x)$ be the degree $t-1$ polynomial we picked when we generate $n$ shares $[[a]]^j_1,...[[a]]^j_n$ of $a^j$ using Shamir Secret Sharing. Normally, we would need $t$ shares

to interpolate $p_j$. However, because $p_j(0)$ is known a priori an attacker could actually interpolate $p_j$ using just $t-1$ shares.

We instead use $2t$ out of $2n$ secret sharing to prevent an attacker who has just $t-1$ shares from interpolating the polynomial $p_j(x)$. The polynomial $p_j(x)$ now has degree $2t-2$ and each share $[[z]]_i$ contains exactly two shares of $a^j$. Given $t-1$ shares $[[z]]_i$ we can only obtain $2t-2$ shares of $a^j$. Even with the "bonus share" $p_j(0)=a^j$ this will be insufficient to interpolate $p_j$. Thus, the $2t-2$ shares of $a^j$ will still be distributed uniformly at random.

### 3.3 Security proof of Construction 1

**Theorem 1.** *The construction* $\Pi_{\mathsf{RRSS}}=(\mathsf{ShareGen},\mathsf{SecretRecover},\mathsf{Identify})$ *described in* ***Construction 1*** *is a* $\left(n,t,\epsilon(\lambda)\right)$*-Random Robust Secret Sharing scheme with* $\epsilon(\lambda)\leq 2^{-\lambda}$*.* $\Pi_{\mathsf{RRSS}}$ *also achieves* $(t-1,\epsilon(\lambda))$*-Security against malicious dealers.*

**Proof of Theorem 1:** (Intuition) The full proof of Theorem 1 is deferred to Appendix A.1. We sketch the intuitive ideas here.

It is straightforward to argue that any subset $S\subseteq[n]$ of $|S|\leq t-1$ shares of $[[z]]_1,\ldots,[[z]]_n$ are distributed uniformly at random in $\left(\mathbb{F}_{2^\lambda}\times\mathbb{F}_{2^{\lambda_1}}^{6n}\right)^{|S|}$. Intuitively, this follows directly from $(t-1)$-perfect privacy of the underlying Shamir secret sharing scheme since each individual component of the share $[[z]]_i=\left([[s]]_i,[[a]]_{2i-1}^1,\ldots,[[a]]_{2i-1}^{3n},[[a]]_{2i}^1,\ldots,[[a]]_{2i}^{3n}\right)$ comes from a Shamir Secret Sharing Scheme.

$t$-Random Robustness follows intuitively because there are $3n$ linear constraints and $2n$ variables. Suppose that the first share $[[z]]_1$ is randomly corrupted to $[[z']]_1=\left([[s']]_i,[[a']]_1^1,\ldots,[[a']]_1^{3n},[[a']]_2^1,\ldots,[[a']]_2^{3n}\right)$. Now consider an a priori fixed variable assignment where $x_1\neq 0$ (or equivalently $x_2\neq 0$). Because each $[[a']]_1^j$ is uniformly random and $x_1\neq 0$ we can view each term $x_1[[a']]_1^j$ as uniformly random for each $j\leq 3n$. This means that the probability that each individual constraint is satisfied by our a priori fixed assignment is at most $2^{-\lambda_1}$ and the probability that *all* $3n$ constraints are satisfied is at most $2^{-3n\lambda_1}$. We can now union bound over all $2^{2n\lambda_1}$ possible variable assignments to conclude that, except with probability $2^{-n\lambda_1}$ we will have $x_{2i-1}=x_{2i}=0$ for all shares $[[z]]_1$ that are randomly corrupted. We note that the same exact argument applies to show that the scheme achieves security against malicious dealers. It doesn't matter whether or not the initial shares $[[z]]_1,\ldots,[[z]]_n$ were generated honestly or not. If $[[z']]_1$ is randomly corrupted and $x_1\neq 0$ then each term $x_1[[a']]_1^j$ is still uniformly random so that the probability each individual constraint is satisfied is at most $2^{-\lambda_1}$ and the probability all $3n$ constraints are satisfied is at most $2^{-3n\lambda_1}$. We can now union bound over all $2^{2n\lambda_1}$ possible variable assignments to conclude that, except with probability $2^{-n\lambda_1}$ we will have $x_{2i-1}=x_{2i}=0$ for all shares $[[z]]_1$ that are randomly corrupted which implies that $i$ will not be in the set of indices returned by $\mathsf{SecretRecover}$. If we assume that at least $n-t+1$ shares are corrupted this implies that, except with probability $2^{-n\lambda_1}$, we will have $\left|\left\{i:x_{2i}\neq 0\vee x_{2i-1}\neq 0\right\}\right|<t$ in which case $\mathsf{Identify}$ will output $\perp$ as required. We note that $2^{-\lambda_1 n}\leq 2^{-(\lambda/n)n}=2^{-\lambda}$.

A similar argument shows that, for any a priori fixed variable assignment with $\left|\{i : x_i \neq 0\}\right| < 2t-1$, the probability that the assignment is consistent with all $3n$ constraints is at most $2^{-3n\lambda_1}$. The primary difference is that we are no longer assuming that any shares are randomly corrupted. However, any subset of $2t-2$ shares from a $2t$ out of $2n$ Shamir Secret Sharing Scheme can be viewed as uniformly random so we can apply the same reasoning. We can once union bound over all $2^{2n\lambda_1}$ possible variable assignments to conclude that, except with probability $2^{-n\lambda_1}$, no assignment with $\left|\{i : x_i \neq 0\}\right| < 2t-1$ will satisfy all of our equations. This implies that the set $S = \{i : x_{2i-1} \neq 0 \vee x_{2i} \neq 0\}$ returned by $\mathsf{Identify}$ must have size at least $t$. In particular, as long there are at least $t$ valid shares to identify we will find at least $t$ valid shares (whp). In particular, if $S \subseteq [n]$ is a set of $|S| \geq t$ uncorrupted shares then we can always obtain a feasible assignment by setting $x_{2i-1} = L_{2i-1,S}(0)$ and $x_{2i} = L_{2i,S}(0)$. □

## 4 Fuzzy Password-based Authenticated Key Exchange

As our first application of Random Robust Secret Sharing, we show how to use RRSS to construct a secure and efficient Fuzzy Password-based Authenticated Key Exchange (fPAKE) protocol. A fPAKE protocol enables two parties holding noisy versions of a shared low-entropy password to securely establish a session key over an unauthenticated channel.

Prior constructions of fPAKE for Hamming distance [5,16] rely on maliciously robust secret sharing with perfect privacy. As a consequence, they achieve only a weakened security notion: the protocol may leak information about the parties' passwords whenever the Hamming Distance is at most $t_2$, while correctness (i.e., successful key exchange) is guaranteed only when the distance is at most $t_1 < t_2$. Thus, leakage may occur even when the passwords are not close enough for successful key exchange. This undesirable behavior stems from the privacy/robustness gap inherent in fully robust secret sharing schemes.

By replacing fully robust secret sharing with our Random Robust Secret Sharing scheme, we eliminate the privacy/robustness gap entirely and obtain full fPAKE security without relaxing the ideal functionality. Our construction is obtained by modifying the protocol of [5] to use RRSS. In the remainder of this section, we first present the ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$. Our definition closely follows [16], but without weakening the functionality. We then describe our construction and prove that it UC-realizes $\mathcal{F}_{\mathsf{fPAKE}}$.

### 4.1 fPAKE Ideal functionality

We present the ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$ in Figure 1. The functionality interacts with $\mathsf{Party}_0$, $\mathsf{Party}_1$, and the adversary $\mathcal{A}$ via three queries: NewSession, TestPwd, and NewKey.

Our functionality closely follows Dupont et al. [16], with one key difference. Prior work introduced a larger leakage threshold $\gamma \geq \ell$, allowing information leakage whenever $\mathtt{Ham}(\mathtt{pwd}, \mathtt{pwd}') \leq \gamma$. We eliminate this relaxation: a password guess either satisfies

$\texttt{Ham}(\texttt{pwd},\texttt{pwd}') \leq \ell$ (in which case key exchange succeeds), or it is treated as an incorrect guess with no additional leakage. Thus, our functionality is strictly stronger than those in [5, 16].

Execution proceeds as follows. A party initiates a session via NEWSESSION, submitting a session identifier, role (sender or receiver), and password pwd. The functionality records the session and informs $\mathcal{A}$ without revealing pwd. The adversary may issue a single TESTPWD query per fresh session to attempt a password guess. If $\texttt{Ham}(\texttt{pwd},\texttt{pwd}') \leq \ell$, the record is marked compromised and $\mathcal{A}$ learns the matching character positions; otherwise, the session is marked interrupted and no further information is revealed.

If the adversary guesses a sufficiently *close* password (Hamming distance at most $\ell$), then the record associated with pwd is marked compromised. Otherwise the record is marked as interrupted. The ideal functionality informs the adversary about the correctness of their guess. The ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$ is parametrized by error tolerance $\ell$ and the Hamming distance as the associated metric and we formally say that two password pwd and pwd′ are close if $d = \texttt{Ham}(\texttt{pwd},\texttt{pwd}') \leq \ell$. So if $d \leq \ell$, the ideal functionality also leaks the location of the matching characters of the tested password pwd′. In [5], the ideal functionality also leaks the locations of the matching characters in the password to the adversary when the password guess is incorrect but satisfies $\ell < d \leq \gamma$ for some parameter $\gamma$ which is related to the robust secret sharing scheme that they use. By contrast, we do not weaken the ideal functionality to allow for leakage when $\ell < d \leq \gamma$. Thus, our ideal functionality is strictly stronger than $\mathcal{F}_{\mathsf{fPAKE}}$ in [5, 16].

Upon a NEWKEY query, the functionality determines the output key according to three cases:

(i) If $\mathcal{A}$ correctly guessed a password (via TESTPWD or corruption), it specifies the resulting key.
(ii) If both parties are honest and $\texttt{Ham}(\texttt{pwd}_0,\texttt{pwd}_1) \leq \ell$, they receive the same key.
(iii) In all other cases, each party receives an different/ independent uniformly random key.

This captures the core guarantee of fPAKE: parties with sufficiently close passwords obtain the same key, while otherwise learning nothing beyond failure.

**Definition 2 (fPAKE correctness [5]).** *We say* fPAKE *protocol* $\pi$ *is* $(\ell,\epsilon)$*-correct if for all* $\texttt{pwd},\texttt{pwd}' \in \mathcal{PWD}$ *with* $\texttt{Ham}(\texttt{pwd},\texttt{pwd}') \leq \ell$*, where* $\mathcal{PWD}$ *is the set of all passwords, and for all passive adversaries* $\mathcal{A}$ *(honest but curious) the probability that an execution of* $\pi$ *with honest players* $\mathsf{Party}_0$ *and* $\mathsf{Party}_1$ *with input passwords* pwd *and* pwd′ *respectively output different keys in the presence of* $\mathcal{A}$ *is bounded as:*

$$\Pr[K \neq K' | (K,K') \leftarrow out_{\mathcal{A},\pi}\langle \mathsf{Party}_0(\texttt{pwd}),\mathsf{Party}_1(\texttt{pwd}')\rangle] \leq \epsilon(\lambda).$$

*where* $(K,K') \leftarrow out_{\mathcal{A},\pi}\langle \mathsf{Party}_0(\texttt{pwd}),\mathsf{Party}_1(\texttt{pwd}')\rangle$ *denotes outputting* $K$ *and* $K'$ *to* $\mathsf{Party}_0$ *and* $\mathsf{Party}_1$ *respectively by executing the protocol* $\pi$*, and the probability is takes over the random coin of* $\mathsf{Party}_0$,$\mathsf{Party}_1$*, the adversary* $\mathcal{A}$ *and the protocol* $\pi$*.*

**Functionality $\mathcal{F}_{\mathsf{fPAKE}}$**

The functionality $\mathcal{F}_{\mathsf{fPAKE}}$ is parametrized by the security parameter $\lambda$ and the Hamming distance tolerance $\ell$. The ideal functionality interacts with the two players $\mathsf{Party}_0$ and $\mathsf{Party}_1$ and the adversary $\mathcal{A}$ via the following queries:

**New Session**: On input $(\textsc{NewSession}, \mathsf{sid}, \mathtt{pwd}_i, \mathsf{role}])$ from party$\mathsf{Party}_i$, where $\mathtt{pwd}_i$ is a password and $\mathsf{role} = \mathsf{sender}$ implies $\mathsf{Party}_i$ wishes to initiate a key exchange, while $\mathsf{role} = \mathsf{receiver}$ implies $\mathsf{Party}_i$ wishes to respond:

- Send $(\textsc{NewSession}, \mathsf{sid}, \mathsf{Party}_i, \mathsf{role})$ to $\mathcal{A}$
- If one of the following is true, record $(\mathsf{Party}_i, \mathtt{pwd}_i)$ and mark this record `fresh`.
  - This is the first NewSession query;
  - This the second NewSession query and there is a record $(\mathsf{Party}_{1-i}, \mathtt{pwd}_{1-i})$.

**Test Password**: On input $(\textsc{TestPwd}, \mathsf{sid}, \mathsf{Party}_i, \mathtt{pwd}^*)$ from the adversary $\mathcal{A}$: if there is a `fresh` record $(\mathsf{Party}_i, \mathtt{pwd}_i)$, then set $d \leftarrow \mathtt{Ham}(\mathtt{pwd}_i, \mathtt{pwd}^*)$ and do:

- If $d \leq \ell$, mark the record `compromised` and reply $\mathcal{A}$ with $(\textsc{CorrectGuess}, L_c(\mathtt{pwd}_i, \mathtt{pwd}^*))$ in which $L_c(\mathtt{pwd}_i, \mathtt{pwd}^*) = \{j \text{ s.t. } \mathtt{pwd}_i[j] = \mathtt{pwd}^*[j]\}$.
- If $d > \ell$, mark the record `interrupted` and reply to $\mathcal{A}$ with $(\textsc{WrongGuess})$.

**New Key**: On $(\textsc{NewKey}, \mathsf{sid}, \mathsf{Party}_i, K)$ from the adversary $\mathcal{A}$, if there is not record of the form $(\mathsf{Party}_i, \mathtt{pwd}_i)$, or if this is not the first NewKey query for $\mathsf{Party}_i$, then ignore this query. Otherwise:

- If at least one of the following is true, then output $(\mathsf{sid}, K)$ to player $\mathsf{Party}_i$:
  - The record is `compromised`;
  - The record is `fresh`, $\mathsf{Party}_{1-i}$ is corrupted, and there is a record $(\mathsf{Party}_{1-i}, \mathtt{pwd}_{1-i})$ with $\mathtt{Ham}(\mathtt{pwd}_i, \mathtt{pwd}_{1-i}) \leq \ell$.
- If this record is `fresh`, both parties are honest, there is a record $(\mathsf{Party}_{1-i}, \mathtt{pwd}_{1-i})$ with $\mathtt{Ham}(\mathtt{pwd}_i, \mathtt{pwd}_{1-i}) \leq \ell$, a key $K'$ was sent to $\mathsf{Party}_{1-i}$, and $(\mathsf{Party}_{1-i}, \mathtt{pwd}_{1-i})$ was fresh at the time, then output $(\mathsf{sid}, K')$ to $\mathsf{Party}_i$.
- In any other case, pick a new random key $K'$ of length $\lambda$ and send $(\mathsf{sid}, K')$ to $\mathsf{Party}_i$.
- Mark the record $(\mathsf{Party}_i, \mathtt{pwd}_i)$ as `completed`.

Figure 1: Ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$ for fPAKE [16].

### 4.2 fPAKE Construction using RRSS

Our fPAKE protocol is described in Figure 2. The protocol closely follows fPAKE protocol of Bootle et al. [5][3]. The differences between our construction and [5] are highlighted in blue. For completeness we also include the original fPAKE construction of Bootle

[3] The original (relaxed) fPAKE protocol of [16] contained a flaw that was later corrected in [5]; this issue was independent of the underlying secret sharing mechanism. The corrected protocol of [5] was proven UC-secure, but only with respect to a weakened ideal functionality that necessarily permits leakage when the Hamming Distance is at most $t_2$—even if it exceeds $t_1$, so that key exchange fails.

et al. [5] in Appendix E — see Figure 11. The primary difference is that we replace the random encoding scheme with our Random Robust Secret Sharing scheme[4].

As in Bootle et al.'s (relaxed) fPAKE protocol, we use an iPAKE protocol $\pi$ as a building block. The iPAKE protocol $\pi$ must UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{iPAKE}}$ [5,16]. For completeness we provide the full $\mathcal{F}_{\mathsf{iPAKE}}$ functionality in Figure 6 in Appendix C.3. An iPAKE is like a regular PAKE protocol except that one party (sender) picks the key $K$. Intuitively[5], the ideal functionality $\mathcal{F}_{\mathsf{iPAKE}}$ guarantees that if the password provided by the second party (receiver) matches the sender's password exactly then the receiver will receive the key $K$; otherwise the ideal functionality sends a uniformly random key $K'$ (unrelated to $K$) to the receiver. As in [5] we also use the split authentication functionality $\mathcal{F}_{\mathsf{SA}}$ of Barak et al. [2], recalled in Figure 7 in Appendix C.4, to model the ability of an attacker to run two separate executions of an authentication scheme (one with each party) without the two parties realizing it. The attacker may choose to actively launch a man-in-the-middle attack and execute separate protocols with each party or the attacker may choose to passively observe the messages exchanged (delaying message delivery if/when s/he chooses). However, if the attacker chooses to actively launch a man-in-the-middle attack the $\mathcal{F}_{\mathsf{SA}}$ functionality prevents the attacker from forwarding messages between separate protocol instances i.e., the attacker must separately interact with each party. The functionality $\mathcal{F}_{\mathsf{SA}}$ can be realized using digital signatures without requiring a public-key infrastructure.

The key idea behind our construction (and [5,16]) is to execute the iPAKE protocol $n$ times — one time for each character in the password. Intuitively, after the $i$th iPAKE execution the receiver will either receive the $i$th key $K_i$ selected by the sender if the two passwords match on character $i$; otherwise the receiver will get a random unrelated key $K'_i$. After the iPAKE protocols are complete the sender will pick a fresh string $s$ and generate $n$ shares $[[z]]_1,...,[[z]]_n$ of $s$ using a $(n,n-\ell,\epsilon(\lambda))$-Random Robust Secret Sharing Scheme[6]. The sender will use each key $K_i$ as a one-time-pad and send $E_1,...,E_n$ to the receiver where $E_i = [[z]]_i \oplus K_i$. Intuitively, if the $i$th character of the passwords matched matched then the receiver will recover $E_i \oplus K_i = [[z]]_i$; and if the passwords did not match on the ith character then the receiver will recover $E_i \oplus K'_i = [[z']]_i$ where $[[z']]_i$ is uniformly random because $K'_i$ is uniformly random. If the passwords matched on at least $n-\ell$ characters then, by $(n-\ell)$ random robustness, the receiver will be able to efficiently identify the correct shares and recover $s$. The final session key $K = \mathcal{H}((\mathsf{sid},\mathsf{sid}_{\mathbb{H}}),s)$ is obtained by hashing $s$ along with the session ids $\mathsf{sid}$ and $\mathsf{sid}_{\mathbb{H}}$. By

[4] The protocol of [5] also required an extra hash value which was necessary due to their use of list decoding during secret recovery — the extra hash value is "helper information" used to identify/validate the correct codeword from a list of possibilities. Because we do not perform list decoding this extra helper information is unnecessary for our protocol and can be eliminated which simplifies our security analysis.

[5] We are oversimplifying a bit. If the adversary guesses the password (or corrupts a participant) then the adversary is allowed to modify the session key $K$ that is delivered. It is also possible that the adversary delays delivery of network messages indefinitely so that the receiver never actually receives $K$.

[6] We take advantage of the fact that our RRSS setup algorithm is deterministic to avoid sending the public parameters $\mathsf{pp}$ and avoid forcing the receiver to verify that $\mathsf{pp}$ was generated correctly.

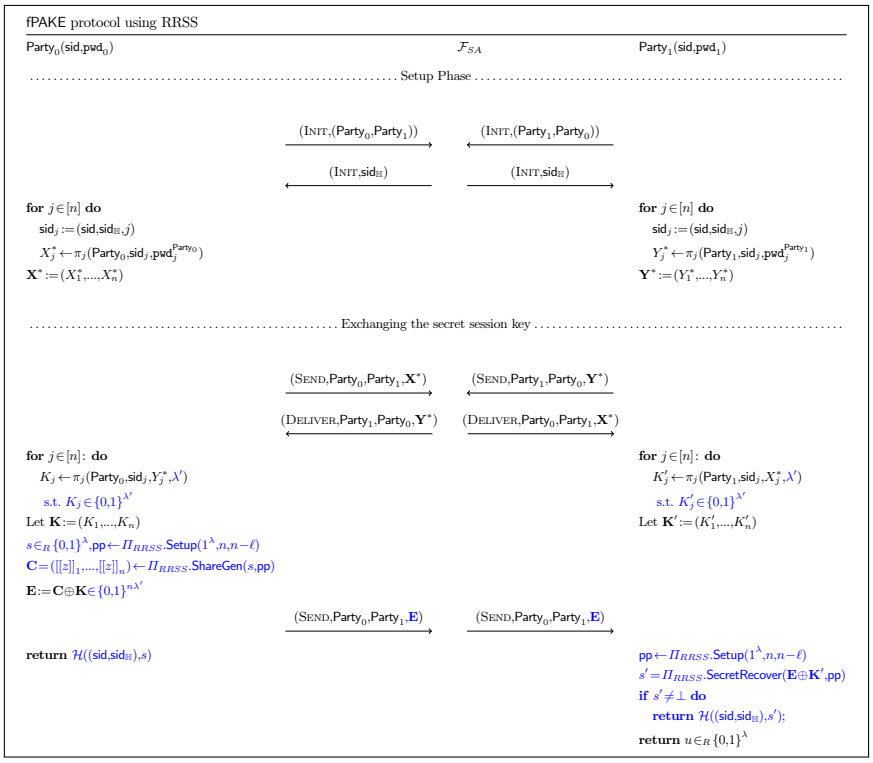



Figure 2: fPAKE protocol from our RRSS.

including the session id $\mathsf{sid}_{\mathbb{H}}$ from $\mathcal{F}_{\mathsf{SA}}$ we ensure that if the attacker tampers with any iPAKE execution then both parties will obtain separate/random keys. This is crucial to protect against an attacker who attempts to guess a single character of the password by actively interfering in the corresponding iPAKE execution — see Remark 3.

If the passwords match on fewer than $n-\ell$ characters then, by $n-\ell-1$-perfect privacy, the receiver will only learn that the passwords were not sufficiently close. By $n-\ell-1$ perfect privacy the correct shares (when the passwords matched at character $i$) will be perfectly indistinguishable from the uniformly random shares that were received (when the passwords did not match at character $i$). Thus, an attacker won't learn which characters of the password matched nor will the attacker learn how many characters were matched overall. At the end of the protocol the sender does not know for certain whether or not the receiver learned the same key. However, this is not part of the protocol or the ideal functionality fPAKE. If desired the receiver could prove that it received the correct key $K$ after the protocol e.g., by sending $\mathcal{H}(K)$ to the sender or by encrypting any message using an authenticated encryption scheme (AES-GCM) with the symmetric key $K$.

### 4.3 fPAKE Analysis

We first prove that our fPAKE is $\epsilon(\lambda)$-correct in Theorem B.1. This means that if the adversary is passive and the input passwords $pwd_0$ and $pwd_1$ for each party have Hamming Distance is at most $\ell$ then the probability that both parties output different keys is at most $\epsilon_{\mathsf{iPAKE}}(\lambda)$. We assume that our hash function $\mathcal{H}$ is a random oracle which UC-realized the ideal functionality $\mathcal{F}_{\mathsf{GRO}}$ for a Global Random Oracle — see Figure 8 in Appendix C.5. Our proof is similar to [5, Thm 1] except that the protocol of [5] requires a helper hash value to help with list decoding and a hash collision could violate the correctness guarantee of the protocol. Our correctness bound for $\epsilon(\lambda)$ does not include a term for hash collisions as we do not require a helper hash value. However, correctness could break if the RRSS identification algorithm fails which happens with probability at most $\epsilon_{\mathsf{RRSS}}(\lambda)$.

**Theorem 2.** *(fPAKE Correctness) If $\Pi_{\mathsf{RRSS}}$ is as an $(n,n-\ell,\epsilon_{\mathsf{RRSS}}(\lambda))$-random robust secret sharing, $\pi$ is $\epsilon_{\mathsf{iPAKE}}(\lambda)$-correct, the hash function $\mathcal{H}$ UC-realizes $\mathcal{F}_{\mathsf{GRO}}$ and UC-realizes $\mathcal{F}_{\mathsf{iPAKE}}$, then the protocol in Figure 2 is $(\ell,\epsilon)$-correct for $\epsilon(\lambda) = n\epsilon_{\mathsf{iPAKE}}(\lambda)+\epsilon_{\mathsf{RRSS}}(\lambda)$ where $q$ upper bounds the total number of queries to $\mathcal{H}$.*

See Appendix B.1 for the formal proof of Theorem B.1.

**Theorem 3 (fPAKE UC-Realization).** *If $\Pi_{\mathsf{RRSS}}$ is an $(n,n-\ell,\epsilon_{\mathsf{RRSS}}(\lambda))$-random robust secret sharing, the hash function $\mathcal{H}$ UC-realizes $\mathcal{F}_{\mathsf{GRO}}$, and $\pi$ is $(t_{\mathsf{iPAKE}}(\lambda),\epsilon_{\mathsf{iPAKE}}(\lambda))$-universally compossible which realizes $\mathcal{F}_{\mathsf{iPAKE}}$ using simulator $\mathtt{Sim}_{\mathsf{iPAKE}}$ in presence of the environment $\mathcal{Z}^*$ running in time at most $t_{\mathsf{iPAKE}}(\lambda)$, then the protocol in Figure 2 UC-realizes $\mathcal{F}_{\mathsf{fPAKE}}$ in the $\mathcal{F}_{\mathsf{SA}}$-hybrid model with error tolerance in password $\ell$ and leakage threshold $\gamma=\ell$.*

The proof of Theorem 3 follows the proof of [5] very closely with nearly identical hybrids. For completeness we include the proof in appendix Appendix B.2.

At a high level, we define a sequence of computationally indistinguishable hybrids. In the first hybrid, the environment interacts with the real-world execution of the protocol. In the final hybrid, the environment interacts with the ideal functionality together with a suitably defined simulator. The indistinguishability of consecutive hybrids implies that no malicious environment can distinguish between the real-world protocol and the ideal-world execution. Consequently, our protocol securely UC-realizes the target ideal functionality $\mathcal{F}_{\mathsf{fPAKE}}$.

The sixth Hybrid is where we invoke $(t-1)$-perfect privacy of an RRSS scheme. By contrast, [5] invokes $t_1$-perfect privacy of their robust secret sharing scheme in this hybrid. The key observation is the following. On the receiver side, each received share is XORed with the corresponding session key obtained from the execution of the $i$-th iPAKE instance. Since $\Pi_{\mathsf{iPAKE}}$ securely realizes the ideal functionality $\mathcal{F}_{\mathsf{iPAKE}}$, the resulting session keys are uniformly random whenever the execution does not result in a matching key. Therefore, if the session keys match, the receiver obtains valid shares. Otherwise, the received masked shares are XORed with independently uniform session keys, resulting in uniformly random (and thus randomly corrupted) shares. Assuming the passwords match on at most $t-1$ characters we obtain at most $t-1$-valid shares which cannot be distinguished from the other randomly corrupted by $(t-1)$-perfect privacy.

As a result, the final inputs to $\Pi_{\mathsf{RRSS}}.\mathsf{SecretRecover}$ are either (1) at least $t$-valid shares if the passwords are close which suffices to recover $s$ under random corruptions, or (2) at most $t-1$-valid shares if the passwords are not close. In the second case $(t-1)$-perfect privacy ensures that all shares can be viewed as uniformly random and fully independent of the secret $s$.

*Remark 3.* It is natural to wonder why we use $\mathcal{H}((\mathsf{sid},\mathsf{sid}_{\mathbb{H}}),s)$ instead of $s$ when $s$ is already uniformly random. The reason is that this prevents the attack of [5] on the fPAKE protocol of [16]. Here an attacker plays man-in-the-middle attacker but only for one of the $\mathsf{iPAKE}$ executions. Because altering a single character of the password may alter the final outcome of the $\mathsf{fPAKE}$ and this final outcome (success or failure) is visible to the attacker, the attacker can exploit this vulnerability to guess one character of the password at a time leading to a much more efficient password recovery attack. Following [5] we use the split authentication functionality [2] $\mathcal{F}_{\mathsf{SA}}$ along with the random oracle $\mathcal{H}$ to prevent this attack. If the attacker plays man-in-the-middle in *any* $\mathsf{iPAKE}$ execution then both honest parties will have different $\mathsf{sid}_{\mathbb{H}}$ resulting in different keys.

## 5 Conditional Encryption for Arbitrary Hamming Distance

As our second application, we show how to use Random Robust Secret Sharing to build an efficient conditional encryption scheme for the Hamming-distance predicate that supports *arbitrary distances*. Conditional encryption is a primitive introduced by Ameri and Blocki [1] to improve the security of the TypTop system [10] for personalized password typo correction.

In TypTop, the server stores prior failed login attempts in an encrypted "vault," using keys derived from the user's password, so that only someone who knows the correct password (or an allowed typo) can decrypt stored attempts and identify plausible typos (e.g., those within small Hamming distance of the true password). However, this creates a security concern: if an attacker compromises the TypTop system and cracks the user's password by brute-force, then the attacker can decrypt *all* stored failed attempts—even those that are not plausible typos. These failed attempts may include sensitive strings, such as the user's passwords for other accounts. Ideally, the server would store a failed login attempt in recoverable form *only if* it is a plausible typo of the true password (e.g., within a threshold Hamming distance). The difficulty is that the server must make this "store-or-not" decision *without knowing the true password*.

Conditional encryption resolves this tension by allowing the server to encrypt a failed attempt conditioned on a predicate involving the unknown password. Concretely, the server can produce a conditional ciphertext of a failed attempt (pwd') such that decryption reveals (pwd') only if (pwd') is within the allowed Hamming distance of the user's true password (pwd); otherwise, even a party with the decryption key learns nothing about (pwd'). In this way, the vault becomes "self-filtering": only plausible typos are ever recoverable, while non-typo failed attempts remain information-theoretically hidden from a later attacker who manages to guess the user's password.

Our conditional encryption construction closely follows the construction of Ameri and Blocki [1], who use Paillier encryption, Shamir secret sharing, and authenticated encryption to design a secure conditional encryption scheme for the Hamming-distance

predicate. However, their conditional decryption algorithm runs in time $\binom{n}{\ell}$, where $n$ is the message length and $\ell$ is the maximum Hamming-distance threshold, making the construction quite inefficient for larger values of $\ell$. At a high level, the inefficiency arises because the decryption algorithm obtains $n$ Shamir shares, but only $n-\ell$ of these shares are guaranteed to be correct while the remaining shares are effectively random. Since the algorithm does not know *a priori* which of the $n$ shares are correct, it must search over all $\binom{n}{\ell}$ possible subsets (equivalently, all candidate sets of $\ell$ potentially corrupted positions) in order to identify a consistent subset of shares that can be used for reconstruction. Our key insight is that we can replace Shamir secret sharing with our Random Robust Secret Sharing (RRSS) scheme. By leveraging robustness against random share corruptions, the decryptor can reconstruct the secret with high probability directly from the full set of $n$ shares, eliminating the need for exhaustive subset enumeration and thereby achieving polynomial-time decryption even for large $\ell$.

### 5.1 Review: Pallier Encryption

The Paillier cryptosystem is a partially homomorphic cryptographic scheme which supports ciphertext addition, plaintext to ciphertext multiplication and subtraction. Specifically, the public key $pk=(N,g)$ (resp. secret key $sk=(\beta,\mu)$) consists of $N=pq$ where $p,q$ are prime numbers and the number $g=N+1\in\mathbb{Z}^*_{N^2}$ (resp. $\beta=\mathsf{lcm}(p-1,q-1)$ and $\mu=\varphi(N)^{-1} \bmod N$). The secret key $sk=(\beta,\mu)$ consists of two parameters $\beta=\mathsf{lcm}(p-1,q-1)$ and $\mu=\varphi(N)^{-1} \bmod N$ is defined to be the multiplicative inverse of $\varphi(N)=(p-1)(q-1)$ modulo $N$.

The algorithm $\mathsf{Enc}_{pk}(m;r)$ takes as input a message $m\in\mathbb{Z}_N$ and a nonce $r\in\mathbb{Z}^*_N$ and outputs $g^m r^N \mod N^2$ as the corresponding ciphertext. The function $\mathsf{Enc}_{pk}$ acts as a bijective map from $\mathbb{Z}_N\times\mathbb{Z}^*_N\to\mathbb{Z}^*_{N^2}$. In particular, for *every* $c\in\mathbb{Z}^*_{N^2}$ there is a message $m\in\mathbb{Z}_N$ and a nonce $r\in\mathbb{Z}^*_N$ such that $c=g^m r^N \bmod N^2$ [22].

The encryption scheme has several homomorphic properties in particular if $c_1=g^{m_1}r_1^N \mod N^2$ and $c_2=g^{m_2}r_2^N \bmod N^2$ encrypt message $m_1,m_2\in\mathbb{Z}_N$ respectively then $c_1c_2=g^{m_1+m_2}(r_1r_2)^N \bmod N^2$ encrypts the message $m_1+m_2 \bmod N$. Similarly, if $c=g^m r^N \bmod N^2$ encrypts the message $m$ then $c^k=g^{mk}(r^k)^N \bmod N^2$ encrypts the message $mk \bmod N$. See Appendix C.1 for a full description of the Paillier encryption scheme.

When we apply the Paillier cryptosystem, our desired message space $\mathcal{M}$ is typically not the set of integers $\mathbb{Z}_N$. Thus, we assume that there is an injective map $\mathsf{ToInt}:\mathcal{M}\to\mathbb{N}$ and $\mathsf{ToInt}^{-1}:\mathbb{N}\to\mathcal{M}$. We will also assume that $|\mathcal{M}|\le N$ and that $\forall m\in\mathcal{M}$ that $0\le\mathsf{ToInt}(m)<|\mathcal{M}|\le N$. Given $x\in\mathbb{Z}_N$ we define $\mathsf{ToInt}^{-1}(x)=\perp$ if $x$ has no preimage i.e., $\forall m\in\mathcal{M}$ we have $\mathsf{ToInt}(m)\neq x$.

### 5.2 Review: Conditional Encryption

A conditional encryption scheme $\Pi=(\mathsf{KeyGen},\mathsf{Enc},\mathsf{Dec},\mathsf{CEnc})$ for a binary predicate $P(\cdot,\cdot)$ consists of four polynomial time algorithms. In comparison to the traditional public key encryption schemes, a conditional encryption scheme contains similar algorithms with the addition of a special algorithm $\mathsf{CEnc}_{pk}$ called conditional encryption. This algorithm takes as inputs: a ciphertext $c=\mathsf{Enc}_{pk}(m_1;r)\in\mathcal{C}_{\mathsf{Enc}}$ (Where $\mathcal{C}_{\mathsf{Enc}}$ is the

ciphertext space of traditional encryption scheme) encrypting some unknown message $m_1 \in \mathcal{M}$ in our message space using random coin $r \in_R \{0,1\}^\lambda$, a control message $m_2$ and a payload message $m_3$. A conditional encryption scheme is defined with respect to a binary predicate $P: \mathcal{M} \times \mathcal{M} \to \{0,1\}$. Intuitively, if $P(m_1,m_2)=1$, then $\mathsf{CEnc}_{pk}(c,m_2,m_3) \in \mathcal{C}_{\mathsf{CEnc}}$[7] should produce valid encryption of our payload message $m_3$; otherwise, if $P(m_1,m_2)=0$ the output should reveal *no information* about any of the messages $m_1,m_2$ or $m_3$ — even to an adversary that knows the secret decryption key $sk$. This property is formalized by the requirement that if $P(m_1,m_2)=0$ then $\mathsf{CEnc}_{pk}(c,m_2,m_3)$ is indistinguishable from a simulated ciphertext generated by a simulator who only knows the public $pk$. In fact, the primary property that distinguishes conditional encryption schemes from regular encryption schemes is the requirement that the scheme satisfies *conditional encryption secrecy*. Intuitively, if messages $m_1$ and $m_2$ are unrelated ($P(m_1,m_2)=0$) and $c_1 = \mathsf{Enc}_{pk}(m_1)$ then for payload message $m_3$ the conditional ciphertext $\mathsf{CEnc}_{pk}(c_1,m_2,m_3)$ should leak no information about any of the messages $m_1,m_2$ or $m_3$ even if the attacker has the secret decryption key $sk$. Ameri and Blocki [1] formalized this intuition by requiring that there is a simulator $\mathtt{Sim}(pk)$ whose outputs are statistically indistinguishable from $\mathsf{CEnc}_{pk}(c_1,m_2,m_3)$ whenever $c_1 = \mathsf{Enc}_{pk}(m_1)$ and the predicate $P(m_1,m_2)=0$ does not hold. See Appendix D for the formal definition of conditional encryption along with the security requirements.

**5.2.1 Review: Equality Predicate Construction.** We start by quick review of Ameri and Blocki's [1] semi-honest conditional scheme for the equality predicate $P_=(m_1,m_2)=1$ if and only if $m_1=m_2$. We review this construction as a warmup because it it a key building block in the Hamming Distance constructions. Note that an honest Pallier ciphertext $c = \mathsf{P.Enc}_{pk=N}(m_1;r)$ for the message $m_1 \in \mathbb{Z}_N$ takes the form $c=(1+N)^{m_1} r^N \mod N^2$ where $r \in \mathbb{Z}_N^*$ is a random nonce. The conditional encryption algorithm $\mathsf{CEnc}_{pk}(c,m_2,m_3)$ returns $(1,c')$ where the flag bit 1 simply indicates that this is a conditional ciphertext and $c' = (1+N)^{-Rm_2+m_3} c^R r'^N \mod N^2$ and the values $R \in_R \mathbb{Z}_N$ and $r' \in \mathbb{Z}_N^*$ are selected randomly. Intuitively, if $c=(1+N)^{m_1} r^N \mod N^2$ then we can rewrite $c' = (1+N)^{R(m_1-m_2)+m_3}(rr')^N \mod N^2$. If $m_1=m_2$, then $c' = (1+N)^{m_3}(rr')^N \mod N^2$ is a valid Pallier Encryption of $m_3$; otherwise $c'$ can be viewed as a uniformly random sample from the ciphertext space $\mathbb{Z}_{N^2}^*$ as long as $|m_1 - m_2| < \min\{p,q\}$ and $\gcd(\phi(N)=(p-1)(q-1),N)=1$. An honestly generated public key $N=pq$ will satisfy the properties that $\gcd((p-1)(q-1),N)=1$ and $\min\{p,q\} \gg |\Sigma|^n$. In particular, for any messages $m_1 \leq |\Sigma|^n$ and $m_2 \leq |\Sigma|^n$ in our message space, we will have $|m_1 - m_2| \leq |\Sigma|^n < \min\{p,q\}$. Thus, to prove conditional encryption secrecy, the simulator $\mathtt{Sim}(N)$ can simply output a uniformly random sample from $\mathbb{Z}_{N^2}^*$.

**5.2.2 Review of Hamming Distance Construction [1]** The Hamming Distance construction of [1] utilized Shamir Secret Sharing, Authenticated Encryption and their (semi-honest) conditional encryption scheme for the equality predicate. The regular encryption algorithm $\mathsf{Enc}_{pk}(m)$ takes as input a message $m \in \Sigma^n$ splits $m$ into $n$ individual characters $m[1],\ldots,m[n]$ and encrypts them one by one to obtain the output ciphertext $c=(c[1],\ldots,c[n])$. Here, $c[i] = (N+1)^{m[i]} r_i^N \mod N^2$ for a random nonce $r_i \in \mathbb{Z}_N^*$. Given $c,m_2$ and $m_3$ as input, the conditional encryption algorithm $\mathsf{CEnc}_{pk}$

[7] Similarly, we define $\mathcal{C}_{\mathsf{CEnc}}$ as the ciphertext space for a conditional encryption scheme.

first picks a random symmetric key $K \in \{0,1\}^\lambda$ and encrypts the payload message $m_3$ with the secret key $K \in \{0,1\}^\lambda$, i.e., $c_{AE} = \mathsf{AuthEnc}_K(m_3)$. The algorithm then uses Shamir Secret Sharing to generate $n$ shares $[[s]]_1,...,[[s]]_n \in \mathbb{Z}_{2^\lambda}$ of the secret key $K$. Each share $[[s]]_i$ is then encoded as a random integer $\hat{s}_i \in \mathbb{Z}_N$ subject to the constraint that $\hat{s}_i = [[s]]_i \bmod 2^\lambda$. Finally, for each character $i$ we generate a conditional ciphertext $c'[i] = (N+1)^{-Rm_2[i]+\hat{s}_i} c[i]^R r_i'^N \bmod N^2$. The output is $(1, c' = (c'[1],...,c'[n]), c_{AE})$. Intuitively, if $m[i] = m_2[i]$ then we can recover the correct share $[[s]]_i$ from $c'[i]$. If the Hamming Distance is at most $\ell$ then we will recover $n-\ell$ shares which is enough to recover $K$ and decrypt $c_{AE}$. However, because we don't know a prior which shares are (in)valid the conditional decryption algorithm will have to enumerate over all $\Omega\left(\binom{n}{\ell}\right)$ subsets of $n-\ell$ shares. The proof of conditional encryption secrecy crucially relied on the observation that any subset of $n-\ell-1$ Shamir Secret Shares can be viewed as independent uniformly random samples from our finite field of size $2^\lambda$. The proof also critically relied on the observation that the random encoding $\hat{s}_i$ of a random share $[[s]]_i$ is statistically indistinguishable from a random sample from $\mathbb{Z}_N$. Taken together these properties ensure that if the predicate does not hold $\mathtt{Ham}(m,m_2) > \ell$, that, even if the strings match at character $i$ ($m[i] = m_2[i]$), we still cannot distinguish $c[i]$ from a uniformly random ciphertext in $\mathbb{Z}^*_{N^2}$. Thus, the simulator can simulate each $c[i]$ by simply picking a uniformly random ciphertext in $\mathbb{Z}^*_{N^2}$. For completeness we include a full description of the Hamming Distance construction from [1] in **Construction** 6 included in Appendix E.

### 5.3 Efficient Scheme for Arbitrary Hamming Distance

**5.3.1 High Level Overview** Our modified construction follows a similar approach as Ameri and Blocki [1]. The key difference is that the conditional encryption algorithm uses $(n-\ell, n, \epsilon(\lambda))$-Random Robust Secret Sharing to generate $n$ shares $[[s]]_1,...,[[s]]_n \in \mathbb{Z}_{2^\lambda}$ of the secret key $K$. Because the RRSS share $[[s]]_i$ can be a bit larger it may be necessary to split the string $[[s]]_i$ into smaller pieces $s_{i,1},...,s_{i,\mathsf{d_{Pack}}}$ each of which can be encoded as a random integer $\hat{s}_{i,j}$ in $\mathbb{Z}_N$ subject to the constraint that $\hat{s}_{i,j} = s_{i,j} \bmod N$. In particular, for each character $i$ we will generate $\mathsf{d_{Pack}}$ conditional ciphertexts $c'[i,j] = (N+1)^{-R_{i,j} m_2[i] + \hat{s}_{i,j}} c[i]^{R_{i,j}} r'^N_{i,j} \bmod N^2$. The final output includes $c'[i,j]$ for each $i \le n$ and $j \le \mathsf{d_{Pack}}$. Intuitively, if $m[i] = m_2[i]$ then for each $j \le \mathsf{d_{Pack}}$ the ciphertext $c'[i,j]$ will decrypt to $\hat{s}_{i,j}$ allowing us to reconstruct the original share $[[s]]_i$. On the other hand if $m[i] \neq m_2[i]$ then we will essentially end up constructing a uniformly random share $[[s']]_i$. Thus, if $\mathtt{Ham}(m,m_2) \le \ell$ (the Hamming Distance between $m$ and $m_2$ is at most $\ell$) then we will recover $n-\ell$ correct shares with $\ell$ random shares mixed in. We can then utilize the $\mathsf{Identify}$ algorithm from our RRSS construction to find $n-\ell$ correct shares and recover the secret key $K$. Thus, the simulator can simulate a conditional ciphertext by simply picking $c'[i,j] \in_R \mathbb{Z}^*_{N^2}$ for each $i \le n$ and $j \le \mathsf{d_{Pack}}$. On the other hand if $\mathtt{Ham}(m,m_2) > \ell$ (the predicate does not hold) we can appeal to $n-\ell-1$ perfect privacy to argue the uncorrupted shares are uniformly distributed.

**5.3.2 Construction** To obtain a more efficient construction for arbitrary $\ell$, we use $(n,t,\epsilon(\lambda))$-RRSS scheme $\Pi_{RRSS} = (\mathsf{Setup}, \mathsf{ShareGen}, \mathsf{SecretRecover}, \mathsf{Identify})$ described in **Construction** 1. We observed that we can reduce the decryption overhead, when we use the RRSS due to $\Pi_{RRSS}.\mathsf{Identify}$ algorithm. In this section, we formally

describe our generic construction of conditional encryption for *arbitrary* Hamming distance with efficient decryption algorithm which uses our $\Pi_{RRSS}$, Paillier Encryption and authenticated encryption as the constructive building blocks. In what follows, we present a detailed description of our conditional encryption scheme $\Pi_{\mathtt{Ham},\ell}=(\mathsf{KeyGen},\mathsf{Enc},\mathsf{CEnc},\mathsf{Dec})$, designed for the Hamming distance predicate $P_{\mathtt{Ham},\ell}$ with arbitrary $\ell$.

Because the shares of a RRSS scheme can be a bit longer it is useful to split each share $[[s]]_i$ into smaller pieces and then randomly encode each piece as an integer in $\mathbb{Z}_N$. For simplicity we will assume that each share is a uniformly random $m$-bit string — for our particular construction we have $m=6n\lambda_1+\lambda$. We can split the string $[[s]]_i\in\{0,1\}^m$ into $\mathsf{d_{Pack}}$ substrings $s_1,\ldots,s_{\mathsf{d_{Pack}}}$ such that the first $\mathsf{d_{Pack}}-1$ substrings have length $|s_i|\leq\left\lceil\frac{m}{\mathsf{d_{Pack}}}\right\rceil$ and the final substring $s_{\mathsf{d_{Pack}}}\in\{0,1\}^r$ has length $r\leq\frac{m}{\mathsf{d_{Pack}}}$. We also rely on a randomized encoding procedure $\mathsf{REnc}(x,a,N)$ which takes as input integers $0\leq x<a\leq N$ and randomly picks an integer $0\leq y<N$ subject to the constraint that $y \bmod a=x$. We will slightly abuse notation interpret each $s_i\in\{0,1\}^{\lceil\frac{m}{d}\rceil}$ as an integer between 0 and $2^{\lceil\frac{m}{d}\rceil}-1$ and encode $\mathsf{REnc}(z_i,a,N)$ with $a=2^{\lceil\frac{m}{d}\rceil}-1$. Ameri and Blocki [1] observed that as long as $a\ll N$ and $z_i$ is uniformly random between 0 and $a-1$ the distribution of $\mathsf{REnc}(z_i,a,N)$ will be statistically indistinguishable from the uniform distribution over $\mathbb{Z}_N$.

- **Key Generation** $(pk',sk')\leftarrow\Pi_{\ell,\mathtt{Ham}}.\mathsf{KeyGen}(1^\lambda,n,\ell)$**:** This algorithm first runs the setup algorithm $\mathsf{pp}=(\boldsymbol{v},n,t=n-\ell,\Pi^{\lambda}_{SSS,n,t},\Pi^{\lambda_1}_{SSS,2n,2t})\leftarrow\Pi_{RRSS}.\mathsf{Setup}(1^\lambda,n,t)$. The algorithm returns $\big(pk'=(pk,\mathsf{pp}),sk'=sk\big)$.
- **Regular Encryption** $c\leftarrow\Pi_{\ell,\mathtt{Ham}}.\mathsf{Enc}_{pk'}(m)$**:** The algorithm $\Pi.\mathsf{Enc}_{pk'}(m)$ takes as input the message $m=(m[1],\ldots,m[n])\in\Sigma^n$ and public key $pk=N\in pk'$ and encrypts $m$ character by character and computes $c=(c[1],\ldots,c[n])$ in which $c[i]=\mathsf{P}.\mathsf{Enc}_{pk}(m[i];r_i)=(1+N)^{m[i]}r_i^N \bmod N^2$. The final regular encryption output is $(b=0,c[1],\ldots c[n])$.
- **Conditional Encryption** $c\leftarrow\Pi.\mathsf{CEnc}_{pk'}(c,m_2,m_3)$**:** The algorithm $\Pi_{\ell,\mathtt{Ham}}.\mathsf{CEnc}_{pk'}(c_1,m_2,m_3)$ takes as input the public key $pk'=(pk,\mathsf{pp})$, a ciphertext $c_1=\big(b=0,c_1[1],\ldots,c_1[n]\big)$ corresponding to some unknown message $m_1=\big(m_1[1],\ldots,m_1[n]\big)$, a control message $m_2=\big(m_2[1],\ldots m_2[n]\big)\in\Sigma^n$ and a payload message $m_3$. The algorithm performs the following steps to generate the ciphertext.
  1. Randomly pick the key $K\in\{0,1\}^\lambda$ and compute $c_{AE}=\mathsf{AuthEnc}_K(m_3)$.
  2. Compute $n$ shares $([[s]]_1,\ldots,[[s]]_n)\leftarrow\Pi_{RRSS}.\mathsf{ShareGen}(K,\mathsf{pp}\in pk')$.
  3. $\forall 1\leq i\leq n$:
     - Partition each $[[s]]_i\in\{0,1\}^m$ into $\mathsf{d_{Pack}}$ strings $s_{1,1},\ldots,s_{1,\mathsf{d_{Pack}}}$ such that $|s_{i,j}|=\lceil m/\mathsf{d_{Pack}}\rceil$ for $j<\mathsf{d_{Pack}}$ and $|s_{1,\mathsf{d_{Pack}}}|=m-\sum_{j=1}^{\mathsf{d_{Pack}}-1}|s_{i,j}|$.
     - Compute $\hat{s}_{i,j}=\mathsf{REnc}(s_{i,j},2^{\lceil m/\mathsf{d_{Pack}}\rceil},N)$ for all $j<\mathsf{d_{Pack}}$ and $\hat{s}_{i,\mathsf{d_{Pack}}}=\mathsf{REnc}(s_{i,\mathsf{d_{Pack}}},2^{m-\mathsf{d_{Pack}}\lceil m/\mathsf{d_{Pack}}\rceil+\lceil m/\mathsf{d_{Pack}}\rceil},N)$.
     - For each $j\leq\mathsf{d_{Pack}}$ compute $c_i^j=(N+1)^{R_{i,j}m_2[i]+\hat{s}_{i,j}}c[i]^{R_{i,j}}r_{i,j}^N \bmod N^2$. Note that if $m_2[i]=m[i]$ (the two strings match at character $i$) then $c_i^j$ will be a valid Pallier encryption of $\hat{s}_{i,j}$. Otherwise, $c_i^j$ will be a uniformly random Pallier ciphertext in $\mathbb{Z}^*_{N^2}$.
     - Set $c'[i]=(c_i^1,\ldots,c_i^{\mathsf{d_{Pack}}})$.

4. Set the final ciphertext as $(b=1,c,c_{AE})$ in which $c=\big(c'[1],...,c'[n]\big)$.

– **Decryption** $m=\Pi.\mathsf{Dec}_{sk}(c)$**:**
  - **Regular ciphertext**: Given a ciphertext $\big(b=0,c[1],...,c[n]\big)$ with $b=0$ the decryption algorithm will simply decrypt character by character to recover $m=\big(m[1],...,m[n]\big)$ where $m[i]=\mathsf{P.Dec}_{sk}(c[i])$.
  - **Conditionally generated ciphertext**: Given a conditionally encrypted ciphertext $(b=1,c[1],...,c[n],c_{AE})$ this algorithm executes the following steps:
    1. For all $\forall i\leq n$: Parse $c[i]=(c_i^1,...,c_i^{\mathsf{d_{Pack}}})$
       - $\forall j\leq \mathsf{d_{Pack}}$: compute $\hat{s}'_{i,j}=\mathsf{P.Dec}_{sk}(c_i^j)$
       - $\forall j<\mathsf{d_{Pack}}$: compute $s'_{i,j}=\hat{s}'_{i,j} \bmod 2^{\lceil m/\mathsf{d_{Pack}}\rceil}$.
       - compute $s'_{i,\mathsf{d_{Pack}}}=\hat{s}'_{i,\mathsf{d_{Pack}}} \bmod 2^{m-\mathsf{d_{Pack}}\lceil m/\mathsf{d_{Pack}}\rceil+\lceil m/\mathsf{d_{Pack}}\rceil}$.
       - Interpret each $s'_{i,j}$ as a bit string and set $[[s']]_i=s'_{i,1}\circ...\circ s'_{i,\mathsf{d_{Pack}}}$.
    2. Set $\boldsymbol{s}'=\big([[s']]_1,...,[[s']]_n\big)$ and compute $K':=\Pi_{RRSS}.\mathsf{SecretRecover}\big(\boldsymbol{s}',\mathsf{pp}\big)$.
    3. Return $\{m,\perp\}:=\mathsf{AuthDec}_{K'}(c_{AE})$.

In the remainder of the paper, we use **Construction 2** to refer to the construction of the conditional encryption scheme $\Pi_{\ell,\mathtt{Ham}}$ for the Hamming distance predicate, featuring an efficient decryption algorithm using RRSS.

We highlight that the construction is very similar to Ameri and Blocki's construction for Hamming distance [1], however, we replace the Shamir secret sharing with our RRSS. Intuitively, the decryption algorithm invokes the identification algorithm $\Pi_{\mathsf{RRSS}}.\mathsf{Identify}$ (is called inside $\Pi_{\mathsf{RRSS}}.\mathsf{SecretRecover}$) to find the location of valid shares which are places that the characters of the control message $m_2$ and the unknown message $m_1$ are matching. If we have enough matching shares, then we can apply the correctness of RRSS to argue that we we can recover the valid shares to reconstruct the authenticated encryption $K$ (is used to decrypt the payload $m_3$). If the Hamming distance is more than the threshold value, then decryption returns $\perp$ since we cannot find enough valid shares.

We argue that one trivial way (if we use traditional Shamir Secret Sharing as in [1]) is to check all possible $\binom{n}{n-\ell}$ possible subsets which may not be efficient. That is why we use our $\Pi_{RRSS}$ is $(k=n-\ell,2^{-n\lambda})$-random robust share identification (Definition 1) which inherently finds the subset of $k=n-\ell$ valid shares way faster ($O(n^\omega)$: complexity of solving a system of linear equations of size $O(n)\times O(n)$) than the brute force.

**Theorem 4 (Conditional Encryption Secrecy of Construction 2 in the semi honest setting).** *Assume that our Authenticated encryption scheme* $\Pi_{AE}=(\mathsf{AuthEnc},\mathsf{AuthDec})$ *is* $\big(t_{AE},\epsilon_{AE}(t_{AE},\lambda)\big)$*-secure for any security parameter* $\lambda$ *and any running time parameter* $t_{AE}$*. Also, let* $\Pi_{RRSS}=(\mathsf{ShareGen}_{2^\lambda},\mathsf{SecretRecover}_{2^\lambda},\mathsf{Identify}_{2^\lambda})$ *be a* $(n,k=n-\ell,\epsilon(\lambda))$*-Random Robust secret sharing scheme. Then for any* $t$ *and any security parameter* $\lambda$ ***Construction** 2 provides* $\big(t,t_{\mathtt{Sim}},\epsilon(t,\lambda)\big)$*-conditional encryption secrecy with* $\epsilon(t,\lambda)\leq\epsilon_{AE}(t',\lambda)+2^{-\lambda}$*,* $t=t_{AE}(\lambda)$ *and* $t_{\mathtt{Sim}}=n\mathsf{d_{Pack}}\cdot t_{P.\mathsf{Enc}}+\mathtt{poly}(\lambda)$*.*

We note that and without loss of generality and for simplicity in proof, we assumed $\lambda=\lambda_1$. The proof almost similar to the proof of conditional encryption for hamming distance construction in [1]. The only difference is that we use RRSS, but we can still follow similar hybrids since both Shamir and RRSS provide $t-1$ perfect privacy, which is the required property used in security proof of their construction. For the completeness we provide the full proof in Appendix D.3. We also prove that

**Construction 2** is correct (see Appendix D.4) and achieves the standard notion of Real-or-Random Security (see Appendix D.5) for a public-key encryption scheme.

## 6 Performance Evaluation and Implementation

In this section, we describe our C++ implementation of Random Robust Secret Sharing (RRSS) from **Construction 1**. We also present implementations of our fPAKE protocol and our conditional encryption scheme for arbitrary Hamming-distance predicates, both of which rely on RRSS. Our source code is available on an anonymous repository at `https://github.com/mhassanameri/RRSS/tree/main`.

### 6.1 CPP Implementation of Random Robust Secret Sharing

We implement our RRSS scheme as a modular, open-source C++ library designed for reuse in future applications and research prototypes. The implementation cleanly separates components for secret sharing, reconstruction, valid-share identification, and finite-field arithmetic.

For Shamir Secret Sharing over large finite fields, we rely on the Crypto++ library to generate the primary shares of the secret. In addition, we provide a lightweight implementation of Shamir Secret Sharing over the finite field $\mathbb{GF}(256)$ for performance-critical components of the protocol. Arithmetic over large integers and finite fields is supported via the NTL and GMP libraries, which serve as dependencies of the Crypto++ backend.

Our valid-share identification procedure is based on Gauss–Jordan elimination. We implement this algorithm over $\mathbb{GF}(256)$, carefully handling finite-field inversion and row-reduction operations to ensure both correctness and efficiency. This functionality is encapsulated in a dedicated validation module and integrated as a wrapper component within the RRSS construction.

The library is written in modern C++ with clear module boundaries and a reproducible build configuration using CMake. All experiments reported in this paper were compiled using standard optimization flags, and the implementation builds on commodity hardware without requiring specialized cryptographic accelerators. This design supports reproducibility and artifact evaluation.

Finally, we implement a serialization and binding layer that converts RRSS inputs and outputs into byte streams, enabling the compiled C++ library to be invoked directly from Python. This facilitates integration with higher-level experimental frameworks and evaluation pipelines.

### 6.2 fPAKE using Random Robust Secret Sharing

Our fPAKE implementation extends the publicly available implementation of the (relaxed) fPAKE protocol by Fomichev et al. [18]. The original implementation relies on a robust secret sharing (RSS) primitive. We replace this component with our *Random Robust Secret Sharing* (RRSS) scheme while keeping the remaining protocol structure unchanged.

By integrating RRSS, we obtain full fPAKE security (i.e., robustness and $(t-1)$-perfect privacy simultaneously), thereby closing the gap present in prior practical implementations. To the best of our knowledge, this is the first practical implementation of an fPAKE protocol achieving full fPAKE security.

**6.2.1 Experimental Methodology** We evaluate the computational and communication overhead introduced by replacing RSS with RRSS. Specifically, We compare RSS-based and RRSS-based implementations using: Computation time (local cryptographic processing), Communication time, Overall execution time, and Total communication overhead (in KiB). All other components of the protocol remain unchanged, allowing for a direct comparison between the original RSS-based construction and our RRSS-based variant.

Experiments were conducted using the same fingerprint inputs (derived from vehicle sensor data) and parameter settings as the original FastZip implementation. Fingerprints are represented as binary strings of length $n = 36$. We evaluate two sensor-derived input types and consider two security levels (128-bit and 244-bit), following the recommended parameter choices in [18].

Both protocol parties were executed locally to eliminate network variability and isolate protocol-induced overhead. We evaluated the performance our implementation of fPAKE using RRSS on a Ubuntu 24.04.3 LTS (Noble Numbat) with a 3.10 GHz 15-Core Intel® Core™ i9-9900 CPU processor and 64 GB 2666 MHz DDR4 RAM memory.

**6.2.2 Implementation Adjustments** Beyond replacing RSS with RRSS, only minimal modifications were required. The first modification was related to Share Encoding. Since RRSS shares are larger than those used in the original implementation, we apply deterministic/pseudorandom key expansion to match share size prior to encoding. The secon one was about handling communication. We adjusted handling of message to accommodate larger share sizes or communicated payloads. These modifications preserve the overall protocol structure, demonstrating that RRSS can be integrated into existing fPAKE deployments with minimal engineering effort.

**6.2.3 Results** Table 1 compares the performance of our full-security fPAKE with RSSS with the relaxed-security fPAKE [18]. We report sender-side and receiver-side computation time, communication time, overall execution time, and communication overhead. We report performance across security levels $\lambda \in \{128, 244\}$, roles (sender/receiver), and fingerprint dataset (h-bar and h-gyrW).

Our results show that while RRSS increases communication overhead due to larger share sizes and strengthened robustness guarantees, the protocol remains practically deployable in vehicular communication settings.

Despite this overhead, execution times remain well below one second across all evaluated settings, demonstrating that full-security fPAKE with RRSS remains practically deployable. Importantly, this additional cost yields strictly stronger security guarantees: unlike prior practical implementations, our construction simultaneously achieves robustness against corrupted shares and $(t-1)$-perfect privacy, thereby eliminating the tradeoff between robustness and privacy inherent in earlier RSS-based designs. Thus, our results show that full fPAKE security can be obtained in practice with modest overhead.

In particular, empirical results from the FastZip [18] study indicate that deriving sufficient fingerprint entropy requires collecting ambient sensor data over a sliding window: approximately 10 seconds. In contrast, the computational and communication overhead introduced by our RRSS-based fPAKE protocol is well below one second, which is negligible compared to the fingerprint acquisition phase. Therefore, the additional cost of providing full fPAKE functionality does not significantly affect the overall execution time, as system latency is dominated by the sensor data collection process, demonstrating that RRSS remains practically deployable.

| Algorithm | Sec. | Type | Role | Run time (calc) | Comm. time (net) | Overall time | Role comm. ovhd. (KiB) | Total comm. ovhd. (KiB) |
|---|---|---|---|---|---|---|---|---|
| **fPAKE RSS** | 128 | acc_h-bar | Sender | 0.0123 | 0.0063 | 0.0186 | 3.3008 | 5.1289 |
| | | | Receiver | 0.0485 | 0.0119 | 0.0605 | 1.8281 | 5.1289 |
| | | acc_h-gyrW | Sender | 0.0125 | 0.0065 | 0.0190 | 3.6523 | 5.6836 |
| | | | Receiver | 0.0626 | 0.0120 | 0.0746 | 2.0313 | 5.6836 |
| | 244 | acc_h-bar | Sender | 0.0351 | 0.0141 | 0.0492 | 5.0771 | 8.0303 |
| | | | Receiver | 0.0661 | 0.0289 | 0.0951 | 2.9531 | 8.0303 |
| | | acc_h-gyrW | Sender | 0.0363 | 0.0155 | 0.0518 | 5.6162 | 8.8975 |
| | | | Receiver | 0.0854 | 0.0300 | 0.1153 | 3.2813 | 8.8975 |
| **fPAKE RRSS** | 128 | acc_h-bar | Sender | 0.2498 | 0.0083 | 0.2580 | 34.2031 | 36.0312 |
| | | | Receiver | 0.0447 | 0.2495 | 0.2943 | 1.8281 | 36.0312 |
| | | acc_h-gyrW | Sender | 0.3387 | 0.0112 | 0.3499 | 41.7383 | 43.7695 |
| | | | Receiver | 0.0591 | 0.3385 | 0.3975 | 2.0313 | 43.7695 |
| | 244 | acc_h-bar | Sender | 0.2682 | 0.0166 | 0.2847 | 35.9795 | 38.9326 |
| | | | Receiver | 0.0588 | 0.2614 | 0.3202 | 2.9531 | 38.9326 |
| | | acc_h-gyrW | Sender | 0.3547 | 0.0195 | 0.3742 | 43.7021 | 46.9834 |
| | | | Receiver | 0.0740 | 0.3466 | 0.4205 | 3.2813 | 46.9834 |

Table 1: fPAKE performance comparison across security levels, roles, and input types. Password length: $n=36$

### 6.3 Conditional Encryption for Arbitrary Hamming Distance

In this paper, we present an efficient conditional encryption scheme with a decryption algorithm which gives us an efficient conditional encryption for *arbitrary* Hamming distance. We already showed that the time complexity of the decryption algorithm is asymptotically reduced to $O(n-\ell)^2$ instead of $O\left(\binom{n}{\ell}\right)$.

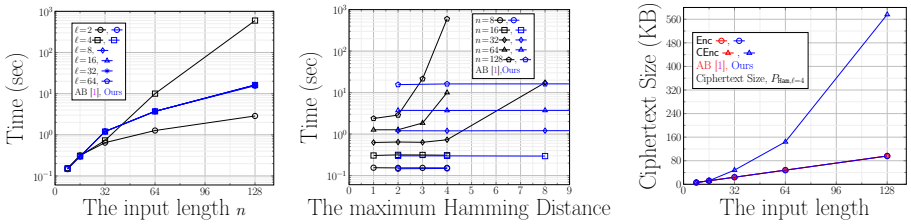
**6.3.1 Implementation** We implemented our conditional encryption schemes in C++. We implemented conditional encryption for a general Hamming distance predicate for an arbitrary value of $\ell$ (as the maximum Hamming distance). We defined our message space as $\Sigma^{\leq n}$ where $\Sigma$ denotes the set of all ASCII characters and $n \in \{8,16,32,64,128\}$ — all $x \in \Sigma^{\leq n}$ are first padded to $\texttt{Pad}(x) \in \left(\Sigma \cup \{y\}\right)^n$ for a special new symbol $y$.

We used the Pallier Library[8] as our implementation of the Pallier cryptosystem, and we used the GMP library[9] for computation with big integers. We instantiated Pallier with a 3072-bit modulus for 128-bit security [23].

[8] https://github.com/mshcruz/PaillierLibrarySamples
[9] https://gmplib.org

Our implementation of conditional encryption schemes use CryptoPP[10] library for Authenticated Encryption and Shamir Secret Sharing. For authenticated encryption, we use AES-GCM with 128 bit keys and use Shamir Secret Sharing over a field of size $2^{128}$ to generate the main shares of the secret symmetric key. We also used and modified the C++ implementation of Shamir Secret sharing in a smaller field of size $2^8 = 256$ ($\lambda_1 = 8$) based on NTL[11] library. Our code is available at our anonymized repository https://github.com/mhassanameri/RRSS/tree/main.

**6.3.2 Evaluation** We evaluated the performance our implementation of conditional encryption on a Ubuntu 24.04.3 LTS (Noble Numbat) with a 3.10 GHz 15-Core Intel® Core™ i9-9900 CPU processor and 64 GB 2666 MHz DDR4 RAM memory (the same machine used for the fPAKE application). We evaluate the performance of our conditional encryption schemes for the predicates $P_{\mathtt{Ham},\ell_1}$, and the results of these evaluations are presented in Figure 3. In particular, Figure 3b and Figure 3a evaluate the performance of our conditional encryption scheme for various Hamming distances $\ell = \{1,2,4,8,16,32,64\}$ and different padding lengths $n = \{8,16,32,64,128\}$. We highlight that for each padding length $n$ we consider Hamming distances at most $\ell_n = \{i : i \in \ell, i < n\}$. For example, for $n = 128$, we have $\ell_{128} = \{1,2,4,8,16,32,64\}$ while for $n = 16$ we have $\ell_{16} = \{1,2,4,8\}$. To show efficiency of our constructions, we compare the performance of our conditional encryption of arbitrary Hamming distance with Ameri and Blocki's conditional encryption (for simplicity, we use AB [1] to refer to their construction, specially in Figure 3). The implementation of AB- [1] currently supports Hamming distances $\ell \in \{1,2,3,4\}$ and various padding lengths $n \in \{8,16,32,64,128\}$, but was not feasible to run the conditional decryption algorithm for higher values of $\ell$.




(a) Conditional Decryption (b) Conditional Decryption (c) Reg/Cond Encryption

Figure 3: Performance Evaluation of our CE for Arbitrary Hamming Distance.

## References


1. Ameri, M.H., Blocki, J.: Conditional encryption with applications to secure personalized password typo correction. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 4643–4657. ACM Press (Oct 2024). https://doi.org/10.1145/3658644.3690374


[10] https://github.com/weidai11/cryptopp
[11] https://libntl.org/

2. Barak, B., Canetti, R., Lindell, Y., Pass, R., Rabin, T.: Secure computation without authentication. In: Shoup, V. (ed.) CRYPTO 2005. LNCS, vol. 3621, pp. 361–377. Springer, Berlin, Heidelberg (Aug 2005). https://doi.org/10.1007/11535218_22
3. Bishop, A., Pastro, V.: Robust secret sharing schemes against local adversaries. In: Cheng, C.M., Chung, K.M., Persiano, G., Yang, B.Y. (eds.) PKC 2016, Part II. LNCS, vol. 9615, pp. 327–356. Springer, Berlin, Heidelberg (Mar 2016). https://doi.org/10.1007/978-3-662-49387-8_13
4. Bishop, A., Pastro, V., Rajaraman, R., Wichs, D.: Essentially optimal robust secret sharing with maximal corruptions. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part I. LNCS, vol. 9665, pp. 58–86. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49890-3_3
5. Bootle, J., Faller, S.H., Hesse, J., Hostáková, K., Ottenhues, J.: Generalized fuzzy password-authenticated key exchange from error correcting codes. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part VIII. LNCS, vol. 14445, pp. 110–142. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8742-9_4
6. Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.D.: Universally composable password-based key exchange. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 404–421. Springer, Berlin, Heidelberg (May 2005). https://doi.org/10.1007/11426639_24
7. Canetti, R., Jain, A., Scafuro, A.: Practical UC security with a global random oracle. In: Ahn, G.J., Yung, M., Li, N. (eds.) ACM CCS 2014. pp. 597–608. ACM Press (Nov 2014). https://doi.org/10.1145/2660267.2660374
8. Carpentieri, M., De Santis, A., Vaccaro, U.: Size of shares and probability of cheating in threshold schemes. In: Helleseth, T. (ed.) EUROCRYPT'93. LNCS, vol. 765, pp. 118–125. Springer, Berlin, Heidelberg (May 1994). https://doi.org/10.1007/3-540-48285-7_10
9. Cevallos, A., Fehr, S., Ostrovsky, R., Rabani, Y.: Unconditionally-secure robust secret sharing with compact shares. In: Pointcheval, D., Johansson, T. (eds.) EUROCRYPT 2012. LNCS, vol. 7237, pp. 195–208. Springer, Berlin, Heidelberg (Apr 2012). https://doi.org/10.1007/978-3-642-29011-4_13
10. Chatterjee, R., Woodage, J., Pnueli, Y., Chowdhury, A., Ristenpart, T.: The TypTop system: Personalized typo-tolerant password checking. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 329–346. ACM Press (Oct / Nov 2017). https://doi.org/10.1145/3133956.3134000
11. Cheraghchi, M.: Nearly optimal robust secret sharing. DCC **87**(8), 1777–1796 (2019). https://doi.org/10.1007/s10623-018-0578-y
12. Cramer, R., Damgård, I., Fehr, S.: On the cost of reconstructing a secret, or VSS with optimal reconstruction phase. In: Kilian, J. (ed.) CRYPTO 2001. LNCS, vol. 2139, pp. 503–523. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_30
13. Cramer, R., Damgård, I.B., Döttling, N., Fehr, S., Spini, G.: Linear secret sharing schemes from error correcting codes and universal hash functions. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part II. LNCS, vol. 9057, pp. 313–336. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46803-6_11
14. Cramer, R., Dodis, Y., Fehr, S., Padró, C., Wichs, D.: Detection of algebraic manipulation with applications to robust secret sharing and fuzzy extractors. In: Smart, N.P. (ed.) EUROCRYPT 2008. LNCS, vol. 4965, pp. 471–488. Springer, Berlin, Heidelberg (Apr 2008). https://doi.org/10.1007/978-3-540-78967-3_27
15. Dolev, D., Dwork, C., Waarts, O., Yung, M.: Perfectly secure message transmission. In: 31st FOCS. pp. 36–45. IEEE Computer Society Press (Oct 1990). https://doi.org/10.1109/FSCS.1990.89522

16. Dupont, P.A., Hesse, J., Pointcheval, D., Reyzin, L., Yakoubov, S.: Fuzzy password-authenticated key exchange. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part III. LNCS, vol. 10822, pp. 393–424. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78372-7_13
17. Fehr, S., Yuan, C.: Towards optimal robust secret sharing with security against a rushing adversary. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 472–499. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_16
18. Fomichev, M., Hesse, J., Almon, L., Lippert, T., Han, J., Hollick, M.: Fastzip: faster and more secure zero-interaction pairing. In: Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services. p. 440–452. MobiSys '21, Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3458864.3467883, https://doi.org/10.1145/3458864.3467883
19. Garay, J.A., Givens, C., Ostrovsky, R.: Secure message transmission with small public discussion. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 177–196. Springer, Berlin, Heidelberg (May / Jun 2010). https://doi.org/10.1007/978-3-642-13190-5_9
20. Garay, J.A., Givens, C., Ostrovsky, R., Raykov, P.: Broadcast (and round) efficient verifiable secret sharing. In: Padró, C. (ed.) ICITS 13. LNCS, vol. 8317, pp. 200–219. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-04268-8_12
21. Hemenway, B., Ostrovsky, R.: Efficient robust secret sharing from expander graphs. Cryptography and Communications **10**(1), 79–99 (2018)
22. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: Stern, J. (ed.) EUROCRYPT'99. LNCS, vol. 1592, pp. 223–238. Springer, Berlin, Heidelberg (May 1999). https://doi.org/10.1007/3-540-48910-X_16
23. Polk, W.T., Dodson, D.F., Burr, W.E., Ferraiolo, H., Cooper, D.: Cryptographic algorithms and key sizes for personal identity verification. NIST Special Publication **800**(78–4), 1–19 (2015)
24. Rabin, T., Ben-Or, M.: Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In: 21st ACM STOC. pp. 73–85. ACM Press (May 1989). https://doi.org/10.1145/73007.73014
25. Roy, L., Xu, J.: A universally composable PAKE with zero communication cost - (and why it shouldn't be considered UC-secure). In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 714–743. Springer, Cham (May 2023). https://doi.org/10.1007/978-3-031-31368-4_25
26. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979)
27. Tompa, M., Woll, H.: How to share a secret with cheaters. Journal of Cryptology **1**(2), 133–138 (Jun 1988). https://doi.org/10.1007/BF02252871
28. Williams, V.V., Xu, Y., Xu, Z., Zhou, R.: New bounds for matrix multiplication: from alpha to omega. In: Woodruff, D.P. (ed.) 35th SODA. pp. 3792–3835. ACM-SIAM (Jan 2024). https://doi.org/10.1137/1.9781611977912.134

## A RRSS Security Proof

In this section, we provide the formal proofs of our RRSS $\Pi_{\mathsf{RRSS}}$. While the protocol $\Pi_{\mathsf{RRSS}}$ was described completely in the main body, we describe it again here in the appendix (see **Construction** 1) in nicely formatted algorithmic environment for convenience.

**Construction 1**

**Setup:** $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n, t)$:

1. Select $\lambda_1 \geq \lceil \frac{\lambda}{n} \rceil$, set $\Pi^\lambda_{SSS,n,t} = (\mathsf{ShareGen}_{2^\lambda}, \mathsf{SecretRecover}_{2^\lambda})$, $\Pi^{\lambda_1}_{SSS,2n,2t} = (\mathsf{ShareGen}_{2^{\lambda_1}}, \mathsf{SecretRecover}_{2^{\lambda_1}})$
2. set $\boldsymbol{a} = (a_1, ..., a_{3n}) \in_R \mathbb{F}^{3n}_{2^{\lambda_1}} \setminus \{0\}$
3. return $\mathsf{pp} = (\boldsymbol{a}, n, t, \Pi^\lambda_{SSS,n,t}, \Pi^{\lambda_1}_{SSS,2n,2t})$

**Share Generation:** $\left([[z]]_1, ..., [[z]]_n\right) \leftarrow \mathsf{ShareGen}(s, \mathsf{pp})$:

1. parse $\boldsymbol{a} = (a_1, ..., a_{3n}) \in \mathsf{pp}$.
2. $([[s]]_1, ..., [[s]]_n) \leftarrow \Pi^\lambda_{SSS,n,t}.\mathsf{ShareGen}_{2^\lambda}(n, t, s)$
3. $\forall j \in [3n]$: compute $[[a]]^j_1, ..., [[a]]^j_{2n} \leftarrow \Pi^{\lambda_1}_{SSS,2n,2t}.\mathsf{ShareGen}_{2^{\lambda_1}}(2n, 2t, a_j)$.
4. $\forall i \in [n]$: Set $[[z]]_i = \left([[s]]_i, [[a]]^1_{2i-1}, ..., [[a]]^{3n}_{2i-1}, [[a]]^1_{2i}, ..., [[a]]^{3n}_{2i}\right)$.
5. return $\left(\left([[z]]_1 ..., [[z]]_n\right)\right)$

**Recovery from Valid Shares:** $s := \mathsf{SecretRecover}(S, \boldsymbol{z}_S)$:

1. parse $\boldsymbol{z}_S$ to recover $\left([[z]]_i\right)_{\forall i \in S}$
2. $\forall i \in S$ extract $[[s]]_i$ from $[[z]]_i = \left([[s]]_i, [[a]]^1_{2i-1}, ..., [[a]]^{3n}_{2i-1}, [[a]]^1_{2i}, ..., [[a]]^{3n}_{2i}\right)$. Let $\boldsymbol{s}_S = \left([[s]]_i\right)_{i \in S}$.
3. set $\boldsymbol{s}_S = \left([[s]]_i\right)_{i \in S}$
4. return $s = \Pi^\lambda_{SSS,n,t}.\mathsf{SecretRecover}_{2^\lambda}(S, \boldsymbol{s}_S)$

**Valid Share Identification:** $S := \mathsf{Identify}\left(\left([[z]]_1 ..., [[z]]_n\right), \mathsf{pp}\right)$ with $S \in 2^{[n]} \cup \{\perp\}$:

1. parse $\boldsymbol{a} = (a_1, ..., a_{3n}) \in \mathsf{pp}$
2. for all $i \in [n]$ parse: $[[z]]_i = \left(s_i, [[a]]^1_{2i-1}, ..., [[a]]^{3n}_{2i-1}, [[a]]^1_{2i}, ..., [[a]]^{3n}_{2i}\right)$
3. Form the system of linear equations $(\boldsymbol{a})^T = AX^T$ in which $A = \begin{bmatrix} a_{1,1} & ... & a_{1,2n} \\ ... & a_{i,j} & ... \\ a_{3n,1} & ... & a_{3n,2n} \end{bmatrix}$ where $a_{i,j} = [[a]]^i_j$ and $X = (x_1, ..., x_{2n})$ is a $2n$ unknown variable vector.
4. Solve $\boldsymbol{a} = AX$ to find the values $x_1, ..., x_{2n}$ and set $S = \{i \in [n] : x_{2i-1} \neq 0 \vee x_{2i} \neq 0\}$. If there are no solutions then return $\perp$.
5. If $|S| < t$, then return $\perp$; otherwise, return $S$ as the set of valid shares.
6. return $S$.

### A.1 Proof of Theorem 1 (Construction 1 is $(n, t, \epsilon(\lambda))$-RRSS)

To prove Theorem 1, our main result, we rely on a couple of helpful lemmas. Corollary 1 states that, except with negligible probability, all solutions $X$ to the (possibly randomly corrupted) system of linear equations will have Hamming Weight at least $n - \ell$. Corollary 1 follows Lemma 2 by applying union bounds over all possible vectors $X = (X_1, ..., X_n) \in \mathbb{Z}^n_p$. Lemma 2 upper bounds the probability that a particular fixed vector $X = (X_1, ..., X_n) \in \mathbb{Z}^n_p$ with $\mathsf{HammWeight}(X) < n - \ell$ is a solution to our system of linear equations $A_{S, \boldsymbol{r}_1, ..., \boldsymbol{r}_{|S|}} X = \boldsymbol{v}$ under random corruptions to the original matrix $A$ defined in **Construction 1**. Lemma 3 is similar to Lemma 2 except that we upper bound the probability that a fixed vector $X = (X_1, ..., X_n) \in \mathbb{Z}^n_p$ is a solution to our

system of linear equations when $X_i \neq 0$ for some index $i \in S$ which corresponds to a randomly corrupted share $\boldsymbol{r}_i$. A straightforward union bound over all possible vectors $X$ tells us that, except with negligible probability, any feasible solution to the system of linear equations will be non-zero only for non-corrupted indices. Taken together with Corollary 1 we can conclude that $NZ(X) = \{i : X_i \neq 0\}$ has size $|NZ(X)| \geq n - \ell$ and only contains valid shares (whp). The proofs for Lemma 2, Lemma 3 and Corollary 1 are postponed to the end of this section.

**Reminder of Theorem 1.** *The construction* $\Pi_{\mathsf{RRSS}} = (\mathsf{ShareGen}, \mathsf{SecretRecover}, \mathsf{Identify})$ *described in* ***Construction 1*** *is a* $(n, t, \epsilon(\lambda))$*-Random Robust Secret Sharing scheme with* $\epsilon(\lambda) \leq 2^{-\lambda}$*.* $\Pi_{\mathsf{RRSS}}$ *also achieves* $(t-1, \epsilon(\lambda))$*-Security against malicious dealers.*

**Proof of Theorem 1:** We first prove that our construction provides perfect $t-1$-perfect private, and then we show the construction provides $t$-random robustness and $t$-random robust share identification.

*Proof of* $(t-1)$*-perfect Privacy.* Let $s, s' \in \mathbb{F}_{2^\lambda}$ be two arbitrary secrets. We compute their corresponding shares $\Big(([[z]]_1, ..., [[z]]_n)\Big) \leftarrow \mathsf{ShareGen}(s, \mathsf{pp})$ and $\Big(([[z']]_1, ..., [[z']]_n)\Big) \leftarrow \mathsf{ShareGen}(s', \mathsf{pp})$. Let $S \subseteq [n]$ be any arbitrary subset with $|S| \leq t-1$. Now we need to show that two vectors $\big([[z]]_i\big)_{\forall i \in S}$ and $\big([[z']]_i\big)_{\forall i \in S}$ have identical distributions. We parse $[[z]]_i = \Big(s_i, [[a]]^1_{2i-1}, ..., [[a]]^{3n}_{2i-1}, [[a]]^1_{2i}, ..., [[a]]^{3n}_{2i}\Big)$ in which $s_i$ is the output of $t-1$-perfect private Shamir secrete sharing $\Pi^{\lambda}_{SSS,n,t}$ over $\mathbb{F}_{2^\lambda}$. This implies that the two vectors $(s_i)_{\forall i \in S}$ and $(s'_1)_{\forall i \in S}$ are identically distributed. Now we need to show the rest of these two vectors have the same distribution as well. The distinguisher knows the public vectors $\boldsymbol{a} = (a_1, ..., a_{3n}) \in \mathsf{pp}$ whose elements $a_i \in \boldsymbol{a}$ are all inputs of $\Pi^{\lambda_1}_{SSS,2n,2t}.\mathsf{ShareGen}_{2^{\lambda_1}}(2n, 2t, a_i)$. As $|S|$ is at most $t-1$ and for all $j, i \in S$, each vector $[[z]]_i$ contains 2 shares of $a_j$, we observe that we have at most $2(t-1) = 2t-2$ shares of $a_j$. Recalling the Shamir secret sharing, the shares are actually the values of a $2t$-degree polynomial and we need $2t$ points to do the Lagrange interpolation and recovering the coefficients of our target polynomial. So, if we are able to reconstruct the polynomial, the $2t-2$-perfect privacy of the Shamir secret sharing scheme will be violated. We highlight that the adversary also knows one extra point of the polynomial which is the value of $a_i$ itself beside other $2t-2$ points. So we have $2t-2+1 = 2t-1$ points. However, we still need one more point (share) to identify the $2t$-degree polynomial. This implies that we can still recall the $2t-2$-perfect private property of $\Pi^{\lambda_1}_{SSS,2n,2t}$ and the corresponding shares of $a_i$ and $a'_i$ (an arbitrary secret chosen uniformly at random) are distributed identically. This tells us that all parts of the two vectors, i.e., $\big([[z]]_i\big)_{\forall i \in S}$ and $\big([[z']]_i\big)_{\forall i \in S}$ are both distributed equally.

*t-Random Robustness.* In this part of the proof, we will show that our $\Pi_{RRSS}$ construction provides $t$-Random Corrupted Share Identification which means that we can identify the valid shares with probability at least $1 - \epsilon(\lambda) = 1 - 2^{-n\lambda}$. Intuitively, we need to prove that if we have at least $t$ valid shares in the given vector of $n$ shares $\Big(([[z]]_1, ..., [[z]]_n)\Big)$, our reconstruction algorithm will *correctly* identify the valid shares required for $\Pi^{\lambda}_{SSS,n,t}.\mathsf{SecretRecover}_{2^\lambda}$ to recover the main secret $s$. Once we

identify the valid shares, we can recover the secret since our $\Pi^{\lambda}_{SSS,n,t}$ is an $(n,t)$-Shamir secret sharing and requires $t$ valid shares to recover the secret $s$ by executing $\Pi^{\lambda}_{SSS,n,t}.\mathsf{SecretRecover}_{2^{\lambda}}(\cdot)$ over the $t$ valid shares as input. In fact, we can use these valid shares to reconstruct the coefficients of the underlying $t$-degree polynomial to recover the secret. We show that by forming and solving the system of linear equations we can identify the valid shares with high probability.

More formally, let $[[z]]_i = \left(s_i,[[a]]^1_{2i-1},...,[[a]]^{2n}_{2i-1},[[a]]^1_{2i},...,[[a]]^{2n}_{2i}\right)$ for all $i \in [n]$ as secret shares and $\boldsymbol{a} = (a_1,\dots,a_{3n}) \in \mathsf{pp}$ as part of the known public parameters. Based on what we described in the reconstruction algorithm $\Pi_{RRSS}.\mathsf{SecretRecover}$, we obtain the system of linear equations $(\boldsymbol{a})^T = AX^T$ and solve it to identify the indexes $S = \{i \in [2n] : x_i \neq 0\}$. We argue that if we have more than $t$ valid shares, then $|S| \geq 2t$ with high probability and we can obtain the set $S' \subseteq S'' = \{i : i = \lceil j/2 \rceil, \forall j \in S\}$ such that $|S'| = t$ as the indexes of the valid shares. We note that, if $|S| \geq 2t$, then we have $|S''| \geq t$. More formally, if we prove the following two claims, then we can safely use the set $Z_{S'} = (s_i)_{\forall i \in S'}$ to recover $s = \Pi^{\lambda}_{SSS,n,t}.\mathsf{SecretRecover}_{2^{\lambda}}(Z_{S'})$ correctly. In the following, we describe and then prove these two claims.

- **Claim 1(Recovering Enough Shares):** The probability of $|S| < 2t$ is negligible (at most $2^{-3n\lambda_1}$). To support this argument, we use the results of Lemma 2 when $m = 3n$ and finite field of size $2^{\lambda_1}$, and we have $\Pr\left[|S| < 2t\right] < \left(\frac{1}{2^{\lambda_1}}\right)^{3n} = 2^{-3n\lambda_1}$. Intuitively, Lemma 2 implies that the probability that we can solve the system of equation such the for $S$ corresponding to the solution has $|S| < 2t$ is negligible when we have at most $2n-2t$ random corruptions on the columns of $A$. We note that the random corruption on the $i$-th share is equivalent to the case that the $i$-th column of matrix $A$ is replaced with uniformly random vector.
- **Claim 2 (Excluding Corrupted Shares)**: The probability that $(S \cap S_{\mathsf{Corrupt}}) \neq \varnothing$ is negligible in which $S_{\mathsf{Corrupt}}$ is the set of corrupted shares indexes. To support this argument we use the results of Corollary 1 (which is basically proved by applying Lemma 3 and taking union bound over all possible values of $X$) when $m' = 2n$ and $n' = 3n$ and finite field of size $O(2^{\lambda_1})$, we have $\Pr\left[|S \cap S_{\mathsf{Corrupt}}| \neq 0\right] \leq \left(\frac{1}{2^{\lambda_1}}\right)^{2n-3n} = 2^{-n\lambda_1}$. Intuitively, we observe that the index of the randomly corrupted column cannot be non-zero and with high probability we are not selecting the index of the corrupted shares in $S$.

We note that **Claim 1** implies that the set $S$ contains at least indexes of $2t$ shares and **Claim 2** tells us that non of these indexes related to the random corrupted shares. So, the set $S'$ identifies $t$ valid shares $Z_{S'}$ of our Shamir secret sharing over the finite field of size $2^{\lambda}$ (Used to share the target secret $s$.), and its correctness ensures that we can reconstruct the shared secret $s$.

Putting the results of these two claims together, we can argue that the probability that we identify the correct shares is lower bounded to at least $1 - \max\left(2^{-3n\lambda_1}, 2^{-n\lambda_1}\right) = 1 - 2^{n\lambda_1}$. Setting $\epsilon(\lambda) = 2^{-n\lambda_1}$ we observe that our $\Pi_{RRSS}$ construction provides $t$-random corrupted share identification.

*Correctness.* To prove that our construction provides this property, we need to show that if we are given $|S| \geq t$ valid shares, we can recover the secret $s$ correctly with probability 1. Let $S$ contain indexes of at least $t$ valid shares (potentially can be obtained by running the $S = \mathsf{Identify}([[z]]_1,...,[[z]]_n,\mathsf{pp})$). Since the set $S$ also identifies $t$ valid shares $Z_S$ of our Shamir secret sharing over the finite field of size $2^\lambda$ (Used to construct the original shares of the target secret $s$.). Referring to the perfect correctness of our Shamir secret sharing, we can reconstruct the secret $s$ if and only if we have at least $t$ vlaid shares.

Considering both properties $t$-random robustness and $(t-1)$-perfect privacy, we observe that **Construction** 1 is a $\left(n,t,\left(\epsilon(\lambda_1) = 2^{-n\lambda_1}\right)\right)$-RRSS.

To finalize the proof of this theorem, we also need to show that **Construction** 1 also provides $((t-1),\epsilon(\lambda_1))$-Security against malicious dealers. The proof of this part is discussed in Lemma 1. □

**Lemma 1 ($(t-1,\epsilon(\lambda))$-Security against malicious dealers).** *Let $\mathbb{S}_\lambda = \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^{\lambda_1}}^{6n}$, $s \in \mathbb{D}_{2^\lambda}$ be any arbitrary secret, $\mathsf{pp} \in \Pi_{\mathsf{RRSS}}.\mathsf{Setup}(1^\lambda,n,t)$ be the public parameter and $\boldsymbol{c} = ([[z]]'_1,...,[[z]]'_n) \leftarrow \Pi_{\mathsf{RRSS}}.\mathsf{ShareGen}(s,\mathsf{pp})$ be set of valid shares. Then for any $S \subseteq [n]$ with $|S| \geq t$ and any $\boldsymbol{c'} = ([[z]]'_1,...,[[z]]'_n) \in \mathbb{S}_\lambda^n$ such that $\boldsymbol{c'}_S = \boldsymbol{c}_S$ we have:*

$$\Pr_{\boldsymbol{c''} \in_R \mathcal{D}(\mathbb{S}_\lambda,\boldsymbol{c'},S)}\left[\mathsf{Identify}(\boldsymbol{c''},\mathsf{pp}) \neq \perp \wedge \mathsf{Identify}(\boldsymbol{c''},\mathsf{pp}) \not\subseteq S\right] \leq 2^{-n\lambda_1} < 2^\lambda$$

*where the randomness is taken over the uniformly choice of $\boldsymbol{c''}_i$ for all $i \notin S$, similarly for any $S \subseteq [n]$ with $|S| \leq t-1$ we have*

$$\Pr_{\boldsymbol{c''} \in_R \mathcal{D}(\mathbb{S}_\lambda,\boldsymbol{c'},S)}\left[\mathsf{Identify}(\boldsymbol{c''},\mathsf{pp}) = \perp\right] \geq 1 - 2^{n\lambda_1} \geq 1 - 2^\lambda$$

**Proof of Lemma 1:** Here we will show that the scheme achieves security against malicious dealers. First of all, we highlight that no matter that the initial shares $\boldsymbol{c} = ([[z]]_1,...,[[z]]_n)$ were generated honestly or not, if (wlog) $[[z']]_1$ is randomly corrupted and $x_1 \neq 0$ ($x_1$ is the unknown variable defined in $\mathsf{Identify}$ algorithm), then each term $x_1[[a']]_1^j \in \mathbb{F}_{2^{\lambda_1}}$ (as part of the share $[[z']]_1$) is still uniformly random. Therefore, the probability that each individual constraint is satisfies is at most $2^{-\lambda_1}$ and the probability all $3n$ constraints are satisfied is at most $2^{-3n\lambda_1}$. We can now union bound over all $2^{2n\lambda_1}$ possible variable assignments to conclude that, except with probability $2^{-n\lambda_1} = 2^{2n\lambda_1} \times 2^{-3n\lambda_1}$ we will have $x_{2i-1} = x_{2i} = 0$ for all shares $[[z]]_1$ that are randomly corrupted. This implies that the index $i$ will not be in the set of indices returned by $\mathsf{Identify}(\boldsymbol{c''},\mathsf{pp})$. If we assume that at least $n-t+1$ shares are corrupted this implies that, except with probability $2^{-n\lambda_1}$, we will have $\left|\{i : x_{2i} \neq 0 \vee x_{2i-1} \neq 0\}\right| < t$ in which case $\mathsf{Identify}$ will output $\perp$ with probability at least $1 - 2^{-n\lambda_1}$, as required. Since in our construction we select $\lambda_1 \geq \lceil \lambda/n \rceil$, we have $2^{-n\lambda_2} \leq 2^{(\lambda/n)n} = 2^\lambda$.

A similar argument shows that, for any a priori fixed variable assignment with $\left|\{i : x_i \neq 0\}\right| < 2t-1$, the probability that the assignment is consistent with all $3n$ constraints is at most $2^{-3n\lambda_1}$. The primary difference is that we are no longer assuming

that any shares are randomly corrupted. However, any subset of $2t-2$ shares from a $2t$ out of $2n$ Shamir Secret Sharing Scheme can be viewed as uniformly random so we can apply the same reasoning. We can once union bound over all $2^{2n\lambda_1}$ possible variable assignments to conclude that, except with probability $2^{-n\lambda_1}$, no assignment with $\left|\{i:x_i\neq 0\}\right|<2t-1$ will satisfy all of our equations. This implies that the set $S=\{i : x_{2i-1}\neq 0\vee x_{2i}\neq 0\}$ returned by $\mathsf{Identify}$ must have size at least $t$. In particular, as long there are at least $t$ valid shares to identify we will find at least $t$ valid shares (whp). In particular, if $S\subseteq[n]$ is a set of $|S|\geq t$ uncorrupted shares then we can always obtain a feasible assignment by setting $x_{2i-1}=L_{2i-1,S}(0)$ and $x_{2i}=L_{2i,S}(0)$. □

**Lemma 2.** Let $\boldsymbol{v}=(v_1,\ldots,v_m)\in\mathbb{Z}_p^m$ be any non-zero vector over a prime field $\mathbb{Z}_p$. Suppose that for each $i\leq m$ we run $(n,n-\ell)$-Shamir secret sharing to obtain shares $[[v]]_{i,1},\ldots,[[v]]_{i,n}$ and consider the matrix $m\times n$ matrix $A\in\mathbb{Z}_p^{m\times n}$ with coefficients $A_{i,j}=[[v]]_{i,j}$. For any $S=\{j_1,\ldots,j_{|S|}\}\subseteq[n]$ and any vectors $\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}\in\mathbb{Z}_p^m$ we define the matrix $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}$ to be the matrix obtained by replacing column $j_k$ of $A$ with the vector $\boldsymbol{r}_k$ for each $k\leq|S|$. Fix any $S\subseteq[n]$ of size $|S|\leq\ell$ then for any fixed vector $X=(X_1,\ldots,X_n)\in\mathbb{Z}_p^n$ s.t. $\mathsf{HammWeight}(X)<n-\ell$ we have $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]\leq p^{-m}$.

**Proof of Lemma 2:** In high level, to prove this lemma, we use equivalent view of the Share generation algorithm of $(n,n-\ell)$-Shamir secret sharing. Namely, we can randomly select $t<n-\ell$ shares randomly from $\mathbb{Z}_p$ and then we use Lagrange interpolation to compute the remaining shares using the chosen $t$-shares. To support this observation, we will show how to generate the shares. We first select $t-1$ shares $y_1,\ldots,y_{t-1}\in_R\mathbb{Z}_p^{t-1}$. Assuming that our secret is $s_0$ (as the constant coefficient associated to the random polynomial used to run the secret sharing), we compute the remaining $n-t+1$ shares for all $t\leq j\leq n$ as follows:

$$y_j=\big(L_{j,S}(0)\big)^{-1}\Big(s_0-\big(L_{1,S}(0)y_1+\ldots+L_{t-1,S}(0)y_{t-1}\big)\Big) \tag{1}$$

in which $L_{j,S}(0)=\prod_{i\in S,|S|=t,S\subset\{1,\ldots,n\}/j}\frac{i}{i-j}$ are the Lagrange coefficients. The way that we compute $y_j$ (for all $t\leq j\leq n$) enforces that:

$$s_0=L_{1,S}(0)y_1+\ldots+L_{t-1,S}(0)y_{t-1}+y_jL_{j,S}(0) \tag{2}$$

and we can recover the secret $s_0$ by the $t=n-\ell$ shares and the constructed shares $y_j$ for all $t\leq j\leq n$ are valid $(n,t=n-\ell)$-Shamir secret shares.

In the following we continue with proving the lemma.

Let $A\in\mathbb{Z}_p^{m\times n}$ with coefficients $A_{i,j}=[[v]]_{i,j}.\ell_j(0)$. For any $S=\{j_1,\ldots,j_{|S|}\}\subseteq[n]$ and any vector $\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}\in\mathbb{Z}_p^m$, we obtain $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}$ where the $j_k$-th columns is replaces with $\boldsymbol{r}_k$ for each $1\leq k\leq|S|$. For any fixed set $S\subseteq[n]$ of size $|S|\leq\ell$ and any fixed vector $X=(X_1,\ldots,X_n)\in\mathbb{Z}_p^n$ s.t. $\mathsf{HammWeight}(X)<t=n-\ell$ we compute an upper bound on $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]$. To do this we define $NZ(X)=\{i:X_i\neq 0\}$ to represent the indexes of non-zero locations of $X$ and $Z(x)=\{i:X_i=0\}$ as the indexes

of the locations that $X_i = 0$. As $\mathsf{HammWeight}(X) < t$, we have $|NZ(X)| < t$. We define $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}/Z(X)}$ as the matrix obtained by removing columns of $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}$ at all locations $i \in Z(X)$. We also define $X_{/Z(X)}$ as the vector obtained by removing all variables $X_i$ such that $i \in Z(X)$.

Now we can view all columns of $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}/Z(X)}$ at locations $j \in NX(X)$ as the random shares of Shamir secret sharing (due to the observation of equivalent view of share generation algorithm of Shamir secret sharing) we can consider all the elements in these columns as random elements chosen from $\mathbb{Z}_p$ and we have: So we have:

$$\begin{aligned}\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X = \boldsymbol{v}\right] &= \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}/Z(X)}X_{/Z(X)} = \boldsymbol{v}\right] \\ &= \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[\boldsymbol{a} = \boldsymbol{v}] \end{aligned} \tag{3}$$

In which $\boldsymbol{a} = \{a_1,\ldots,a_m\}$ in obtained by computing $\boldsymbol{a} = A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}/Z(X)}X_{/Z(X)}$. Due to equivalent view of Shamir share generation algorithm, for all $i \in NZ(X)$ the values of $a_i \in_R \mathbb{Z}_p$ are uniformly random and $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[a_i = v_i] \leq \left(\frac{1}{p}\right)$ as we have $p$ elements in $\mathbb{Z}_p$. So we have:

$$\begin{aligned}\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X = \boldsymbol{v}\right] &= \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}/Z(X)}X_{/Z(X)} = \boldsymbol{v}\right] \\ &= \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[\boldsymbol{a} = \boldsymbol{v}] \\ &= \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[a_1 = v_1] \cdots \Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[a_m = v_m] \\ &\leq \left(\frac{1}{p}\right)^m \end{aligned} \tag{4}$$

□

**Lemma 3.** Let $\boldsymbol{v} = (v_1,\ldots,v_m) \in \mathbb{Z}_p^m$ be any non-zero vector over a prime field $\mathbb{Z}_p$. Suppose that for each $i \leq m$ we run $(n, n-\ell)$-Shamir secret sharing to obtain shares $[[v]]_{i,1},\ldots,[[v]]_{i,n}$ and consider the matrix $m \times n$ matrix $A \in \mathbb{Z}_p^{m\times n}$ with coefficients $A_{i,j} = [[v]]_{i,j} \cdot \ell_j(0)$ for $\ell_j(x) = \prod_{m\in S,|S|=\ell-n,S\subset\{1,\ldots,n\}/j} \frac{m-x}{m-j}$. For any $S = \{j_1,\ldots,j_{|S|}\} \subseteq [n]$ and any vectors $\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|} \in \mathbb{Z}_p^m$ we define the matrix $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}$ to be the matrix obtained by replacing column $j_k$ of $A$ with the vector $\boldsymbol{r}_k$ for each $k \leq |S|$. Fix any $S \subseteq [n]$ of size $|S| \geq 1$, and fix any vector $X = (X_1,\ldots,X_n) \in \mathbb{Z}_p^n$ s.t. $|NZ(X) \cap S| \geq 1$ we have $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X = \boldsymbol{v}\right] \leq p^{-m}$ where $NZ(X) = \{X_i : X_i \neq 0\}$ and the randomness is taken over the selection of $\boldsymbol{r}_1\ldots,\boldsymbol{r}_{|S|}$.

**Proof of Lemma 3:** Let $A \in \mathbb{Z}_p^{m\times n}$ be the matrxi defined in Lemma 3. Fix any $S \subseteq [n]$ s.t. $|S| \geq 1$, and set $S_{\mathsf{Corrupt}} = NZ(X) \cap S$. Let $A_{S,\boldsymbol{r}_1\ldots,\boldsymbol{r}_{|S|}\setminus S_{\mathsf{Corrupt}}} \in \mathbb{Z}^{m\times n-|S_{\mathsf{Corrupt}}|}$ be a matrix obtained by removing $i$-th columns of $A_{S,\boldsymbol{r}_1\ldots,\boldsymbol{r}_{|S|}}$ for all $i \in S_{\mathsf{Corrupt}}$ and $X_{\setminus S_{\mathsf{Corrupt}}}$ be the vector of length $n - |S_{\mathsf{Corrupt}}|$ which is obtained by removing $i$-th variable $X_i$ for all $i \in S_{\mathsf{Corrupt}}$. Now we compute a bound on the probability

$\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]$ as follows:

$$\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]$$
$$=\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[\sum_{i\in S_{\mathsf{Corrupt}}}X_i\boldsymbol{r}_i=\boldsymbol{v}-A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}\setminus S_{\mathsf{Corrupt}}}X_{\setminus S_{\mathsf{Corrupt}}}\right]$$

As we have $S_{\mathsf{Corrupt}}=NZ(X)\cap S$ and $|S_{\mathsf{Corrupt}}|\geq 1$ we have $X_i\neq 0$ for all $i\in S_{\mathsf{Corrupt}}\neq\varnothing$ and all $\boldsymbol{r}_i$ are vectors of random elements in $\mathbb{Z}_p$, we can view $\boldsymbol{R}_{S_{\mathsf{Corrupt}}}=\sum_{i\in S_{\mathsf{Corrupt}}}X_i\boldsymbol{r}_i$ as vector of size $m$ chosen uniformly at random. We also call $\boldsymbol{a}=\boldsymbol{v}-A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}\setminus S_{\mathsf{Corrupt}}}X_{\setminus S_{\mathsf{Corrupt}}}\in\mathbb{Z}_p^m$ as constant fixed vector of size $m$. Now we have:

$$\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]=\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[\boldsymbol{R}_{S_{\mathsf{Corrupt}}}=\boldsymbol{a}\right]$$

We note that for fixed values $a=(a_1,\ldots,a_m)$ and any arbitrary vector $\boldsymbol{R}_{S_{\mathsf{Corrupt}}}=(R_1,\ldots,R_m)$, the probability that $R_j=a_j$ for all $j\in[m]$ is at most $\frac{1}{p}$ since there are $p$ elements in $\mathbb{Z}_p$. As all these $m$ events are independent, we can bound the probability of all $m$ equalities holding as follows:

$$\begin{aligned}Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]&=\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[\boldsymbol{R}_{S_{\mathsf{Corrupt}}}=\boldsymbol{a}\right]\\&=\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}[R_1=a_1,\ldots,R_m=a_m]\\&\leq\Pr_{R_1}[R_1=a_1]\ldots\Pr_{R_m}[R_m=a_m]\\&\leq\left(\frac{1}{p}\right)^m\end{aligned}$$

□

Intuitively, Lemma 3 tells us that with high probability the set $NZ(X)\subseteq\{i:i\text{ not corrupted}\}$ and if we solve the system of euqations and compute $X$, then the location of non-zero values are identifying the non-corrupted columns which equivalently tell us the indexes of non-corrupted shares (locations that characters of the unknown message and control messages are matching. )

As a useful result of Lemma 3, we state the following corollary.

**Corollary 1.** *Let $\boldsymbol{v}=(v_1,\ldots,v_m)\in\mathbb{Z}_p^m$ be any non-zero vector over a prime field $\mathbb{Z}_p$. Suppose that for each $i\leq m$ we run $(n,n-\ell)$-Shamir secret sharing to obtain shares $[[v]]_{i,1},\ldots,[[v]]_{i,n}$ and consider the matrix $m\times n$ matrix $A\in\mathbb{Z}_p^{m\times n}$ with coefficients $A_{i,j}=[[v]]_{i,j}\cdot\ell_j(0)$ for $\ell_j(x)=\prod_{m\in S,|S|=\ell-n,S\subset\{1,\ldots,n\}/j}\frac{m-x}{m-j}$. For any $S=\{j_1,\ldots,j_{|S|}\}\subseteq[n]$ and any vectors $\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}\in\mathbb{Z}_p^m$ we define the matrix $A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}$ to be the matrix obtained by replacing column $j_k$ of $A$ with the vector $\boldsymbol{r}_k$ for each $k\leq|S|$. Fix any $S\subseteq[n]$ of size $|S|\geq 1$, and fix any vector $X=(X_1,\ldots,X_n)\in\mathbb{Z}_p^n$, then we have $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[\exists X:NZ(X)\cap S\neq\varnothing,A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}X=\boldsymbol{v}\right]\leq p^{n-m}$ where $NZ(X)=\{X_i:X_i\neq 0\}$ and the randomness is taken over the selection of $\boldsymbol{r}_1\ldots,\boldsymbol{r}_{|S|}$.*

**Proof of Corollary 1:** The proof is basically is applying the results of Lemma 3 and taking union bound over all possible values of $X$. As $X \in \mathbb{Z}_p^n$, we have $p^n$ possible vectors $X$. For any fixed $S$ and fixed $X$ s.t. $|NZ(X) \cap S| \geq 1$ (i.e., $NZ(X) \cap S \neq \varnothing$), Lemma 3 tells that $\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}} X = \boldsymbol{v}\right] \leq p^{-m}$. So applying the union bound gives us:

$$\begin{aligned}
&\Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[\exists X : NZ(X) \cap S \neq \varnothing, A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}} X = \boldsymbol{v}\right] \\
&\leq \sum_{\forall X \in \mathbb{Z}_p^n} Pr_{\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}}\left[A_{S,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{|S|}} X = \boldsymbol{v}\right] \\
&= p^n \left(\frac{1}{q}\right)^m = p^{n-m} \ .
\end{aligned}$$

□

# B Missing Proofs of fPAKE from RRSS

## B.1 Proof of Theorem B.1

**Reminder of Theorem B.1.** *(*fPAKE *Correctness) If* $\Pi_{\mathsf{RRSS}}$ *is as an* $(n, n-\ell, \epsilon_{\mathsf{RRSS}}(\lambda))$*-random robust secret sharing,* $\pi$ *is* $\epsilon_{\mathsf{iPAKE}}(\lambda)$*-correct, the hash function* $\mathcal{H}$ *UC-realizes* $\mathcal{F}_{\mathsf{GRO}}$ *and UC-realizes* $\mathcal{F}_{\mathsf{iPAKE}}$*, then the protocol in Figure 2 is* $(\ell, \epsilon)$*-correct for* $\epsilon(\lambda) = n\epsilon_{\mathsf{iPAKE}}(\lambda) + \epsilon_{\mathsf{RRSS}}(\lambda)$ *where* $q$ *upper bounds the total number of queries to* $\mathcal{H}$*.*

**Proof of Theorem B.1:** Assume that $\mathsf{Party}_0$ and $\mathsf{Party}_1$ execute the protocol in Figure 2 honestly on inputs $\mathtt{pwd}_0$ and $\mathtt{pwd}_1$ with $\mathtt{Ham}(\mathtt{pwd}_0, \mathtt{pwd}_1) \leq \ell$, and let $\mathcal{A}$ be a passive adversary so that $\mathsf{sid}$ and $\mathsf{sid}_{\mathbb{H}}$ are consistent for each party. Thus, as long as $s = s'$ in the final step the final output keys $K = \mathcal{H}((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), s)$ and $K' = \mathcal{H}((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), s')$ will be identical.

By assumption, each execution of the underlying iPAKE protocol $\pi_j$ is $\epsilon_{\mathsf{iPAKE}}(\lambda)$-correct (Definition 3).

We define a sequence of $n+1$ hybrids $\mathcal{H}_0, \ldots, \mathcal{H}_n$ as follows: in hybrid $\mathcal{H}_j$, we abort if the $j$-th instance $\pi_j$ outputs different keys despite being run on matching inputs $\mathtt{pwd}_j^{\mathsf{Party}_0} = \mathtt{pwd}_j^{\mathsf{Party}_1}$. (If the inputs differ, we do not abort.)

By $\epsilon_{\mathsf{iPAKE}}(\lambda)$-correctness of $\pi$, each consecutive pair $\mathcal{H}_{j-1}$ and $\mathcal{H}_j$ differs with probability at most $\epsilon_{\mathsf{iPAKE}}(\lambda)$. By a union bound over all $n$ hybrids, the probability that any matching-character instance produces inconsistent keys is at most $n\epsilon_{\mathsf{iPAKE}}(\lambda)$. In the final hybrid $\mathcal{H}_n$ we are guaranteed that, if we don't abort, that we have $K_j = K'_j$ for at least $n-\ell$ keys since there are at least $n-\ell$ indices $j$ for which $\mathtt{pwd}_j^{\mathsf{Party}_0} = \mathtt{pwd}_j^{\mathsf{Party}_1}$. For indices $j \in [n]$ where $\mathtt{pwd}_j^{\mathsf{Party}_0} \neq \mathtt{pwd}_j^{\mathsf{Party}_1}$ the key $K'_j$ is uniformly random and independent of $K_j$.

Consequently, the reconstructed shares $\mathbf{C}' = \mathbf{E} \oplus \mathbf{K}'$ contain at least $n-\ell$ valid shares (equivalently, at most $\ell$ shares are randomly corrupted) and every invalid share was randomly corrupted. Since $\Pi_{\mathsf{RRSS}}$ is an $(n, n-\ell, \epsilon_{\mathsf{RRSS}}(\lambda))$-RRSS scheme, the secret $s$ is recovered with probability at least $1 - \epsilon_{\mathsf{RRSS}}(\lambda)$ whenever at least $n-\ell$ shares are correct.

Combining all of the error events yields

$$\Pr[K \neq K' \mid (K,K') \leftarrow out_{\mathcal{A},\pi}\langle \mathsf{Party}_0(\mathtt{pwd}^{\mathsf{Party}_0}), \mathsf{Party}_1(\mathtt{pwd}^{\mathsf{Party}_1})\rangle] \leq n\epsilon_{\mathsf{iPAKE}}(\lambda) + \epsilon_{\mathsf{RRSS}}(\lambda) \,.$$

□

### B.2 Proof of Theorem 3

**Reminder of Theorem 3.** *If $\Pi_{\mathsf{RRSS}}$ is an $(n, n-\ell, \epsilon_{\mathsf{RRSS}}(\lambda))$-random robust secret sharing, the hash function $\mathcal{H}$ UC-realizes $\mathcal{F}_{\mathsf{GRO}}$, and $\pi$ is $(t_{\mathsf{iPAKE}}(\lambda), \epsilon_{\mathsf{iPAKE}}(\lambda))$-universally compossible which realizes $\mathcal{F}_{\mathsf{iPAKE}}$ using simulator $\mathtt{Sim}_{\mathsf{iPAKE}}$ in presence of the environment $\mathcal{Z}^*$ running in time at most $t_{\mathsf{iPAKE}}(\lambda)$, then the protocol in Figure 2 UC-realizes $\mathcal{F}_{\mathsf{fPAKE}}$ in the $\mathcal{F}_{\mathsf{SA}}$-hybrid model with error tolerance in password $\ell$ and leakage threshold $\gamma = \ell$.*

**Proof of Theorem 3:** To prove this theorem, we need to show that no PPT environment $\mathcal{Z}$ has non-negligible advantage in distinguishing a real execution of the protocol in Figure 2 from interaction with the ideal world abstraction $\mathcal{F}_{\mathsf{fPAKE}}$ and the simulator described in Figure 4 and Figure 5.

So, we define a series of hybrids $\mathbf{H_0}$ to $\mathbf{H_{11}}$ similar to what we have in proof of Theorem 2 in [5]. We focus on the differeces and higlight them.

- $\mathbf{H_0}$: This is the real execution of the protocol in Figure 2.
- $\mathbf{H_1}$: **Create functionality and simulator**. This hybrid is exactly the same as $\mathbf{G_1}$ in [5]. More specifically, in this game we introduce a simulator and reformulate the experiment so that the execution of the real protocol is carried out inside the simulator. We also introduce dummy parties and an ideal functionality that simply forwards messages to enable this execution. When a party initiates a session with input $(TscNewSession, \mathsf{sid}, \mathtt{pwd}_{\mathsf{Party}_i}, \mathsf{role})$, the dummy party sends this input to the ideal functionality functionality, which forwards $(\textsc{NewSession}, \mathsf{sid}, \mathsf{Party}_i, \mathtt{pwd}_{\mathsf{Party}_i}, \mathsf{role})$ to the simulator without modifying the password. The simulator then internally runs the protocol instances using the parties' real passwords and produces the corresponding session keys. The functionality additionally provides an interface $(\textsc{NewKey}, \mathsf{sid}, \mathsf{Party}_i, sk)$ that allows the simulator to instruct the functionality to deliver a chosen key $sk$ as the output of party $\mathsf{Party}_i$. Since the protocol execution and outputs remain exactly the same as in the real experiment—only the location of execution changes—this transformation is purely syntactic, and the distributions in the real experiment and $H_0$ and $H_1$ are information theoretically identical and we have

$$\Pr[H_1] = \Pr[H_0]$$

.

- $\mathbf{H_2}$: **Abort on Collision of $\mathcal{H}$**. This hybrid is also the same as the $\mathbf{G_2}$ in [5]. More specifically, we modify the simulator so that it aborts the execution if a collision occurs in the underlying hash function $\mathcal{H}$ which UC-Realizes our $\mathcal{F}_{\mathsf{GRO}}$. More precisely, the simulator terminates the simulation whenever there exist two distinct inputs $s$ and $s'$ with $s \neq s'$ such that $\mathcal{H}((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), s) = \mathcal{H}((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), s')$.

Since the outputs of $\mathcal{H}$ are modeled as being chosen uniformly at random for each new query, the probability that a collision occurs among polynomially many queries is negligible. Therefore, the probability that the simulator aborts in this game is negligibly small, and the distributions in the previous hybrid and this game remain computationally indistinguishable and we have:

$$\big|\Pr[H_2]-\Pr[H_1]\big|\leq \mathsf{negl}(\lambda)$$

.

– **$\mathbf{H_3}$ Exchanging real $\mathsf{iPAKE}$ for simulated ones**. This hybrid is also the same as the $\mathbf{G_3}$ in [5]. Summary: introducing the binary variable MitM which is 0 indicating that there is not Meet-in-the-middle-attack; and 1, otherwise. In the presence of MitM, at most $2n$ separate $\mathsf{iPAKE}$ will be executed which are executed independently over $\mathcal{F}_{\mathsf{SA}}$ and applying the union bound we have

$$\big|\Pr[\mathbf{H_3}]-\Pr[\mathbf{H_2}]\big|\leq 2n\epsilon_{\mathsf{iPAKE}}(\lambda)$$

– **$\mathbf{H_4}$: Functionality makes records of running sessions, marks them, and reacts accordingly.** This hybrid is also the same as the $\mathbf{G_4}$ in [5]. Summary: Changing the ideal functionality by equipping it with NewSession, NewKey and TestPwd interface. Here, our simulator never calls $\mathcal{F}_{\mathsf{fPAKE}}$'s TestPwd interface and therefore the record never can be `interrupted`. So this hybrid and the previous hybrid are information theoretically equivalent.

$$\Pr[\mathbf{H_3}]=\Pr[\mathbf{H_4}]$$

– **$\mathbf{H_5}$: Simulator handles TestPwd queries of $\mathcal{F}_{\mathsf{fPAKE}}$ for honest and undisturbed sessions**. This hybrid is also the same as the $\mathbf{G_5}$ in [5]. In the following we provide a brief summary which describes a high level intuition.
**Summary:** We just modify the simulator. We assume that the parties are honest and not MitM attack. Therefore, when `Sim` receives a TestPwd query from $\mathtt{Sim}_{\mathsf{iPAKE}}$, it just ignores it and does not use the passwords of the parties. `Sim` still uses the parties passwords to simulate NewKey queries. The goal is to gradually update the hybrids such that in the last hybrids the simulator is not using the users' passwords. Here the key observation is that that $\pi$ is correct and the simulators cannot guess the password without quering the TestPwd.

– **$\mathbf{H_6}$: Simulator handles attacks on honest senders**. The basic idea is that the simulator collects the test password queries from $\mathtt{Sim}^i_{\mathsf{iPAKE}}$ (for the $i$-th character of the password), and then concatenate them as $\mathtt{pwd}^*=c_1\|c_2\|...\|c_n$ and make a TestPwd query to $\mathcal{F}_{\mathsf{fPAKE}}$. If $\mathtt{Ham}(\mathtt{pwd}_i,\mathtt{pwd}^*)\leq\ell$, then the ideal functionality reveals the location of the matching characters to the simulator `Sim` and the simulator use them to answer the TestPwd queries received from $\mathtt{Sim}^i_{\mathsf{iPAKE}}$ (for the correct characters). This the why we need the ideal functionality reveals the location of the matching characters in the ideal functionality. If $\mathtt{Ham}(\mathtt{pwd}_i,\mathtt{pwd}^*)>\ell$, then the simulator uses the users passwords to see where are the matching characters and use them to answer the test password queries from $\mathtt{Sim}_{\mathsf{iPAKE}}$.
In this hybrid, we need to apply the security of our $\Pi_{\mathsf{RRSS}}$. The key idea that we can appeal the security of our $\mathsf{RRSS}$ is that, we have two possible cases for each

individual character. If the characters are matching or not. If they are matching, then $\mathtt{Sim}^i_{\mathsf{iPAKE}}$ returns the valid session key and XORing with the $E_i$ we will receive the valid share for RRSS. If the character is not matching, then $\mathtt{Sim}^i_{\mathsf{iPAKE}}$ returning uniformly at random key $K_i$ and therefore $E_i \oplus K_i$ is looking uniformly at random and we can count it as a uniformly random corrupted share for our RRSS. So we observe that the shares are either correct or uniformly at random corrupted shares. We have two main cases.

1. Guessing at least $n-\ell$ correct characters: This hybrid is information theoretically the same as the previous hybrid. The only difference is that $\mathcal{F}_{\mathsf{fPAKE}}$ marks the corresponding record of the correct guesses as `compromised` instead of `fresh`.
2. Guessing less than $n-\ell$ valid shares: In this cases we randomly select a character and answer the TestPwd queries for the $\mathtt{Sim}^j_{\mathsf{iPAKE}}$ using the users' passwords. For the wrong positions `Sim` chooses uniformly at random keys and for the correct positions $j$, the simulator `Sim` uses the $K_j$ provided by the simulator $\mathtt{Sim}^j_{\mathsf{iPAKE}}$. Here can apply the security of our RRSS and we can distinguish between hybrid $\mathbf{H_5}$ and $\mathbf{H_6}$ with advantage at most $\epsilon_{\mathsf{RRSS}}(\lambda)$.

– $\mathbf{H_7}$: **Use random $\Longrightarrow$ keys in case of attack on honest sender**: Here the simulator is updated just in the case that MitM is done or the receiver is corrupted. The case that the $\mathcal{Z}$ guess at least $n-\ell$ correct character of the password, this game is the same as the previous hybrid. If $\mathcal{Z}$ guess at most $n-\ell-1$ correct characters, then we use random keys to simulate $\mathcal{F}_{\mathsf{iPAKE}}$. However, since the receiver received correct shares on at most $n-\ell-1$ locations, $n-\ell$ uniformity of our RRSS guarantees that $\mathbf{H_6}$ and $\mathbf{H_7}$ are information theoretically identical. So we have:

$$\Pr[\mathbf{H_6}] = \Pr[\mathbf{H_7}]$$

– $\mathbf{H_8}$: **Simulator handle attacks on honest receivers.** Similar to $\mathbf{H_6}$. We also need to consider the CR property of our hash function (OR random Oracle model).
– $\mathbf{H_9}$: **Simulator does not give passwords to $\mathcal{F}_{\mathsf{iPAKE}}$ instances.** The simulator does not use $\mathtt{pwd}_i[j]$ to run $\mathcal{F}_{\mathsf{iPAKE}}$ it receives (NewSession,sid,$\mathsf{Party}_i$,$\mathtt{pwd}_i$,role) from $\mathcal{F}_{\mathsf{fPAKE}}$ and forwards (NewSession,sid,$\mathsf{Party}_i$,role) to $\mathtt{Sim}^j_{\mathsf{iPAKE}}$ without using the $j$-th character of the password. The idea is that we know that $\mathtt{Sim}^j_{\mathsf{iPAKE}}$ can simulate $\mathcal{F}_{\mathsf{iPAKE}}$ with adavatage of at most $\epsilon_{\mathsf{iPAKE}}(\lambda)$.
– $\mathbf{H_{10}}$: **Simulator makes honest and undisturbed parties output a key.** This hybrid is also the same as the $\mathbf{G_{10}}$ in [5].
– $\mathbf{H_{11}}$: **Simulator does not get inputs of the parties anymore** This hybrid is also the same as the $\mathbf{G_{10}}$ in [5].

Note that $\mathbf{H_{11}}$ is the same as ideal execution of $\mathcal{F}_{\mathsf{fPAKE}}$ with the simulator from Figure 4 and Figure 5.

□

**Simulator $\mathtt{Sim}$ for $\mathcal{F}_{\mathsf{fPAKE}}$ (Part I)** **On** (NewSession, sid, $P_i$, role) from $\mathcal{F}_{\mathsf{fPAKE}}$ (or (Init,$\langle P_0,P_1\rangle$,sid) from a corrupted $P_i$ to $\mathcal{F}_{\mathsf{SA}}$):
- Create record ($P_i$,sid,role,$\perp$) and mark it fresh.
- Send (Init,$\langle P_0,P_1\rangle$,sid) to $\mathcal{Z}$ in the name of $\mathcal{F}_{\mathsf{SA}}$.

**On** (Init,sid,$P_i$,$H$,$\mathsf{sid}_{\mathbb{H}}$) from $\mathcal{Z}$ in the name of $\mathcal{A}$ to $\mathcal{F}_{\mathsf{SA}}$:
- Perform the same checks as $\mathcal{F}_{\mathsf{SA}}$ on $\mathsf{sid}_H$ and $H$ and ignore the message if one fails.
- If $P_i$ is corrupted then provide output (Init,sid,$\mathsf{sid}_H$) to $P_i$.
- If $\mathbb{H}=\{P_0,P_1\}$ then set MitM$:=0$, else MitM$:=1$.
- Retrieve ($P_i$,sid,role,$\perp$) and replace $\perp$ by MitM.
- If there is no instance of $\mathcal{F}_{\mathsf{iPAKE}}$ for session id ($\mathsf{sid},\mathsf{sid}_H,j$) for $j\in[n]$, create one.
- For all $j\in[n]$ send (NewSession,($\mathsf{sid},\mathsf{sid}_H,j$),$P_i$,role) to $\mathcal{F}_{\mathsf{iPAKE}}$.

**On** receiving output $Z^j$ from all $n$ instances of $\mathcal{F}_{\mathsf{iPAKE}}$:
- Store (sid,$P_i$,$P_{1-i}$,msg$:=(Z^1,\ldots,Z^n)$).

**On instruction from $\mathcal{Z}$ to send** (Deliver,sid,$P_i$,$P_{1-i}$,msg) to $\mathcal{F}_{\mathsf{SA}}$:
- If MitM$=0$:
  - If msg is the one sent by $\mathtt{Sim}$, instruct $\mathcal{F}_{\mathsf{iPAKE}}$ to send it.
  - Remove one occurrence of the stored record.
- Else (MitM$=1$): give the message to $\mathcal{Z}$.

**On** (TestPwd,($\mathsf{sid},\mathsf{sid}_{\mathbb{H}},j$),$P_i$,$\mathtt{pwd}_j$) from $\mathcal{F}_{\mathsf{iPAKE}}$:
- Store (TestPwd,sid,$P_i$,$j$,$\mathtt{pwd}_j$).

**On instruction from $\mathcal{Z}$ to send** (Deliver,sid,$P_{1-i}$,$P_i$,$(E',h')$) to $\mathcal{F}_{\mathsf{SA}}$:
- Retrieve record ($P_i$,sid,role,MitM,$K$) marked waiting.
- If MitM$=0$ and both parties are honest:
  - Remove the record.
  - If role = receiver then send (NewKey,sid,$P_i$,$\perp$) to $\mathcal{F}_{\mathsf{fPAKE}}$.
- Else if $P_i$ is honest and role = receiver:
  - Compute $s':=\Pi_{\mathsf{RRSS}}.\mathsf{SecretRecover}(\boldsymbol{E'}\oplus\boldsymbol{K})$.
  - If $s'\neq\perp$, then output key $sk$ accordingly.
  - Else output $\perp$.
- Else if $P_i$ is corrupt, output (Received,sid,$P_{1-i}$,$P_i$,$(E',h')$) to $P_i$.

**Random oracle queries**
- On query ($\mathsf{sid},\mathsf{sid}_{\mathbb{H}},s$) to $\mathcal{H}_1$: return stored value or sample $k\leftarrow\{0,1\}^\lambda$.
- On query ($\mathsf{sid},\mathsf{sid}_{\mathbb{H}},s$) to $\mathcal{H}_0$: return stored value or sample $\rho\leftarrow\{0,1\}^\lambda$ (abort on collisions).

Figure 4: The first part of the simulator $\mathtt{Sim}$ for $\mathcal{F}_{\mathsf{fPAKE}}$ (with list decoding).

# C Review of Cryptographic Tools and Building Blocks

## C.1 Details on Paillier Cryptosystem

In this part we will provided a formal definition of Paillier cryptosystem with more details. We should highlight that the Paillier cryptosystem [22] consists of three main algorithms $\Pi_{\mathtt{P}}$ = (P.KeyGen,P.Enc,P.Dec), and two main operations P.Add,P.PlainToCtxMul which are described as follows.

- $(pk,sk)\leftarrow\mathsf{P.KeyGen}\Big(1^\lambda;(p,q)\Big)$: This algorithm takes as input the security parameter $\lambda$ and two uniformly at random sampled large $\mathsf{poly}(\lambda)$[12] bit prime numbers $p,q\leq 2^{\mathsf{poly}(\lambda)}$. If the values of $p$ and $q$ are not large enough to provide $\lambda$-bit security level, we resample the values of prime numbers $p$ and $q$. Then, the algorithm generates the secret-public key pair $(pk,sk)$ as follows. It sets $N=pq$ and computes $\beta=\mathsf{lcm}(p-1,q-1)$. Let we define the function $L(u)=\frac{u-1}{N}$ in which $u\in[N^2]$ is a variable. So using the defined function to compute $\mu=(L(g^\beta \bmod N^2))^{-1} \bmod N$ in which $g\in\mathbb{Z}^*_{N^2}$ is sampled uniformly at random. Finally, the public key and its corresponding secret key is set as follows: $pk=(N,g)$ and $sk=(\beta,\mu)$. In this paper we considered $g=N+1$. In this case, $\beta=\mathsf{lcm}(p-1,q-1)$ and $\mu=\Phi(N)^{-1} \bmod N$.

[12] We should highlight that based on the security parameter we desire, there exists a polynomial function like poly which determined the bit length of the prime numbers.

**Simulator $\mathtt{Sim}$ for $\mathcal{F}_{\mathsf{fPAKE}}$ (Part II)**

**On** (NewKey, sid, $P_i$, $sk_j$) from $\mathtt{Sim}^j_{\mathsf{iPAKE}}$: record $\langle$NewKey, sid, $P_i$, $sk_j\rangle$. If this is the $n$-th NewKey query for $P_i$, do:

- Retrieve record $\langle P_i$,sid,role,MitM$\rangle$ marked $\mathtt{fresh}$.
- If MitM$=0$ and both parties of session sid are honest: choose $K \leftarrow \mathcal{F}_q^n$.
- Else // MitM$=1$ or one party is corrupted
  - Retrieve records $\langle$TestPwd, sid, $P_i$, $j$, $\mathtt{pwd}'_j\rangle$ for any $j \in [n]$ where such a record exists. Let $J$ be the set of $j$ for which a record exists.
  - If $|J| \geq n - \gamma$ such records, choose $\mathtt{pwd}'_t \leftarrow \{0,1\}$ for $t \in [n] \setminus J$ and send (TestPwd, sid, $P_i$, $\mathtt{pwd}'_1 \| \cdots \| \mathtt{pwd}'_n$) to $\mathcal{F}_{\mathsf{fPAKE}}$.
  - Else // $|J| < n-\gamma$ choose $K \leftarrow \mathcal{F}_q^n$ and send (TestPwd, sid, $P_i$, $\perp$) to $\mathcal{F}_{\mathsf{fPAKE}}$. // interrupt the record
  - Regardless, wait for $\mathcal{F}_{\mathsf{fPAKE}}$'s response:
    - ♠ If $\mathcal{F}_{\mathsf{fPAKE}}$ responds with ($T$,"correct guess") or ($T$,"wrong guess"), check if $|T \cap J| < n-\gamma$. If so, mark $\langle P_i$,sid,role,MitM$\rangle$ as Accident. Else retrieve records $\langle$NewKey, sid, $P_i$, $sk_t\rangle$ and set $K_t := sk_t$ for all $t \in T \cap J$ and choose $K_t \leftarrow \mathcal{F}_q$ for all $t \in [n] \setminus (T \cap J)$. Mark the record $\langle P_i$,sid,role,MitM$\rangle$ as $\mathtt{compromised}$.
    - ♠ Else // $\mathcal{F}_{\mathsf{fPAKE}}$ responds with "wrong guess" choose $\boldsymbol{K} \leftarrow \mathcal{F}_q^n$ and mark the record $\langle P_i$,sid,role,MitM$\rangle$ as $\mathtt{interrupted}$.
- Regardless:
  - If role$=$Receiver then append $\boldsymbol{K}$ to the record $\langle P_i$,sid,role,MitM$\rangle$ and mark it Waiting.
  - Else // role$=$Sender
    - ♠ Choose $s \leftarrow \{0,1\}^\lambda$ and $\mathsf{pp} \leftarrow \Pi_{\mathsf{RRSS}}.\mathsf{Setup}(1^\lambda)$, and set $C := \Pi_{\mathsf{RRSS}}.\mathsf{ShareGen}(s, \mathsf{pp})$, $\boldsymbol{E} := \boldsymbol{C} \oplus \boldsymbol{K}$, and $h := \mathcal{H}_0((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), C)$.
    - ♠ If the record $\langle P_i$,sid,role,MitM$\rangle$ is marked Accident, choose $\widehat{sk} \leftarrow \{0,1\}^\lambda$.
    - ♠ Else if it is marked $\mathtt{compromised}$, set $\widehat{sk} := \mathcal{H}_1((\mathsf{sid}, \mathsf{sid}_{\mathbb{H}}), s)$.
    - ♠ Else // $\mathtt{interrupted}$ or $\mathtt{fresh}$ set $\widehat{sk} := \perp$.
    - ♠ Regardless, send (NewKey, sid, $P_i$, $\widehat{sk}$) to $\mathcal{F}_{\mathsf{fPAKE}}$. Store $\langle \mathsf{sid}, P_i, P_{1-i}, (\boldsymbol{E}, h)\rangle$ and send $(P_i, P_{1-i}, (\boldsymbol{E}, h))$ to $\mathcal{Z}$ as a message from $\mathcal{F}_{\mathsf{SA}}$. Mark $\langle P_i$,sid,role,MitM$\rangle$ as completed.

Figure 5: The second part of the simulator $\mathtt{Sim}$ for $\mathcal{F}_{\mathsf{fPAKE}}$ and our Fuzzy PAKE with list decoding. $\mathtt{Sim}$ simulates $\mathcal{F}_{\mathsf{SA}}$ and the random oracles $H_0$ and $H_1$. If we write "retrieve record" and the record does not exist, $\mathtt{Sim}$ ignores the message.

- $c \leftarrow \mathsf{P.Enc}_{pk}(m;r)$: The encryption algorithm takes as input the message $0 \leq m < N$ and random coin $r \in_R \mathbb{Z}_N^*$, and computes the ciphertext as follows: $c := (1+N)^m r^N \mod(N^2)$.
- $m := \mathsf{P.Dec}_{sk}(c)$: The decryption algorithm takes as input the ciphertext $c \in \mathbb{Z}_{N^2}^*$ and decrypts it as follows: $m := L(c^\beta \mod N^2).\mu \mod N$.
- $c = \mathsf{P.Add}(c_1, c_2)$: Given $c_1 = \mathsf{P.Enc}_{pk}(m_1)$ and $c_2 = \mathsf{P.Enc}_{pk}(m_2)$ for two messages $m_1, m_2$, this algorithm computes a ciphertext of $m_1 + m_2 \mod N$ under the same public keys as follows. $c = c_1.c_2 \mod N^2$. Intuitively we have:

$$\begin{aligned} c &= (g^{m_1} r_1^N).(g^{m_2} r_2^N) \mod N^2 \\ &= g^{m_1+m_2}(r_1 r_2)^N \mod N^2 \\ &= g^m r^N \mod N^2 \end{aligned} \tag{5}$$

in which $m = m_1 + m_2$ and the resulting randomness is $r = r_1 r_2$.
For sake of simplicity, we use symbol $\boxplus$ for this algorithm and we have $c_1 \boxplus c_2 = \mathsf{P.Add}(c_1, c_2)$. Similarly, for the subtraction, we can define $\boxminus$ and we have $c_1 \boxminus c_2 = \mathsf{P.Add}(c_1, c_1^{-1})$. We can also define $\uplus$ which add $k$ ciphertexts $c_1, ..., c_k$ and we have $\uplus_{i=1}^k c_i = c_1 \boxplus ... \boxplus c_k$.

– $c = \mathtt{P.PlainToCtxMul}(m_1, c_2)$: This algorithm takes as input the plaintext message $m_1$ and $c_2 = \mathtt{P.Enc}_{pk}(m_2)$, and computes ciphertext of $m_1.m_2$ as follows:

$$\begin{aligned} c &= (c_2)^{m_1} \mod N^2 = (g^{m_2} r_2^N)^{m_1} = g^{m_1 m_2} (r_2^{m_1})^N \mod N^2 \\ &= g^m r^N \mod N^2 \end{aligned} \tag{6}$$

in which $m = m_1 m_2$ and the resulting randomness is $r = r_2^{m_1} \mod N^2$.
For sake of simplicity, we use symbol $\boxtimes$ for this algorithm and we have $m_1 \boxtimes c_2 = \mathtt{P.PlainToCtxMul}(m_1, c_2)$.

### C.2 Shamir Secret Sharing

Let $\mathbb{F}$ be a field of size $|\mathbb{F}| \geq n$. We define interpolation algorithm $\mathsf{InterPol}$ over $t$-degree polynomial $f : \mathbb{F} \to \mathbb{F}$ which takes as input $t$ tuples $(x_i = i, y_i = f(i))$ for $1 \leq i \leq t$, and outputs the main secret $f(0)$. In particular, we have $f(x) = \sum_{i \in S} y_i L_{i,S}(x)$ where $L_{i,S}(x) = \prod_{j \in S \setminus \{i\}} \left( \frac{x - x_i}{x_j - x_i} \right)$ denotes the Lagrange interpolating polynomial which can be computed. Finally, the original secret can be obtained by computing $f(0)$. For simplicity, we use $= \mathsf{InterPol}((x_1, f(x_1)), \ldots, (x_t, f(x_t)))$ to denote the Lagrage interpolation. The $(n,t)$- secret sharing scheme for the secret message $s \in \mathbb{F}$ is defined based on two algorithms $\mathsf{ShareGen}$, $\mathsf{SecretRecover}$ which are described in what follows.

– $([[s]]_1, \ldots, [[s]]_n) \leftarrow \mathsf{ShareGen}(n, t, s)$. This algorithm takes as input the secret $s \in \mathbb{F}$, the threshold value $t$ and the number of shares $n$. Then, it randomly samples the $(t-1)$-degree polynomial $\phi : \mathbb{F} \to \mathbb{F}$ such that $s = \phi(0)$ and sets the shares as $[[s]]_i = \phi(i)$, for all $1 \leq i \leq n$. To randomly sample the $t-1$-degree polynomial $\phi(x) = s + a_1 x + a_2 x^2 + \ldots + a_{t-1} x^{t-1}$, we simply select $\{a_i \in_R \mathbb{F}\}_{\forall i \in [t]}$ uniformly at random and form $\phi(x) = s + \sum_{i=1}^{t-1} a_i x^i$.
– $s \leftarrow \mathsf{SecretRecover}(i_1, \ldots, i_t, [[s]]_{i_1}, \ldots, [[s]]_{i_t})$. This algorithm takes as input a list $1 \leq i_1 < \ldots < i_t \leq n$ of $t$ distinct indices along with their corresponding shares $[[s]]_{i_1}, \ldots, [[s]]_{i_t}$. To recover the secret $s$ the algorithm simply runs the interpolation algorithm $s = \mathsf{InterPol}((i_1, [[s]]_{i_1}), \ldots, (i_t, [[s]]_{i_t}))$.

It is easy to verify that the $(t,n)$-secret sharing scheme $\Pi = (\mathsf{ShareGen}, \mathsf{SecretRecover})$ described above is perfectly correct and provides $t-1$-perfect privacy. In Lemma 4 we formally prove that Shamir Secrete Sharing provides $t-1$-perfect privacy. We stress that we are not claiming that this result is new or novel. However, for completeness, we provide an independent proof here in the appendix[13].

**Lemma 4 ($t-1$-Perfect privacy for Shamir Secret Sharing).** *$(n,t)$-Shamir secret sharing scheme provides $(t-1)$-perfect privacy.*

**Proof of Lemma 4:** Let $\mathbb{F}$ be a finite field, let the secret be $s \in \mathbb{F}$, and let $\alpha_1, \alpha_2, \ldots, \alpha_n \in \mathbb{F}$ be distinct nonzero public evaluation points. In a $(t,n)$-Shamir secret sharing scheme, the dealer samples $(a_1, a_2, \ldots, a_{t-1}) \xleftarrow{\$} \mathbb{F}^{t-1}$ independently and

[13] This proof was developed with the assistance of ChatGPT for drafting and exposition. The authors have verified all arguments and take full responsibility for the correctness and content of the result.

uniformly at random, and defines the polynomial $f(z)=s+a_1z+a_2z^2+\cdots+a_{t-1}z^{t-1}$. The $i$-th share is then $y_i=f(\alpha_i)$. We want to prove that the joint distribution of any subset of at most $t-1$ shares is uniform over the appropriate vector space, independently of the secret $s\in\mathbb{F}$. More precisely, for every $k\leq t-1$, for every subset of indices $\{i_1,...,i_k\}\subseteq[n]$ and for every vector $(x_1,...,x_k)\in\mathbb{F}^k$ we will prove that for any fixed $s$

$$\Pr\left[(y_{i_1},...,y_{i_k})=(x_1,...,x_k)\right]=\frac{1}{|\mathbb{F}|^k}$$

where $y_i=f(\alpha_i)$ for each $i$ and the randomness is taken over the selection of the coefficients $\alpha_1,...,\alpha_t$.

The proof of this lemma has two steps. In the first step we first translate our target event to its equivalent linear system of equations. Then, we compute the rand of the resulting matrix. In the third step of the proof, we count the number of all possible solutions to our system of linear equations. Finally, we conclude the proof by computing the probability of the event we plan to compute.

***Step 1**: Translate the event into a linear system.* Fix a secret $s\in\mathbb{F}$, fix distinct indices $i_1,...,i_k$, and fix target values $x_1,...,x_k\in\mathbb{F}$. The event $(y_{i_1},...,y_{i_k})=(x_1,...,x_k)$ means exactly that $f(\alpha_{i_r})=x_r$ for each $r=1,...,k$. Substituting the definition of $f$, this becomes $s+a_1\alpha_{i_r}+a_2\alpha_{i_r}^2+\cdots+a_{t-1}\alpha_{i_r}^{t-1}=x_r$ for $r=1,...,k$ Equivalently, $a_1\alpha_{i_r}+a_2\alpha_{i_r}^2+\cdots+a_{t-1}\alpha_{i_r}^{t-1}=x_r-s$ for $r=1,...,k$.

Thus, the event that the selected shares equal $(x_1,...,x_k)$ is equivalent to saying that the random coefficient vector $A=(a_1,...,a_{t-1})^\top\in\mathbb{F}^{t-1}$ satisfies a linear system $MA=b$ where $M=\begin{pmatrix}\alpha_{i_1} & \alpha_{i_1}^2 & \cdots & \alpha_{i_1}^{t-1}\\ \alpha_{i_2} & \alpha_{i_2}^2 & \cdots & \alpha_{i_2}^{t-1}\\ \vdots & \vdots & & \vdots\\ \alpha_{i_k} & \alpha_{i_k}^2 & \cdots & \alpha_{i_k}^{t-1}\end{pmatrix}\in\mathbb{F}^{k\times(t-1)}$ and $b=\begin{pmatrix}x_1-s\\ x_2-s\\ \vdots\\ x_k-s\end{pmatrix}\in\mathbb{F}^k$.

***Step 2**: Show that the matrix $M$ has rank $k$.* The matrix $M$ is a rectangular Vandermonde-type matrix. Since the evaluation points $\alpha_{i_1},...,\alpha_{i_k}$ are distinct and nonzero, its rows are linearly independent.

A convenient way to see this is to factor one $\alpha_{i_r}$ from each row:$(\alpha_{i_r},\alpha_{i_r}^2,...,\alpha_{i_r}^{t-1})=\alpha_{i_r}(1,\alpha_{i_r},...,\alpha_{i_r}^{t-2})$. Because each $\alpha_{i_r}\neq0$, multiplying a row by $\alpha_{i_r}$ does not affect linear independence. Therefore the row rank of $M$ is the same as the row rank of$M'=\begin{pmatrix}1 & \alpha_{i_1} & \cdots & \alpha_{i_1}^{t-2}\\ 1 & \alpha_{i_2} & \cdots & \alpha_{i_2}^{t-2}\\ \vdots & \vdots & & \vdots\\ 1 & \alpha_{i_k} & \cdots & \alpha_{i_k}^{t-2}\end{pmatrix}$. Now look at the first $k$ columns of $M'$. They form the $k\times k$ Vandermonde matrix $V=\begin{pmatrix}1 & \alpha_{i_1} & \alpha_{i_1}^2 & \cdots & \alpha_{i_1}^{k-1}\\ 1 & \alpha_{i_2} & \alpha_{i_2}^2 & \cdots & \alpha_{i_2}^{k-1}\\ \vdots & \vdots & \vdots & & \vdots\\ 1 & \alpha_{i_k} & \alpha_{i_k}^2 & \cdots & \alpha_{i_k}^{k-1}\end{pmatrix}$. Its determinant is $\det(V)=$

$\prod_{1\le r<\ell\le k}(\alpha_{i_\ell}-\alpha_{i_r})$, which is nonzero because the $\alpha_{i_r}$ are distinct. Hence $V$ is invertible, so its rows are linearly independent. Therefore the rows of $M'$ are linearly independent, and hence the rows of $M$ are linearly independent. We conclude that $\mathsf{rank}(M)=k$.

***Step 3**: Count the number of solutions to $MA=b$.* The unknown vector $A$ has $t-1$ coordinates, so the ambient space is $\mathbb{F}^{t-1}$. Since $M$ has rank $k$, then we can solve the system of linear equations for $k$ variables and the remaining $t-1-k$ variables are free. Each free variable has $|\mathbb{F}|$ possible cases. So, all possible solutions is $|\mathbb{F}|^{t-1-k}$.

Finally the probability $\Pr[y_1,...,y_k=x_1,...,x_k|s]$ is equivalent to the ratio of all valid solutions for our system of linear equation to the all possible values for the vector $(a_1,...,a_{t-1})$ and we have:

$$\Pr[y_1,...,y_k=x_1,...,x_k]=\frac{|\mathbb{F}|^{t-1-k}}{|\mathbb{F}^{t-1}|}=\frac{1}{|\mathbb{F}^k|}$$

Since the above equations is correct for all $k\le t-1$, so we have:

$$\Pr[y_1,...,y_k=x_1,...,x_k]=\frac{1}{|\mathbb{F}^{t-1}|}$$

and the joint distribution of any subset of $t-1$ shares is uniform.

***Conclusion.*** We have shown that for every fixed secret $s\in\mathbb{F}$, every subset of indices $\{i_1,...,i_k\}$ with $k\le t-1$, and every target vector $(x_1,...,x_k)\in\mathbb{F}^k$,

$$\Pr\big[(y_{i_1},...,y_{i_k})=(x_1,...,x_k)\big]=\frac{1}{|\mathbb{F}|^k}\ .$$

□

### C.3 Implicit-only PAKE (iPAKE)

Both [5] and [16] constructions used iPAKE as the main building block of their construction. An iPAKE is a specific type of PAKE which only achieves implicit authentication with respect to the honest parties and the adversary. In other words, the implicit terminology implies that in the end of the protocol, the two involved parties may not know if they extracted the same key or not. In addition, an active adversary who executes the attack on the protocol by guessing the password does not get any feedback on the correctness of their guess. We follow the ideal functionality $\mathcal{F}_{\mathsf{iPAKE}}$ defined by Dupont et al. [16] which is described in Figure 6. We should point out that Roy and Xu [25] showed that the ideal UC-functionality for PAKE like [6] do not offer correctness. Intuitively, the two parties can securely realize the PAKE functionality when the parties always output random keys. They showed that it is impossible to incorporate the correctness into ideal functionality, without considering the Man-In-the-Middle Attack. Additionally, they proved that this formulation is equivalent to demanding correctness from the PAKE protocol as a separate property. Similar to [5] we also require the correctness for iPAKE as well and is defined as follows.

The ideal functionality $\mathcal{F}_{\mathsf{iPAKE}}$ where $\lambda$ is the security parameter. It interacts with two involved parties $\mathsf{Party}_0$,$\mathsf{Party}_1$ and the adversary $\mathcal{A}$ via the following queries:

- **Upon receiving a query** (**NewSession**,sid,$\mathtt{pwd}_i$,role) **from** $\mathsf{Party}_i$**:**
  - Send (NewSession,sid,$\mathsf{Party}_1$,role) to $\mathcal{A}$.
  - If one of the following is true, record ($\mathsf{Party}_i$,$\mathtt{pwd}_i$) and mark this record as `fresh`:
    - This is the first NewSession query.
    - This is the second NewSession query and there is a record ($\mathsf{Party}_{1-i}$,$\mathtt{pwd}_{1-i}$).
- **Upon receiving a query** (**TestPwd**,sid,$\mathsf{Party}_i$,$\mathtt{pwd}^*$) **from** $\mathcal{A}$**:**
  If there is a `fresh` record ($\mathsf{Party}_i$,$\mathtt{pwd}_i$), then:
  - If $\mathtt{pwd}_i = \mathtt{pwd}^*$, mark the record `compromised`;
  - If $\mathtt{pwd}_i \neq \mathtt{pwd}^*$, mark the record as `interrupted`;
- **Upon receiving query** (**NewKey**,sid,$\mathsf{Party}_i$,$K$) **from** $\mathcal{A}$**, where** $|K| = \lambda$**:**
  If there is no record of the form ($\mathsf{Party}_i$,$\mathtt{pwd}_i$), or if this is not the first NewKey query for $\mathsf{Party}_i$, then ignore this query; otherwise:
  - If at least one of the following is true, then output (sid,$K$) to player $\mathsf{Party}_i$:
    - The record is `compromised`.
    - $\mathsf{Party}_i$ is corrupted.
    - The record is `fresh`, $\mathsf{Party}_i$ is corrupted, and there is a record ($\mathsf{Party}_{1-i}$,$\mathtt{pwd}_i$) with $\mathtt{pwd}_i = \mathtt{pwd}_{i-1}$.
  - If this record is `fresh`, both parties are honest, there is a record ($\mathsf{Party}_{1-i}$,$\mathtt{pwd}_{1-i}$) with $\mathtt{pwd}_i = \mathtt{pwd}_{1-i}$, a key $K'$ was sent to $\mathsf{Party}_{1-i}$, and ($\mathsf{Party}_{1-i}$,$\mathtt{pwd}_{1-i}$) was `fresh` at the time, then output (sid,$K'$) to $\mathsf{Party}_i$;
  - In any other case, pick a new random key $K'$ of length $\lambda$ and send (sid,$K'$) to $\mathsf{Party}_i$.
  - Mark the record ($\mathsf{Party}_i$,$\mathtt{pwd}_i$) as `completed`.

Figure 6: Functionality $\mathcal{F}_{\mathsf{iPAKE}}$ [16]

**Definition 3 (iPAKE Correctness [5]).** *We say an iPAKE protocol $\pi$ is $\epsilon$-correct if for all $\mathtt{pwd} \in \mathcal{PWD}$, for some finite dictionary $\mathcal{PWD}$, and for all passive (i.e., honest-but-curious) adversaries $\mathcal{A}$, the probability that in an execution of $\pi$ with honest parties $\mathsf{Party}_0$, $\mathsf{Party}_1$ the first party $\mathsf{Party}_0$ on input $\mathtt{pwd}$ outputs a different key than the second party $\mathsf{Party}_1$ on input $\mathtt{pwd}$ in the presence of $\mathcal{A}$ is at most $\epsilon(\lambda)$ and we have*

$$\Pr[K \neq K' | (K,K') \leftarrow out_{\mathcal{A},\pi}\langle \mathsf{Party}_0(\mathtt{pwd}),\mathsf{Party}_1(\mathtt{pwd})\rangle] \leq \epsilon(\lambda)$$

*where the probability is taken over the random coin of $\mathsf{Party}_0$,$\mathsf{Party}_1$ and $\mathcal{A}$ and the protocol $\pi$. The notation $(K,K') \leftarrow out_{\mathcal{A},\pi}\langle \mathsf{Party}_0(\mathtt{pwd}),\mathsf{Party}_1(\mathtt{pwd})$ implies the output of the protocol $\pi$ in presence of adversary $\mathcal{A}$ where $K$ and $K$ are given to $\mathsf{Party}_0$ and $\mathsf{Party}_1$, respectively.*

### C.4 Split Authentication

Another building used in the fPAKE construction in [5] is *Split Authentication*, introduced by Barak et al. [2]. Essentially, a split authentication (SA) is a protocol which implements something close to an authenticated channel in an unauthenticated setting. In this setting, the adversary has an additional ability to run two separate

executions of an authentication scheme without (one with each party) without the two parties realizing it. In the case that the parties are both honest, the adversary in the beginning of the protocol has the option of deciding to launch a man-in-the-middle attack and execute separate protocols with each party or to just observe the exchanged messages or add adversarial delay to the message delivery. The ideal functionality $\mathcal{F}_{\mathsf{SA}}$ for split authentication is formalized by Barak et al. [2] and we recall it in Figure 7.

**Functionality $\mathcal{F}_{\mathsf{SA}}$**

**Initialization:**

- **Activation**: Upon activation with input (INIT,$\mathsf{sid}$), where $\mathsf{sid} = (\mathbb{P},\mathsf{sid}')$ and $\mathbb{P}$ is the set of PIDs: record $\mathbb{P}$ and forward it to the adversary.
- **Registration**: On (INIT,$\mathsf{sid}$,$\mathcal{P}$,$\mathbb{H}$,$\mathsf{sid}_{\mathbb{H}}$) from the adversary, verify that:
  - $\mathcal{P} \in \mathbb{P}$ and $\mathcal{P} \in \mathbb{H}$.
  - for all recorded sets $\mathbb{H}'$, either
    - $\mathbb{H} \cap \mathbb{H}'$ contains only corrupt parties and $\mathsf{sid}_{\mathbb{H}} \neq \mathsf{sid}_{\mathbb{H}'}$ or
    - $\mathbb{H}' = \mathbb{H}$ and $\mathsf{sid}_{\mathbb{H}} = \mathsf{sid}_{\mathbb{H}'}$.

  If so, then output (INIT,$\mathsf{sid}$,$\mathsf{sid}_{\mathbb{H}}$) to $\mathcal{P}$ and record ($\mathbb{H}$,$\mathsf{sid}_{\mathbb{H}}$) if not yet recorded. Else, do nothing.

**Message Authentication:**

- **Send**: On input (SEND,$\mathsf{sid}$,$\mathcal{P}$,$\mathcal{P}'$,$msg$) from $\mathcal{P}$, where $\mathcal{P}' \in \mathbb{P}$, send ($\mathcal{P}$,$\mathcal{P}'$,$msg$) to $\mathcal{A}$ and add ($\mathcal{P}$,$\mathcal{P}'$,$msg$) to an (initially empty) list $\mathbb{W}$ of waiting messages. Note that the same triple can be appear multiple times in the list. Else, do nothing.
- **Deliver**: On (DELIVER,$\mathsf{sid}$,$\mathcal{P}$,$\mathcal{P}'$,$msg$) from $\mathcal{A}$, proceed as follows:
  - If $\mathcal{P}'$ did not previously receive an (INIT,$\mathsf{sid}$,$\cdot$), then do nothing.
  - Else, if $\mathcal{P}$ is in the authentication set $\mathbb{H}$ of $\mathcal{P}'$, and $\mathcal{P}$ is honest, then do:
    - ♠ If there is a triple ($\mathcal{P}$,$\mathcal{P}'$,$msg$) $\in \mathbb{W}$, then remove one appearance of the triple from $\mathbb{W}$ and output (RECEIVED,$\mathsf{sid}$,$\mathcal{P}$,$\mathcal{P}'$,$msg$) to $\mathcal{P}'$.
    - ♠ Otherwise, do nothing.
  - Else (i.e., $\mathcal{P}'$ received (INIT,$\mathsf{sid}$,$\cdot$), and either $\mathcal{P}$ is corrupted or $P \notin \mathbb{H}$), output (RECEIVED,$\mathsf{sid}$,$\mathcal{P}$,$\mathcal{P}'$,$msg$) to $\mathcal{P}'$, regardless of $\mathbb{W}$.

Figure 7: Ideal functionality $\mathcal{F}_{\mathsf{SA}}$ for split authentication [2].

### C.5 Global Random Oracle Model

Here we described the ideal functionality of Global random oracle model [7] $\mathcal{F}_{GRO}$ which is UC-realized using random oracle model. Basically, we can model the random oracle as a global setup functionality $\mathcal{F}_{\mathsf{GRO}}$ that is accessible to all parties and sessions. The functionality maintains a list of past queries and their responses, ensuring consistency across invocations. Upon receiving a query $x$, it returns a uniformly random string of length $\ell(n)$ if $x$ has not been queried before, and otherwise returns the previously assigned value. Queries made by the adversary are additionally recorded as illegitimate queries. This global functionality enables all protocol instances to share

the same oracle while preserving the unpredictability and consistency properties of a random oracle in the UC framework.

**Functionality $\mathcal{F}_{\text{GRO}}$**

**Parameters:** Output length $\ell(n)$ and a list $\bar{\mathcal{F}}$ of ideal functionality programs.

**State:** An initially empty list $Q$ of query–response pairs and, for each session identifier $\mathsf{sid}$, a list $Q_{\mathsf{sid}}$ of illegitimate queries.

1. Upon receiving a query $x$ from some party $P=(\mathsf{pid},\mathsf{sid})$ or from the adversary $S$:
   - If $(x,v)\in Q$ for some $v\in\{0,1\}^{\ell(n)}$, return $v$ to $P$.
   - Otherwise, sample $v\leftarrow\{0,1\}^{\ell(n)}$ uniformly at random, store $(x,v)$ in $Q$, and return $v$ to $P$.

   Parse $x$ as $(\mathsf{sid},x')$. If $\mathsf{sid}\neq\perp$, add $(\mathsf{sid},x',v)$ to the list $Q_{\mathsf{sid}}$ of illegitimate queries.
2. Upon receiving a request from an instance of an ideal functionality in $\bar{\mathcal{F}}$ with session identifier $\mathsf{sid}$, return the list $Q_{\mathsf{sid}}$.

Figure 8: Global Random Oracle functionality $\mathcal{F}_{\mathsf{GRO}}$. [7]

# D Conditional Encryption

In this section, we review formal definition of a semi-honest *conditional encryption* scheme as introduced in [1]. In comparison to the traditional public key encryption schemes, a conditional encryption scheme contains similar algorithms with the addition of a special algorithm $\mathsf{CEnc}_{pk}$ called conditional encryption. This algorithm takes as inputs: a ciphertext $c=\mathsf{Enc}_{pk}(m_1;r)\in\mathcal{C}_{\mathsf{Enc}}$ (Where $\mathcal{C}_{\mathsf{Enc}}$ is the ciphertext space of traditional encryption scheme) encrypting some unknown message $m_1\in\mathcal{M}$ in our message space using random coin $r\in_R\{0,1\}^\lambda$, a control message $m_2$ and a payload message $m_3$. A conditional encryption scheme is defined with respect to a binary predicate $P:\mathcal{M}\times\mathcal{M}\to\{0,1\}$. Intuitively, if $P(m_1,m_2)=1$, then $\mathsf{CEnc}_{pk}(c,m_2,m_3)\in\mathcal{C}_{\mathsf{CEnc}}$[14] should produce valid encryption of our payload message $m_3$; otherwise, if $P(m_1,m_2)=0$ the output should reveal *no information* about any of the messages $m_1$,$m_2$ or $m_3$ — even to an adversary that knows the secret decryption key $sk$.

## D.1 Conditional Encryption Syntax, Correctness and Error Detection

**Definition 4 (Algorithms).** *[1] A conditional encryption scheme $\Pi$ for a binary predicate $P:\left(\mathcal{M}\cup\{\perp\}\right)\times\mathcal{M}\to\{0,1\}$ consists of four main algorithms* $(\mathsf{KeyGen},\mathsf{Enc},\mathsf{CEnc},\mathsf{Dec})$ *which are described as follows:*

[14] Similarly, we define $\mathcal{C}_{\mathsf{CEnc}}$ as the ciphertext space for a conditional encryption scheme.

- $(sk,pk) \leftarrow \mathsf{KeyGen}(1^\lambda;r)$*: takes as input the security parameter* $\lambda$ *and random coins* $r \leftarrow_R \{0,1\}^{p(\lambda)}$ *and generates a secret key* $sk \in \mathcal{SK}$ *and the corresponding public key* $pk \in \mathcal{PK}$ *for our conditional encryption scheme.*
- $(b=0,c) = \mathsf{Enc}_{pk}(m_1;r)$*: takes as input a plaintext message* $m_1 \in \mathcal{M}$ *the public key* $pk$ *random nonce* $r \leftarrow_R \{0,1\}^{p(\lambda)}$ *and outputs a ciphertext* $(b,c) \in \{0\} \times \mathcal{C}$ *encrypting* $m_1$*. The flag* $b=0$ *indicates that* $c$ *is output of the regular encryption scheme* $\mathsf{Enc}_{pk}$*.*
- $(b=1,\tilde{c}) = \mathsf{CEnc}_{pk}\Big(\big(0,c_{m_1}\big),m_2,m_3;r\Big)$*: This conditional encryption algorithm takes as input a public key* $pk$*, a ciphertext* $(0,c_{m_1})$ *with* $c_{m_1} \in \mathcal{C}$ *corresponding to an unknown message* $m_1 \in \mathcal{M}$*, a control message* $m_2$*, a payload message* $m_3 \in \mathcal{M}$*, and random nonce* $r \leftarrow_R \{0,1\}^{p(\lambda)}$ *and outputs a ciphertext* $(b,\tilde{c}) \in \{1\} \times \mathcal{C}$*. The flag* $b=1$ *indicates that this ciphertext is the output of the conditional encryption* $\mathsf{CEnc}$ *algorithm. Note: When the control message and the payload message are the same* $m_2 = m_3$ *we will sometimes write* $\mathsf{CEnc}_{pk}(c_{m_1},m_2;r)$ *instead of* $\mathsf{CEnc}_{pk}(c_{m_1},m_2,m_2;r)$*. If the input ciphertext takes the form* $(b=1,c_m)$ *then* $\mathsf{CEnc}_{pk}\Big(\big(1,c_{m_1}\big),m_2,m_3;r\Big) = \perp$ *i.e.,* $(b=0,c_{m_1})$ *must be the output of the regular encryption scheme.*
- $\{m,\perp\} = \mathsf{Dec}_{sk}(c)$*: takes as input a ciphertext* $c \in \{0,1\} \times \mathcal{C}$ *and the secret key* $sk$ *and outputs a message* $m \in \mathcal{M}$ *or* $\perp$ *(indicating failure).*

We require that for any valid pair $(sk,pk)$ produced that $\Pr\Big[\mathsf{Dec}_{sk}\big(\mathsf{Enc}_{pk}(m)\big) = m\Big] = 1$ i.e., perfect correctness for ciphertexts output by the regular encryption algorithm. For correctness of the conditional encryption algorithm we want to ensure that $\mathsf{Dec}_{sk}\Big(\mathsf{CEnc}_{pk}\big(c',m_2,m_3\big)\Big) = m_3$ whenever $\exists r',m_1$ s.t. $c' = \mathsf{Enc}_{pk}(m_1;r')$ and $P(m_1,m_2) = 1$. Intuitively, we can extract our intended payload $m_3$ if and only if $P(m_1,m_2) = 1$. For conditional encryption we relax our requirement of perfect correctness and instead require that $\Pr\Big[\mathsf{Dec}_{sk}\Big(\mathsf{CEnc}_{pk}\big(c',m_2,m_3\big)\Big) = m_3\Big] \geq 1-\epsilon(\lambda)$ for a negligible function $\epsilon(\cdot)$ whenever $\exists r',m_1$ s.t. $c' = \mathsf{Enc}_{pk}(m_1;r')$ and $P(m_1,m_2)=1$. We stress that the correctness condition only holds when when $c' \leftarrow \mathsf{Enc}_{pk}(m_1)$ was the output of the regular encryption algorithm i.e., if $c' = \mathsf{CEnc}_{pk}(c'',m_2,m_3)$ we make no guarantees. In fact, if $c=(1,\tilde{c})$ was generated by conditional encryption, then the algorithm $\mathsf{CEnc}_{pk}(c,m_2,m_3)$ can output $\perp$ as is the case in all of our constructions.

**Definition 5 (Correctness).** *[1] We say that* $\Pi$ *is* $1-\epsilon(\cdot)$*-correct if the following conditions hold:*

- ***Regular encryption correctness****.* $\forall r_1,r_2 \in \{0,1\}^{p(\lambda)},m \in \mathcal{M}$ *we have* $\mathsf{Dec}_{sk}\big(\mathsf{Enc}_{pk}(m;r_2)\big) = m$ *whenever* $\{sk,pk\} \leftarrow \mathsf{KeyGen}(1^\lambda;r_1)$*.*
- *(****Conditional encryption correctness****. )* $\forall r_1,r_2 \in_R \{0,1\}^{p(\lambda)},m_1 \in \mathcal{M},m_2,m_3 \in \mathcal{M}$ *such that* $P(m_1,m_2)=1$ *we have*

$$\Pr\Big[\mathsf{Dec}_{sk}\Big(\mathsf{CEnc}_{pk}\big(\mathsf{Enc}_{pk}(m_1;r_2),m_2,m_3;r_3\big)\Big) = m_3\Big] \geq 1-\epsilon(\lambda)$$

*where the randomness is taken over the random coins* $r_3$ *of* $\mathsf{CEnc}$ *and we fix* $\{sk,pk\} \leftarrow \mathsf{KeyGen}(1^\lambda;r_1)$*.*

*If* $\epsilon(\lambda)=0$ *we simply say that* $\Pi$ *is correct.*

**Definition 6 (Efficiency).** *[1] We say that the conditional encrypt scheme* $\Pi = (\mathsf{KeyGen},\mathsf{Enc},\mathsf{CEnc},\mathsf{Dec})$ *is efficient if all four algorithms run in probabilistic polynomial time in the security parameter* $\lambda$.

Optionally, conditional encryption $\Pi$ should be error-detecting, i.e., whenever $P(m_1,m_2)=0$, the ciphertext $c=\mathsf{CEnc}_{pk}(c_1,m_2,m_3)$ decrypts to a special symbol $\mathsf{Dec}_{sk}(c)=\perp$ (whp). If $\Pi$ is error detecting, this allows us to detect which outputs of the $\mathsf{CEnc}$ algorithm are (in)valid.

**Definition 7 (Error Detecting).** *[1] We say that* $\Pi$ *is* $1-\epsilon(\cdot)$*-error detecting if* $\forall r_1,r_2 \in_R \{0,1\}^{p(\lambda)}, m_1 \in \mathcal{M}, m_2,m_3 \in \mathcal{M}$ *such that* $P(m_1,m_2)=0$ *we have* $\Pr\left[\mathsf{Dec}_{sk}\left(\mathsf{CEnc}_{pk}\left(\mathsf{Enc}_{pk}(m_1;r_2),m_2,m_3;r_3\right)\right)=\perp\right] \geq 1-\epsilon(\lambda)$ *where the randomness is taken over the selection of the random coins* $r_3$ *of* $\mathsf{CEnc}$ *and we fix* $\{sk,pk\} \leftarrow \mathsf{KeyGen}(1^\lambda;r_1)$.

We now formally review the security of a conditional encryption scheme. The security is defined based on a game between the challenger and adversary/distinguisher. In the security game, we ask the distinguisher to distinguish between a simulated ciphertext $\mathtt{Sim}(pk)$ and a conditionally encrypted ciphertext $\mathsf{CEnc}_{pk}(c_1,m_2,m_3)$ — assume that $c_1=\mathsf{Enc}_{pk}(m_1,r_1)$ with $P(m_1,m_2)=0$. Clearly, the simulated ciphertext $\mathtt{Sim}(pk)$ cannot leak any information to the adversary as it is generated without knowledge of the control message $m_2$, the payload message $m_3$ or the ciphertext $c_1$.

**D.1.1 Conditional Encryption Secrecy** We assume that the public key $(sk,pk)=\mathsf{KeyGen}(1^\lambda)$ and the ciphertext $c_1=\mathsf{Enc}_{pk}(m_1,r_1)$ were both generated honestly. Semi-honest security is easier to achieve and the security guarantee is already sufficient for applications such as password typo vaults [1, 10] where the keys and ciphertexts are generated by trusted parties.

**Definition 8 (Conditional Encryption Secrecy).** *[1] We say that conditional encryption scheme* $\Pi=(\mathsf{KeyGen},\mathsf{Enc},\mathsf{CEnc},\mathsf{Dec})$ *provides* $\left(t(\cdot),t_{\mathtt{Sim}}(\cdot),\epsilon(\cdot)\right)$*-conditional encryption secrecy if there exists a simulator* $\mathtt{Sim}$ *running in time at most* $t_{\mathtt{Sim}}(\lambda)$ *such that for all messages* $m_1,m_2,m_3 \in \mathcal{M}$ *such that* $P(m_1,m_2)=0$*, all* $\lambda \in \mathbb{N}$*,* $r,r_1 \in \{0,1\}^\lambda$ *and all distinguishers* $\mathcal{D}$ *running in time at most* $t(\lambda)$

$$\begin{aligned}&\left|\Pr\left[\mathcal{D}\left(sk,pk,m_1,m_2,m_3,c_1,\mathtt{Sim}(pk)\right)=1\right]\right.\\&\left.-\Pr\left[\mathcal{D}\left(sk,pk,m_1,m_2,m_3,c_1,\mathsf{CEnc}_{pk}(c_1,m_2,m_3)\right)=1\right]\right|\\&\leq \epsilon\left(t(\lambda),\lambda\right)\end{aligned} \tag{7}$$

*where the randomness is taken over the random coins of the distinguisher and the conditional encryption algorithm* $\mathsf{CEnc}$*. We use* $(sk,pk)=\mathsf{KeyGen}(1^\lambda;r)$ *and* $c_1=\mathsf{Enc}_{pk}(m_1;r_1)$ *to denote public/secret key and ciphertext computed under the apriori fixed random strings* $r$ *and* $r_1$*. If* $\epsilon(\cdot)=0$ *and* $t(\cdot)=\infty$ *then we say that* $\Pi$ *has perfect conditional encryption secrecy.*

**Real or Random Security.** Similar to traditional public key encryption schemes, it is required that the encryption scheme is secure against any computationally bounded

adversary who does not have the secret key $sk$. A conditional encryption scheme must also satisfy the traditional notion of Real-Or-Random (ROR) security for public-key encryption. In the ROR security game, the attacker is given access to an oracle $O_b$ that takes as input a message $m$ and outputs $\mathsf{Enc}_{pk}(m)$ when $b=1$ or outputs a random unrelated ciphertext when $b=0$. The attacker's goal is to predict the bit $b$, i.e. does the oracle output honest ciphertexts or random ones? Ameri and Blocki [1] extended the notion of real-or-random security to allow the attacker oracle access to the conditional encryption algorithm as well. In the following, we also review the Real-or-Random security based on the a security experiment/game called $\mathtt{CE}_{\mathcal{A}}^{ROR}(1^{\lambda})$.

Intuitively, the attacker accesses a conditional encryption oracle that accepts a triple $(c,m_2,m_3)$ as input (the attacker also accesses a regular encryption oracle that accepts $m$ as input and returns its encryption or the encryption of a randomly chosen message). If $b=0$, then the oracle simply returns the honestly computed ciphertext $c_0=\mathsf{CEnc}_{pk}(c,m_2,m_3)$. Otherwise, the oracle picks random messages $r'$ and $r''$ that are not related to the control message $m_2$ and the payload message $m_3$ and returns $c_1=\mathsf{CEnc}_{pk}(c,r',r'')$. We remark that the attacker gets to pick the ciphertext $c$ in addition to the messages $m_2$ and $m_3$. Thus, if the attacker cannot distinguish $c_0$ from $c_1$, the attacker cannot learn anything about the payload message $m_2$ or $m_3$ even if it carries out a malicious input $c'$.

**RoR Experiment:** In the security game, we pick a random $(sk,pk)=\mathsf{KeyGen}(1^{\lambda};r)$ and the attacker tries to distinguish the encryption oracles $\mathsf{Enc}_{pk}(\cdot)$ and $\mathsf{CEnc}_{pk}(\cdot,\cdot)$ from random encryption oracles. More precisely, we consider the following experiment $\mathtt{CE}_{\mathcal{A}}^{ROR}(1^{\lambda})$: (1) The challenger $\mathcal{C}$ picks a random bit $b\in\{0,1\}$ and generates a random public/secret key pair $(sk,pk)\leftarrow\mathsf{KeyGen}(1^{\lambda})$; (2) The challenger sends $pk$ to the attacker $\mathcal{A}(1^{\lambda})$; (3) Whenever the attacker submits a message $m$ to the encryption oracle the challenger sets $m_0=m$ and picks a random message $m_1$ and sends back $\mathsf{Enc}_{pk}(m_b)$. (4) Whenever the attacker $\mathcal{A}$ submits a pair $(c,m,m')$ to the conditional encryption oracle the challenger sets $c_0=\mathsf{CEnc}_{pk}(c,m,m')$ and sets $c_1=\mathsf{CEnc}_{pk}(c,r,r')$ for a random messages $r,r'$ and sends back $c_b$. (5) The game ends when $\mathcal{A}$ outputs a bit $b'$ and the output of the experiment is $\mathtt{CE}_{\mathcal{A}}^{ROR}(1^{\lambda})=1$ if and only if $b=b'$.

**Definition 9 (Real or Random Security).** *We say that a conditional encryption scheme* $\Pi=(\mathsf{KeyGen},\mathsf{Enc},\mathsf{CEnc},\mathsf{Dec})$ *is* $\big(t(\cdot),q(\cdot),\epsilon(\cdot)\big)$*-secure if for all attackers* $\mathcal{A}$ *running in time* $t(\lambda)$ *and making at most* $q(\lambda)$ *oracle queries in total we have* $\Pr\left[\mathtt{CE}_{\mathcal{A}}^{ROR}\left(1^{\lambda}\right)=1\right]\leq\frac{1}{2}+\epsilon\big(t(\lambda),q(\lambda),\lambda\big)$.

### D.2 Construction: Arbitrary Hamming Distance

In the main body we fully described our efficient conditional encryption construction supporting *arbitrary* Hamming distances. For convenience, we describe the construction again in **Construction 2** in a separate algorithmic environment.

### D.3 Proving Conditional Encryption Secrecy for Construction 2

**Reminder of Theorem 4.** *Assume that our Authenticated encryption scheme* $\Pi_{AE}=(\mathsf{AuthEnc},\mathsf{AuthDec})$ *is* $\big(t_{AE},\epsilon_{AE}(t_{AE},\lambda)\big)$*-secure for any security parameter* $\lambda$

**Construction 2**

$\underline{(sk,pk) \leftarrow \mathsf{KeyGen}(1^\lambda,n,\ell)}$:

1. run $\left(\mathtt{P}.sk,\mathtt{P}.pk=(N)\right) \leftarrow \mathtt{P}.\mathsf{KeyGen}(1^\lambda;r)$.
2. set $\mathtt{SS}=\Pi_{RRSS}.(\mathsf{ShareGen},\mathsf{SecretRecover},\mathsf{Identify})$ over finite field $\mathbb{F}_{2^\lambda}$.
3. set $t=n-\ell$, compute $\mathsf{pp}=(\boldsymbol{v},n,t,\Pi^\lambda_{SSS,n,t},\Pi^{\lambda_1}_{SSS,2n,2t}) \leftarrow \mathtt{SS}.\mathsf{Setup}(1^\lambda,n,t)$
4. return $\left(pk'=(\mathtt{P}.pk=N,\mathsf{pp}),sk'=\mathtt{P}.sk\right)$.

$\underline{c \leftarrow \mathsf{Enc}_{pk}(m;r)}$:

1. parse $m=(m[1],...,m[n]), r=(r_1,...,r_n) \in \left(\mathbb{Z}_N^*\right)^n$.
2. $\forall 1 \le i \le n$:
   - compute $\hat{m}[i]=\mathsf{ToInt}(m[i])$
   - compute $c_i=(1+N)^{\hat{m}[i]} {r_i}^N \mod N^2$
3. output $c=(b=0,c_1,...,c_n)$.

$\underline{c \leftarrow \mathsf{CEnc}_{pk}(c_m,m_2,m_3;r)}$:

1. parse $m_2 = (m_2[1],...,m_2[n]) \in \Sigma^n$, $r=(r_1,...,r_n) \in \left(\mathbb{Z}_N^*\right)^n$, $c_m = (b,c_1,\cdots,c_n)$, $pk=(\mathtt{P}.pk=N,\mathsf{pp},\mathsf{d_{Pack}},\lambda,\lambda_1)$;
2. if $b=1$, output $\perp$.
4. $\forall i \in [n]$: compute $\hat{m}_2[i]=\mathsf{ToInt}(m_2[i])$;
5. sample $K \in_R \{0,1\}^\lambda$, compute $c_{AE}=\mathsf{Auth.Enc}_K(m_3)$;
6. $\left([[s]]_1,...,[[s]]_n\right) \leftarrow \mathtt{SS}.\mathsf{ShareGen}(K,\mathsf{pp})$;
7. $\forall 1 \le i \le n$:
   - Partition each $[[s]]_i \in \{0,1\}^m$ into $\mathsf{d_{Pack}}$ strings $s_{1,1},...,s_{1,\mathsf{d_{Pack}}}$ such that $|s_{i,j}|=\lceil m/\mathsf{d_{Pack}} \rceil$ for $j<\mathsf{d_{Pack}}$ and $|s_{1,\mathsf{d_{Pack}}}|=m-\sum_{j=1}^{\mathsf{d_{Pack}}}|s_{i,j}|$.
   - Compute $\hat{s}_{i,j} = \mathsf{REnc}(s_{i,j}, 2^{\lceil m/\mathsf{d_{Pack}} \rceil}, N)$ for all $j < \mathsf{d_{Pack}}$ and $\hat{s}_{i,j} = \mathsf{REnc}(s_{i,j},2^{m-\mathsf{d_{Pack}}\lceil m/\mathsf{d_{Pack}} \rceil+\lceil m/\mathsf{d_{Pack}} \rceil},N)$.
   - For each $j \le \mathsf{d_{Pack}}$ compute $c_i^j = (N+1)^{R_{i,j}m_2[i]+\hat{s}_{i,j}} c[i]^{R_{i,j}} r_{i,j}^N \bmod N^2$. Note that if $m_2[i]=m[i]$ (the two strings match at character $i$) then $c_i^j$ will be a valid Pallier encryption of $\hat{s}_{i,j}$. Otherwise, $c_i^j$ will be a uniformly random Pallier ciphertext in $\mathbb{Z}^*_{N^2}$.
   - Set $c'[i]=(c_i^1,...,c_i^{\mathsf{d_{Pack}}})$.
8. Set the final ciphertext as $(b=1,c,c_{AE})$ in which $c=\left(c'[1],...,c'[n]\right)$.

$\underline{\{m,\perp\}=\mathsf{CDec}_{sk}\left((b,c)\right)}$:

1 if $b=1$ // The ciphertext is extracted from $\mathsf{CEnc}(\cdot)$

1.i For all $\forall i \le n$: Parse $c[i]=(c_i^1,...,c_i^{\mathsf{d_{Pack}}})$
   - $\forall j \le \mathsf{d_{Pack}}$: compute $\hat{s}'_{i,j}=\mathtt{P}.\mathsf{Dec}_{sk}(c_i^j)$
   - $\forall j < \mathsf{d_{Pack}}$: compute $s'_{i,j}=\hat{s}'_{i,j} \bmod 2^{\lceil m/\mathsf{d_{Pack}} \rceil}$.
   - compute $s'_{i,\mathsf{d_{Pack}}}=\hat{s}'_{i,\mathsf{d_{Pack}}} \bmod 2^{m-\mathsf{d_{Pack}}\lceil m/\mathsf{d_{Pack}} \rceil+\lceil m/\mathsf{d_{Pack}} \rceil}$.
   - Interpret each $s'_{i,j}$ as a bit string and set $[[s']]_i=s'_{i,1} \circ ... \circ s'_{i,\mathsf{d_{Pack}}}$.

1.ii Set $\boldsymbol{s}'=\left([[s']]_1,...,[[s']]_n\right)$ and compute $K':=\Pi_{RRSS}.\mathsf{SecretRecover}\left(\boldsymbol{s}',\mathsf{pp}\right)$.

1.iii Return $\{m,\perp\}:=\mathsf{AuthDec}_{K'}(c_{AE})$.

2 if $b=0$ // The ciphertext is extracted from $\mathsf{Enc}(\cdot)$
   - parse $c=(c_1,\cdots,c_n)$, AND set $m=\perp$
   - for $1 \le i \le n$, compute $\hat{m}_i=\mathtt{P}.\mathsf{Dec}_{sk}(c_i)$
   - return $m=\mathsf{ToInt}^{-1}(\hat{m}_1)\|...\|\mathsf{ToInt}^{-1}(\hat{m}_n)$

Figure 9: Concrete construction of conditional encryption over binary predicate $P_{\ell,\mathtt{Ham}}$.

*and any running time parameter $t_{AE}$. Also, let $\Pi_{RRSS}=(\mathsf{ShareGen}_{2^\lambda},\mathsf{SecretRecover}_{2^\lambda},\mathsf{Identify}_{2^\lambda})$ be a $(n,k=n-\ell,\epsilon(\lambda))$-Random Robust secret sharing scheme. Then for any $t$ and any security parameter $\lambda$ **Construction 2** provides $\left(t,t_{\mathtt{Sim}},\epsilon(t,\lambda)\right)$-conditional encryption secrecy with $\epsilon(t,\lambda) \le \epsilon_{AE}(t',\lambda)+2^{-\lambda}$, $t=t_{AE}(\lambda)$ and $t_{\mathtt{Sim}}=n\mathsf{d_{Pack}} \cdot t_{P.\mathsf{Enc}}+\mathtt{poly}(\lambda)$.*

**Proof of Theorem 4:** For this proof, we use hybrid argument and we define a sequence of indistinguishable hybrids. In the first hybrid, the distinguisher is given the output of conditional encryption ciphertext, and in the last hybrid, the distinguisher is given the output of our simulator described in Figure 10. Indistinguishability of these hybrids implies the indistinguishability of the conditional ciphertext and the simulator $\mathtt{Sim}(pk)$, that is, satisfying conditional encryption secrecy in the malicious setting.

Design of simulator $\mathtt{Sim}(pk)$

1. Sample, $r''_1,\ldots,r''_{\mathsf{d_{Pack}}\times n} \in_R \mathbb{Z}_N^{*\,n}, R''_1,\ldots,R''_n \in_R \mathbb{Z}_N{}^{\mathsf{d_{Pack}}\times n}$ uniformly at random
2. For all $1 \le i \le n$ compute $\tilde{c}'_i = \mathsf{P.Enc}_{pk}(R''_i; r''_i)$
3. Pick $R_K \in_R \{0,1\}^{l(\lambda)}$ uniformly at random and set $c'_{AE} = R_K$. //$l(\lambda)$ represents the ciphertext size of our authenticated encryption.
4. Output $\tilde{c}' = (1, \tilde{c}'_1, \cdots, \tilde{c}'_{n\mathsf{d_{Pack}}}, c'_{AE})$.

Figure 10: The simulator $\mathtt{Sim}$ for **Construction 2**

- **Hybrid 0**: In this hybrid the distinguisher $\mathcal{D}$ is given $\left(sk, pk, m_2, m_3, c_1, (1, \mathsf{d_{Pack}}, \tilde{c})\right)$ in which $\tilde{c} = (\tilde{c}_1, \ldots, \tilde{c}_n, c_{AE}) \leftarrow \mathsf{CEnc}_{pk}(c_1, m_2, m_3)$.
- **Hybrid 1**: Let $T_1 = \{j : m_2[j] \neq m_1[j]\}$. We define **Hybrid 1** similar to **Hybird 0**, except for all $j \in T_1$ we replace

$$\tilde{c}[j] = \left( c[j,i] = \mathsf{P.Enc}_{pk}\left( R_{j,i}\left(m_1[j] - m_2[j]\right) + \mathsf{REnc}\left(p_{j,i}, a_i, N\right)\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$$

  with $\tilde{c}[j] = \left(\mathsf{P.Enc}_{pk}(R'_{j,i})\right)_{i=1}^{\mathsf{d_{Pack}}}$ where $R'_{j,i} \in_R \mathbb{Z}_N$ are fresh and uniform random values chosen from $\mathbb{Z}_N$ and $a_i = 2^{\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$ for all $1 \le i \le \mathsf{d_{Pack}} - 1$ and $a_{\mathsf{d_{Pack}}} = 2^{m - \mathsf{d_{Pack}} \lceil \frac{m}{\mathsf{d_{Pack}}} \rceil + \lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$.
- **Hybrid 2** Let $T_2 = [n] \setminus (T_1)$ be the remaining indices. We define **Hybrid 2** similar to **Hybird 1**, except for all $j \in T_2$ we replace

$$\tilde{c}[j] = \left( c[j,i] = \mathsf{P.Enc}_{pk}\left( R_{j,i}\left(m_1[j] - m_2[j]\right) + \mathsf{REnc}\left(p_{j,i}, a_i, N\right)\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$$

  with $\tilde{c}[j] = \left( c[j,i] = \mathsf{P.Enc}_{pk}\left( R_{j,i}\left(m_1[j] - m_2[j]\right) + \mathsf{REnc}\left(r_{j,i}, a_i, N\right)\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$ where $r_{j,i} \in_R [a_i]$ are fresh and uniform random values chosen from $[a_i]$ and $a_i = 2^{\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$ for all $1 \le i \le \mathsf{d_{Pack}} - 1$ and $a_{\mathsf{d_{Pack}}} = 2^{m - \mathsf{d_{Pack}} \lceil \frac{m}{\mathsf{d_{Pack}}} \rceil + \lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$.
- **Hybrid 3** We define **Hybrid 3** similar to **Hybird 2**, except for all $j \in T_2$ we replace

$$\tilde{c}[j] = \left( c[j,i] = \mathsf{P.Enc}_{pk}\left( R_{j,i}\left(m_1[j] - m_2[j]\right) + \mathsf{REnc}\left(r_{j,i}, a_i, N\right)\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$$

with

$$\tilde{c}[j]=\left(c[j,i]=\mathsf{P.Enc}_{pk}\left(R_{j,i}\left(m_1[j]-m_2[j]\right)+\hat{R}_{j,i}\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$$

where $\hat{R}_{j,i}\in_R\mathbb{Z}_N$ are fresh and uniform random values chosen from $\mathbb{Z}_N$ and $a_i=2^{\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil}$ for all $1\leq i\leq\mathsf{d_{Pack}}-1$ and $a_{\mathsf{d_{Pack}}}=2^{m-\mathsf{d_{Pack}}\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil+\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil}$.

– **Hybrid 4** We define **Hybrid 4** similar to **Hybird 3**, except for all $j\in T_2$ we replace

$$\tilde{c}[j]=\left(c[j,i]=\mathsf{P.Enc}_{pk}\left(R_{j,i}\left(m_1[j]-m_2[j]\right)+\hat{R}_{i,j}\right)\right)_{i=1}^{\mathsf{d_{Pack}}}$$

with $\tilde{c}[j]=\left(\mathsf{P.Enc}_{pk}(R'_{j,i})\right)_{i=1}^{\mathsf{d_{Pack}}}$ where $R'_{j,i}\in_R\mathbb{Z}_N$ are fresh and uniform random values chosen from $\mathbb{Z}_N$.

**Hybrid 5**: This hybrid is exactly the same as the previous hybrid except we replace $c_{AE}$ with $c'_{AE}\in_R\{0,1\}^{l(\lambda)}$ a $l(\lambda)$-bit string chosen uniformly at random. We note that $l(\lambda)$ is a polynomial over the security parameter $\lambda$ which represents the ciphertext size of authenticated encryption.

– **Hybrid 6**: We replace the ciphertext of the conditional encryption with the output of the simulator `Sim` described in Figure 10.

Now, we will show that all these hybrids are indistinguishable. We define $\mathcal{D}^i=1$ to denote that the event that the distinguisher output 1 in hybrid $i$.

– **Hybrid 0 ≡ Hybrid 1**: These hybrids are equivalent and we have $\Pr[\mathcal{D}^{H_0}=1]=\Pr[\mathcal{D}^{H_1}=1]$. The argument is similar to the security proof for the equality test, as well as the first hybrid in the proof of Hamming distance in [1]. Intuitively, since these are the location of non-matching characters, then the condition encryption decrypts to a uniformly chosen random number and we can replace all the corresponding ciphertexts with uniformly chosen ciphertext.

– **Hybrid 1 ≡ Hybrid 2**: We are replacing each
In Hybrid 1 we information theoretically eliminated information about any share $s]]_i$ with $j\in T_1$. Since $P_{\ell,\mathtt{Ham}}(m_1,m_2)=0$, we have $|T_1|>\ell$ which implies that the set $T_2$ of valid shares has size $|T_2|<n-\ell$. Let $T_2=\{i_1,...,i_t\}$ with $t<n-\ell$. $n-\ell-1$-perfect privacy of our RRSS (**Construction 1**) guarantees that the shares $\left([[s]]_{i_1},...,[[s]]_{i_t}\right)$ are uniformly random elements and we can replace $[[s]]_j$ with the random element $s_{j,r}$. This is equivalent to picking uniformly binary string $\hat{s}_{j,r}\in\{0,1\}^m$ and divide it in $\mathsf{d_{Pack}}$ smaller parts. We then compute its equivalent integers in either $\mathbb{Z}_{a_i}$, that is $\left(p_{j,i,r}\right)_{\forall i\in[\mathsf{d_{Pack}}]}\in\mathbb{Z}_{2^{\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil}}^{\mathsf{d_{Pack}}-1}\times\mathbb{Z}_{a_{\mathsf{d_{Pack}}}}$. Then, $\left(p_{j,i,r}\right)_{\forall i\in[\mathsf{d_{Pack}}]}$ is a vector of integers representing $\hat{s}_{j,r}$ through a one-to-one mapping of $2^m$ possible $m$-bit share $\hat{s}_{j,r}$ to $(2^{\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil})^{\mathsf{d_{Pack}}-1}\times a_{\mathsf{d_{Pack}}}=(2^{\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil})^{\mathsf{d_{Pack}}-1}\times 2^{m-\mathsf{d_{Pack}}\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil+\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil}=2^m$ possible vectors of $\mathsf{d_{Pack}}$ integers. So, all the individual $\hat{p}_{i,j,r}$ integers look uniformly at random and we have $\Pr[\mathcal{D}^1=1]=\Pr[\mathcal{D}^2=1]$.

– **Statistically indistinguishability of Hybrid 2 and 3** For each $i\in T_2$ and each $i\in[\mathsf{d_{Pack}}]$ we are replacing the randomized encoding $\mathsf{REnc}(\hat{p}_{i,j,r},a_i,N)$ with a uniformly random element in $\mathbb{Z}_N$. To bound the statistical distance we apply Theorem 3 with parameters $a\in\{2^{\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil},2^{m-\mathsf{d_{Pack}}\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil+\lceil\frac{m}{\mathsf{d_{Pack}}}\rceil}\}\leq 2^{m/\mathsf{d_{Pack}}}$, $k=\lfloor\frac{N}{a}\rfloor$

and $b = N = pq$. Union bounding over all shares $i \in T_2$ and all $j \in [\mathsf{d_{Pack}}]$ the statistical distance between Hybrid 2 and 3 is upper bounded by

$$n\mathsf{d_{Pack}}\frac{2^{m/\mathsf{d_{Pack}}}}{N} \leq 2^{-\lambda}$$

By proper selection of the values of $\mathsf{d_{Pack}}$ and $N$ we ensure that $n\mathsf{d_{Pack}}\frac{2^{m/\mathsf{d_{Pack}}}}{N} \leq 2^{-\lambda}$ and we get the distinguishing advantage we like, i.e., $2^{-\lambda}$. So we have $\left|\Pr\left[\mathcal{D}^2=1\right]-\Pr\left[\mathcal{D}^3=1\right]\right| \leq 2^{-\lambda}$.

- **Hybrid 3≡ Hybrid 4**: These hybrids are information theoretically indistinguishable as $R_{i,j}(m_1[j]-m_2[j])+R'_{j,i}$ is already uniformly random in $\mathbb{Z}_N$. This is because of the fact that the $j$-th character of the control message $m_2$ and unknown message $m_1$ are not matching, i.e., $m_2[j] \neq m_1[j]$ and we have $m_1[j]-m_2[j] \neq 0$. Since we are in the semi-hones model, we have $m_1[j]-m_2[j] < \min\{p,q\}$ and the message space is smaller than $\min\{p,q\}$. This implies that $\gcd(m_1[j]-m_2[j],N)=1$ and therefore $R_{i,j}(m_1[j]-m_2[j])$ is uniformly random i.e., for all $x \in \mathbb{Z}_N$ we have $\Pr_R[R(m_1[j]-m_2[j])=x] = \Pr_R[R = x(m_1[j]-m_2[j])^{-1}] = \frac{1}{N}$. Therefore, these two hybrids are information theoretically equivalent and we have $\Pr[\mathcal{D}^3=1]=\Pr[\mathcal{D}^4=1]$.
- **Indistinguishability of Hybrid 4 and 5**: By **Hybrid 4**, we have information theoretically eliminated any information about the secret key $K$ for our authentication encryption scheme from $(\tilde{c}_1,\ldots,\tilde{c}_n)$. Thus, by AE security any adversary running in time at most $t_{AE} = t_{AE}(\lambda)$ can distinguish between $c_{AE}$ and $c'_{AE}$ with the advantage of at most $\epsilon_{AE}(t_{AE},\lambda)$. We defined $t=t_{AE}$ so we have

$$\left|\Pr\left[\mathcal{D}^{H_4}\right]-\Pr\left[\mathcal{D}^{H_5}=1\right]\right| \leq \epsilon_{AE}(t_{AE},\lambda) \tag{8}$$

  **Hybrid 5≡ Hybrid 6** Looking at the definition of our simulator in Figure 10, we can see that the conditionally encrypted ciphertext in Hybrids 5 and 6 are generated in exactly the same way. It follows that the hybrids are information-theoretically equivalent and we have

$$\Pr\left[\mathcal{D}^{H_5}\right]=\Pr\left[\mathcal{D}^{H_6}\right] \tag{9}$$

Putting everything together we have

$$\begin{aligned}
&\left|\Pr\left[\mathcal{D}\left(sk,pk,m_2,m_3.\mathsf{CEnc}_{pk}\left(\mathsf{Enc}_{pk}(m_1),m_2,m_3\right)\right)=1\right]\right. \\
&\left.-\Pr\left[\mathcal{D}\left(sk,pk,m_2,m_3,\mathtt{Sim}(pk)\right)=1\right]\right| \\
&=\left|\Pr\left[\mathcal{D}^{H_0}\right]-\Pr\left[\mathcal{D}^{H_6}\right]\right| \\
&\leq \sum_{i=0}^{5}\left|\Pr\left[\mathcal{D}^{H_i}\right]-\Pr\left[\mathcal{D}^{H_{i+1}}\right]\right| \\
&< \epsilon_{AE}(t',\lambda)+2^{-\lambda}\ .
\end{aligned}$$

□

*Statistical Distance for Randomized Encoding* Our analysis of conditional encryption security utilizes the following observation of Ameri and Blocki [1]. Theorem 3 is used to analyze the statistical distance between $\mathsf{REnc}(x,a,N)$ (when $x$ randomly chosen in $\mathbb{Z}_a$) and the uniform distribution over $\mathbb{Z}_N$.

**Theorem 3 ( [1]).** *Let $b=ak+r$ where $0\leq r<a$ is the reminder (i.e., $r=b \mod a$). Consider the uniform distributions $\mathcal{U}_b$ which outputs a random value in $\mathbb{Z}_b$ and the distribution $\mathcal{U}_{ak}$ which outputs random values between $0,\cdots,ak-1$. Then the statistical distance between these two distributions is $\mathsf{SD}(\mathcal{U}_{ak},\mathcal{U}_b)=\frac{r}{b}\leq\frac{1}{k+1}$.*

### D.4 Proving Correctness for Construction 2

**Theorem 4 (Correctness of Construction 2).** *Let $\Pi_{RRSS}$ be an $(n,k=n-\ell,\epsilon_{\mathsf{RRSS}}(\lambda)=2^{-\lambda})$-Random Robust secret sharing scheme. Then **Construction 2** is a $1-\epsilon_1(\lambda)$-correct with $\epsilon_1(\lambda)=2^{-\lambda+1}$. The scheme is $1-\epsilon(\lambda)$-error detecting conditional encryption scheme with $\epsilon(\lambda)=\binom{n}{\ell}\epsilon_{AE}(\lambda)+2^{-\lambda}$. Here,*

$$\epsilon_{AE}(\lambda)\doteq\max_m \Pr_{K_1,K_2\in\{0,1\}^\lambda}\left[\mathsf{Auth.Dec}_{K_1}\left(\mathsf{Auth.Enc}_{K_2}(m)\right)\neq\perp\right]$$

*denotes the negligible probability that the ciphertext $c=\mathsf{Enc}_{K_2}(m)$ is still valid under an unrelated key $K_1$. If $\Pi_{RRSS}$ achieves $(k-1,\epsilon_2(\lambda))$-security against malicious dealers then the scheme is $1-\epsilon_2(\lambda)-2^{-\lambda}$ error detecting.*

**Proof of Theorem 4:** We first note Authenticated Encryption security implies that the term $\epsilon_{AE}(\lambda)$ is negligible. Otherwise, an AE attacker could simply pick a random key $K'$ and use $c=\mathsf{Enc}_{K'}(m)$ as an attempted forgery for the unknown secret key $K$!

There are two conditions in Definition 5 which need to be proved. The first condition is regular encryption correctness and the other one is conditional encryption correctness. We also need to show that the scheme is error detecting.

**Correct of Regular Encryption:** The observation that $\mathsf{Dec}_{sk}\left(\mathsf{Enc}_{pk}(m;r)\right)=m$ for all messages $m\in\Sigma^n$, random coins $r$ and all $(sk,pk)$ in the support of the honest key generation algorithm follows immediately from the correctness of Pallier encryption. We now consider correctness for the conditional encryption algorithm.

**Correctness of Conditional Encryption:** We show that for all messages $m_1,m_2\in\Sigma^n$ such that $P_{\ell,\mathtt{Ham}}(m_1,m_2)=1$, all payload messages $m_3$, all $\{sk,pk\}$ in the support of our Key Generation algorithm and all random coins $r_1$ used to generate $c_1=\mathsf{Enc}_{pk}(m_1;r_1)$ we have

$$\Pr\left[\mathsf{Dec}_{sk}\left(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\right)=m_3|c_1=\mathsf{Enc}_{pk}(m_1;r_1)\right]\geq 1-\epsilon_1(\lambda)$$

where the randomness is taken over the selection of the random coins $r_2$ of the conditional encryption algorithm $\mathsf{CEnc}$.

Let $K$ denote the authenticated encryption key generated when we run $\mathsf{CEnc}$, let $\mathsf{d_{Pack}} \in pk$, let $[[s]]_1,...,[[s]]_n$ denote the shares $\{[[s]]_i \in \{0,1\}^m\}_{\forall i \in [n]}$ of $K$ such that are generated by our underlying random robust secret sharing. Let $\tilde{c}=(b,\tilde{c}_1,...,\tilde{c}_n,C_{AE})$ denote the output of $\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2))$, where $\forall i \in [n] : \tilde{c}_i = \left(c_i^1,c_i^1,...,c_i^{\mathsf{d_{Pack}}}\right)$. We recall that in the encryption, we divide the shares $[[s]]_i \in \{0,1\}^m$ into $\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil$ parts $[[s]]_{i,1},...,[[s]]_{i,\mathsf{d_{Pack}}} \in \{0,1\}^{\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$ and randomly encode each of them $[[\hat{s}]]_{i,j} \leftarrow \mathsf{REnc}([[s]]_{i,j},a_j,N) \in \mathbb{Z}_N^*$ as a random Paillier plaintext and we use to follow the equality test predicate to compute $(c_i^j)_{\forall 1 \le j \le \mathsf{d_{Pack}}}$. Here, for all $1 \le j \le \mathsf{d_{Pack}}-1$ we have $a_j = 2^{\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$ and $a_{\mathsf{d_{Pack}}} = 2^{m-\mathsf{d_{Pack}}\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil + \lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}$. So, in the conditional decryption algorithm, after decryption of all the Paillier ciphertexts associated to the same share, i.e., $\lceil \hat{s}' \rceil_{i,j} = \mathsf{P.Dec}_{sk}(c_i^j)$, we decode them to $[[s']]_{i,j} = [[\hat{s}']]_{i,j} \bmod a_j$ and combine all $\mathsf{d_{Pack}}$ together to reconstruct its corresponding share $[[s']]_i \in \{0,1\}^m$. Since the (de)packing scheme is bijective, for all uncorrupted indices we will recover valid shares, and for all corrupted indices we obtain a randomly corrupted share. Now, without loss of generality, we continue with the recovered shares.

Now, let $S^* = \{i \in [n] \,:\, m_2[i] = m_1[i]\}$ denote the indices of the characters where $m_2$ and $m_1$ match. For analytical simplicity, we initially assume that for all $i \in [n] \setminus S^*$, the shares $[[s]]_i \in_R \{0,1\}^m$ are drawn uniformly at random. We subsequently extend the proof to the actual distribution. So, if $P_{\mathtt{Ham},\ell}(m_1,m_2) = 1$, we have $|S^*| \ge n-\ell$ and, due to $(n,t=n-\ell,2^{-n\lambda_1})$-robustness of our RRSS scheme, we have

$$\Pr\left[S \subseteq S^* \wedge |S| \ge n-\ell \;\middle|\; S = \Pi_{\mathsf{RRSS}}.\mathsf{Identify}\left(\left([[s']]_i\right)_{\forall i \in [n]},\mathsf{pp}\right)\right] \ge 1-2^{-\lambda} \;,$$

where the probability is taken over the uniformly random corruptions of the shares $[[s']]_i$ for $i \notin S^*$. As long as $S \subseteq S^*$ we are guaranteed to recover the correct symmetric key i.e.,

$$K = K_S = \Pi_{\mathsf{RRSS}}.\mathsf{SecretRecover}\left(\left\{\left(n,k=n-\ell,[[s']]_i\right)_{i \in S}\right\},\mathsf{pp}\right) \;.$$

From the correctness of the authenticated encryption scheme it follows that $\mathsf{Auth.Dec}_{K_S}(c_{AE}) = m_3$.

In the previous paragraph, we assumed that the value $x_i = [[s']]_i \in_R \{0,1\}^m$ is uniformly random for each $i \notin S^*$. This is close, but it is not quite true. In reality, the distribution of $x_i = [[s']]_i$ is only close to the uniform distribution. According to the conditional encryption algorithm, we observe that for $i \notin S^*$ the distribution of $[[s']]_i \in \{0,1\}^m$ is as follows, sample uniformly random vector $(\hat{y}_0,\hat{y}_1,...,\hat{y}_{\mathsf{d_{Pack}}}) \in_R \mathbb{Z}_N^{\mathsf{d_{Pack}}}$ and $\forall j \le \mathsf{d_{Pack}}$ compute $\hat{y}'_j = \hat{y}_j \bmod a_j$ (and represent it as a $\log_2 a_j$-bit number and when we concatenate all these bits strings together we get $\hat{y}'_1 \| ... \| \hat{y}'_{\mathsf{d_{Pack}}} \in \{0,1\}^m$ which is $m$-bit string. By applying the results of Theorem 3 with $a_0 = a_j$ and $k_0 = \lfloor \frac{N}{a_j} \rfloor - 1$, the statistical distance between original/modified distributions of our recovered shares

$x_1,\ldots,x_n$ is upper bounded by $|S|\left(\mathsf{d}_{\mathsf{Pack}}\frac{1}{k_0+1}\right)\leq\frac{n\mathsf{d}_{\mathsf{Pack}}a_0}{N}\leq\frac{n\mathsf{d}_{\mathsf{Pack}}2^{\lceil\frac{m}{\mathsf{d}_{\mathsf{Pack}}}\rceil}}{N}$. By choice of $\mathsf{d}_{\mathsf{Pack}}$ we have $\frac{n\mathsf{d}_{\mathsf{Pack}}2^{\lceil\frac{m}{\mathsf{d}_{\mathsf{Pack}}}\rceil}}{N}\leq 2^{-\lambda}$. It follows that

$$\Pr\left[\mathsf{Dec}_{sk}\left(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\right)=m_3\middle|c_1=\mathsf{Enc}_{pk}(m_1;r_1)\right]\geq 1-2^{-\lambda}-2^{-\lambda}=1-\epsilon_1(\lambda)\ .$$

**Error Detection for Conditional Ciphertexts:** We will show that for all messages $m_1,m_2\in\Sigma^n$ such that $P_{\ell,\mathtt{Ham}}(m_1,m_2)=0$, all payload messages $m_3$, all $\{sk,pk\}$ in the support of our Key Generation algorithm and all random strings $r_1\in_R(\mathbb{Z}_N^*)^n$ we have

$$\Pr\left[\mathsf{Dec}_{sk}\left(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\right)=\perp\middle|c_1=\mathsf{Enc}_{pk}(m_1;r_1)\right]\geq 1-2^{-n\lambda}-2^{-\lambda}\geq 1-\epsilon(\lambda)\ .$$

where the randomness is taken over the random coins $r_2$ of the conditional encryption algorithm $\mathsf{CEnc}$.

As above we let $S^*\subseteq[n]$ denote the correct indices where $m_1$ and $m_2$ match. Because the predicate $P_{\ell,\mathtt{Ham}}(m_1,m_2)=0$ we have $|S^*|<n-\ell$. For simplicity we will first assume that for each $i\notin S^*$ the corresponding share $[[s']]_i$ is uniformly randomly corrupted. By perfect privacy the uncorrupted shares in $S^*$ are also distributed uniformly at random. For each fixed set $S\subseteq[n]$ of size $|S|=n-\ell$ we can view

$$K_S=\Pi_{\mathsf{RRSS}}.\mathsf{SecretRecover}\left(\left\{\left(n,k=n-\ell,[[s']]_i\right)_{i\in S}\right\},\mathsf{pp}\right)\ ,$$

as uniformly random. If $K_S$ is uniformly random then $\Pr[\mathsf{Auth.Dec}_{K_S}(c_{AE})\neq\perp]\leq\epsilon_{AE}(\lambda)$. Union bounding over all possible $S\subseteq[n]$ of size $|S|=n-\ell$ we have

$$\Pr\left[\exists S\subseteq[n].|S|=n-\ell\wedge\mathsf{Auth.Dec}_{K_S}(c_{AE})\neq\perp\right]\leq\binom{n}{\ell}\epsilon_{AE}(\lambda)\ .$$

If $K_S$ fails to decrypt $c_{AE}$ for all possible $S$ then the conditional decryption algorithm must output $\perp$ as $\Pi_{\mathsf{RRSS}}.\mathsf{Identify}\left(\left([[s']]_i,\right)_{\forall i\in[n]},\mathsf{pp}\right)$ will either output $\perp$ or a set $S$ of size $|S|=n-\ell$.

In the previous paragraph, we assumed that the value $x_i=[[s']]_i\in_R\{0,1\}^m$ is uniformly random for each $i\notin S^*$. This is close, but it is not quite true. In reality, the distribution of $x_i=[[s']]_i$ is close to the uniform distribution. According to the conditional encryption algorithm, we observe that for $i\notin S^*$ the distribution of $[[s']]_i\in\{0,1\}^m$ is as follows, sample uniformly random vector $(\hat{y}_0,\hat{y}_1,\ldots,\hat{y}_{\mathsf{d}_{\mathsf{Pack}}})\in_R\mathbb{Z}_N^{\mathsf{d}_{\mathsf{Pack}}}$ and $\forall j\leq\mathsf{d}_{\mathsf{Pack}}$ compute $\hat{y}'_j=\hat{y}_j \bmod a_j$ (and represent it as a $\log_2 a_j$-bit number and when we concatenate all these bits strings together we get $\hat{y}'_1\|\ldots\|\hat{y}'_{\mathsf{d}_{\mathsf{Pack}}}\in\{0,1\}^m$ which is $m$-bit string. By applying the results of Theorem 3 with $a_0=a_j$ and $k_0=\lfloor\frac{N}{a_j}\rfloor-1$, the statistical distance between original/modified distributions of our recovered shares $x_1,\ldots,x_n$ is upper bounded by $|S|\left(\mathsf{d}_{\mathsf{Pack}}\frac{1}{k_0+1}\right)\leq\frac{n\mathsf{d}_{\mathsf{Pack}}a_0}{N}\leq\frac{n\mathsf{d}_{\mathsf{Pack}}2^{\lceil\frac{m}{\mathsf{d}_{\mathsf{Pack}}}\rceil}}{N}\leq 2^{-\lambda}$. It follows that

$$\Pr\left[\mathsf{Dec}_{sk}\left(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\right)=\perp\middle|c_1=\mathsf{Enc}_{pk}(m_1;r_1)\right]\geq 1-2^{-\lambda}-\binom{n}{\ell}\epsilon_{AE}(\lambda)\geq 1-\epsilon(\lambda)\ .$$

**Error Detection for Conditional Ciphertexts with Dealer Private RRSS Scheme:** Assuming that the RRSS sharing scheme is $(n-\ell-1,\epsilon_2(\lambda))$-dealer private we will show that for all messages $m_1,m_2 \in \Sigma^n$ such that $P_{\ell,\mathtt{Ham}}(m_1,m_2)=0$, all payload messages $m_3$, all $\{sk,pk\}$ in the support of our Key Generation algorithm and all random strings $r_1 \in_R (\mathbb{Z}_N^*)^n$ we have

$$\Pr\Big[\mathsf{Dec}_{sk}\big(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\big)=\perp \Big| c_1=\mathsf{Enc}_{pk}(m_1;r_1)\Big] \geq 1-\epsilon_2(\lambda)-2^{-\lambda} \geq 1-\epsilon(\lambda) \ .$$

where the randomness is taken over the random coins $r_2$ of the conditional encryption algorithm $\mathsf{CEnc}$.

Let $S^*=\{i : m_1[i]=m_2[i]\}$ and observe that $|S^*| \leq n-\ell-1$. The remaining shares $[[s']]_i$ for $i \notin S^*$ are statistically close to the uniform distribution. For now let us view $[[s']]_i$ as uniformly random for each $i \notin S^*$. Then by $(n-\ell-1,\epsilon_2(\lambda))$-dealer privacy we have

$$\Pr\left[\Pi_{\mathsf{RRSS}}.\mathsf{Identify}\Big(\big([[s']]_i\big)_{\forall i\in[n]},\mathsf{pp}\Big)=\perp\right] \geq 1-\epsilon_2(\lambda) \ .$$

As long as $\Pi_{\mathsf{RRSS}}.\mathsf{Identify}\Big(\big([[s']]_i\big)_{\forall i\in[n]},\mathsf{pp}\Big)=\perp$ the conditional decryption algorithm will certainly output $\perp$.

In the previous paragraph, we assumed that the value $x_i=[[s']]_i \in_R \{0,1\}^m$ is uniformly random for each $i \notin S^*$. This is close, but it is not quite true. In reality, the distribution of $x_i=[[s']]_i$ is close to the uniform distribution. According to the conditional encryption algorithm, we observe that for $i \notin S^*$ the distribution of $[[s']]_i \in \{0,1\}^m$ is as follows, sample uniformly random vector $(\hat{y}_0,\hat{y}_1,...,\hat{y}_{\mathsf{d_{Pack}}}) \in_R \mathbb{Z}_N^{\mathsf{d_{Pack}}}$ and $\forall j \leq \mathsf{d_{Pack}}$ compute $\hat{y}'_j=\hat{y}_j \bmod a_j$ (and represent it as a $\log_2 a_j$-bit number and when we concatenate all these bits strings together we get $\hat{y}'_1\|...\|\hat{y}'_{\mathsf{d_{Pack}}} \in \{0,1\}^m$ which is $m$-bit string. By applying the results of Theorem 3 with $a_0=a_j$ and $k_0=\lfloor \frac{N}{a_j} \rfloor -1$, the statistical distance between original/modified distributions of our recovered shares $x_1,...,x_n$ is upper bounded by $|S|\left(\mathsf{d_{Pack}}\frac{1}{k_0+1}\right) \leq \frac{n\mathsf{d_{Pack}}a_0}{N} \leq \frac{n\mathsf{d_{Pack}}2^{\lceil \frac{m}{\mathsf{d_{Pack}}} \rceil}}{N} \leq 2^{-\lambda}$. It follows that

$$\Pr\Big[\mathsf{Dec}_{sk}\big(\mathsf{CEnc}_{pk}(c_1,m_2,m_3;r_2)\big)=\perp | c_1=\mathsf{Enc}_{pk}(m_1;r_1)\Big] \geq 1-2^{-\lambda}-\epsilon_2(\lambda) \ .$$

□

### D.5 Proof of ROR security of Construction 2

We also prove that Construction 2 provides ROR security.

**Theorem 5 (ROR security of Construction 2).** *Assume that the Paillier encryption scheme is $(t_{\mathtt{P}}(\lambda),q_{\mathtt{P}}(\lambda),\epsilon_{\mathtt{P}}(t_{\mathtt{P}}(\lambda),q_{\mathtt{P}}(\lambda),\lambda))$-Real-Or-Random secure and that $\Pi_{AE}$ is $(t_{AE}(\lambda),q_{AE}(\lambda),\epsilon_{AE}(t_{AE}(\lambda),q_{AE}(\lambda),\lambda))$-Real-Or-Random. Then Construction 2 is $(t'(\lambda),q'(\lambda),\epsilon'(t'(\lambda),q'(\lambda),\lambda))$-ROR[15] secure with $t'(\lambda)=\min\{t_{\mathtt{P}}(\lambda),t_{AE}(\lambda)\}-O(n)$*

[15] We use ROR instead of Real-Or-Randonm for simplicity.

*and* $\epsilon'(t'(\lambda),q'(\lambda),\lambda) = 2n(\mathsf{d_{Pack}}+1)\epsilon_{\mathsf{P}}(t_{\mathsf{P}}(\lambda),q_{\mathsf{P}}(\lambda),\lambda) + 2\epsilon_{AE}(t_{AE}(\lambda),q_{AE}(\lambda),\lambda)$ *and* $q'(\lambda) = \min\{\frac{q_{\mathsf{P}}(\lambda)}{2n(\mathsf{d_{Pack}}+1)}, \frac{q_{AE}(\lambda)}{2}\}$.

**Proof of Theorem 5:** (Sketch) Let $c_1 \in \mathbb{Z}_{N^2}$ (we do not assume $c_1 \in \mathbb{Z}^*_{N^2}$). And consider the query $(c_1, m_2, m_3)$ to the encryption oracle. If $b = 1$, then we have $(b = 1, \mathsf{d_{Pack}}, c, c_{AE})$ in which $c = \big(c[1],...,c[n]\big) \leftarrow \mathsf{CEnc}_{pk}(c_1, m_2, m_3)$ where $c[i] = \big(c[i,1],...,c[i,\mathsf{d_{Pack}}]\big)_{\forall i \in [n]}$ and for all $j \in [\mathsf{d_{Pack}}]$ we have $c[i,j] = c_1[i]^{R_{i,j}}((N+1)^{-m_2[i]} r_{1,i,j}^N)^{R_{i,j}}(1+N)^{p_{i,j}} r_{2,i,j}^N$ for uniformly random $R_{i,j} \in \mathbb{Z}_N$ and $r_{1,i,j}, r_{2,i,j} \in_R \mathbb{Z}^*_N$. By ROR security of our Paillier encryption scheme, we can apply a Hybrid argument, and can swap $(1+N)^{-m_2[i]} r_{1,i,j}^N$ (resp. $(1+N)^{p_{i,j}} r_{2,i,j}^N$) with $(1+N)^{-r_1[i]} r_{1,i,j}^N$ (resp. $(1+N)^{-r_2[i]} r_{2,i,j}^N$) with $r_1[i], r_2[i] \in_R \mathbb{Z}_N$. After $n + n\mathsf{d_{Pack}} = n(\mathsf{d_{Pack}} + 1)$ hybrids, we have replaced each $c[i,j]$ with a ciphertext of the form $c_1[i]^{R_{i,j}}(1+N)^{-r_1[i]}(r_{1,i,j})^N)^{R_{i,j}}(1+N)^{r_2[i]} r_{2,i,j}^N \bmod N^2$. In particular, all information about the secret symmetric key $K$ is removed so we can invoke ROR security for Authenticated encryption to replace $c_{AE}$ with a random ciphertext. We can then reverse the hybrids to move to transition from $\mathsf{CEnc}_{pk}(c_1, m_2, m_3)$ (when $b = 0$) all the way to $b = 1$ case $\mathsf{CEnc}_{pk}(c_1, m'_2, m'_3)$ for random messages $m'_2, m'_3$. Thus, we adjust $q'(\lambda) = q_{\mathsf{P}}(\lambda)/(2n(\mathsf{d_{Pack}}+1))$ since we invoke ROR security for Paillier during $(2n(\mathsf{d_{Pack}}+1)) \times q'(\lambda)$ hybrids. Similarly, we invoke ROR security for authenticated encryption in $q_{AE}(\lambda) = 2q'(\lambda)$ hybrids. So, we can set $q'(\lambda) = \min\{\frac{q_{\mathsf{P}}(\lambda)}{2n(\mathsf{d_{Pack}}+1)}, \frac{q_{AE}(\lambda)}{2}\}$. Applying the union bound, we have $\epsilon'(t'(\lambda),q'(\lambda),\lambda) = 2n(\mathsf{d_{Pack}}+1)\epsilon_{\mathsf{P}}(t_{\mathsf{P}}(\lambda),q_{\mathsf{P}}(\lambda),\lambda) + 2\epsilon_{AE}(t_{AE}(\lambda),q_{AE}(\lambda),\lambda)$ where $t'(\lambda) = \min\{t_{\mathsf{P}}(\lambda), t_{AE}(\lambda)\} - O(n)$. □

### D.5.1 Useful Lemma for Equivalency of the share Distributions after Packing the RRSS Shares

After packing and unpacikng the output of our RRSS scheme, one may be wondering if the resulting distribution is the same os the shares distribution or not. Since we using one-to-one mapping, the distribution will remain the same even after packing/unpacking. So for the sake of completeness we provide the formal lemma and its proof to support our argument.

**Lemma 5 (Substrings of a Uniform Binary String are Uniform).** *Let* $x = ab + r \geq 1$ *s.t.* $r < 0 \leq b$, $\mathsf{MaxM} = 2^x = 2^{ad+r}$ *be integers. Let* $R_{bin} \in_R \{0,1\}^{ad+r}$ *be* $x$*-bit binary string chosen uniformly at random and its corresponding decimal integer be the uniformly random number* $R \in \mathbb{Z}_{2^x}$*. We write* $R$ *as follows using a vector* $a+1$ *elements* $(d_0,...,d_{a-1},d_a)$ *such that* $(d_i)_{\forall 1 \leq i \leq a-1} \in \mathbb{Z}_{2^d}$ *and* $d_a \in \mathbb{Z}_{2^r}$ *as follows:*

$$R = \sum_{i=0}^{a-2} d_i 2^i b + r_a 2^{(a-1)d+r}, \qquad \forall 0 \leq i \leq a-1 : d_i \in \mathbb{Z}_{2^d}, d_a \in \mathbb{Z}_{2^r}.$$

*Then the digits* $(d_0,...,d_a)$ *are independent and, for each* $0 \leq i \leq a$, $d_i$ *is uniform on* $\mathbb{Z}_{2^d}$ *and* $d_a \in_R \mathbb{Z}_{2^r}$.

**Proof of Lemma 5:** Consider the map

$$f : \{0,1,...,2^d-1\}^a \times \{0,1,...,2^r-1\} \longrightarrow \{0,1,...,2^x-1 = 2^{ad+r}-1\}, f(d_0,...,d_a) = \left(\sum_{i=0}^{a-2} d_i 2^{di}\right) + d_a 2^{d(a-1)+r}.$$

We note that as all $2^x$ possible element are mapped one-to-one, then $f$ is a bijection.

Since $R$ is uniform on $\{0,...,2^x-1\}$ and $(d_0,...,d_a)=f^{-1}(R)$, it follows that $(d_0,...,d_a)$ is uniform on the $2^{ad+r}$-element set $\{0,...,2^d-1\}^a\times\{0,...,2^r-1\}$.

Fix any index $0\le i\le a-1$ and any $v\in\{0,...,2^d-1\}$. The number of $a+1$-tuples with $i$-th coordinate equal to $v$ is $2^{d(a-1)\times 2^r}$, hence

$$\Pr[d_i=v]=\frac{\left|\{(d_0,...,d_a):d_i=v\}\right|}{2^{ad+r}}=\frac{2^{d(a-1)}\times 2^r}{2^{ad+r}}=\frac{1}{2^d},$$

so each $d_i$ is uniform on $\{0,...,2^d-1\}$.

With similar argument we can show that the $a+1$-th index $d_a\in\nvDash$ is also chosen uniformly at random and we have

$$\Pr[d_a=v]=\frac{\left|\{(d_0,...,d_a):d_a=v\}\right|}{2^{ad+r}}=\frac{2^{d(a)}}{2^{ad+r}}=\frac{1}{2^r}\ .$$

which shows the digits are independently chosen uniformly at random. □

## E Constructions from Prior Work for Comparison

We highlight that fPAKE protocol from [5] (Figure 11) provides the relaxed security definition. The relaxed security definition is identical to the ideal functionality in Figure 1 except that the test password oracle TestPwd is replaced with the following test password query which uses three following leakage functions $L_c,L_m,L_f$ for two $n$ characters passwords $\mathtt{pwd}=(\mathtt{pwd}_1,...,\mathtt{pwd}_n)$ and $\mathtt{pwd}'=(\mathtt{pwd}'_1,...,\mathtt{pwd}'_n)$:

- $L_c(\mathtt{pwd},\mathtt{pwd}'):=\left(\left\{j:\text{ s.t. }\mathtt{pwd}_j=\mathtt{pwd}'_j\right\},\text{“correct guess”}\right)$.
- $L_m(\mathtt{pwd},\mathtt{pwd}'):=\left(\left\{j\text{ s.t. }\mathtt{pwd}_j=\mathtt{pwd}'_j\right\},\text{“wrong guess”}\right)$
- $L_f(\mathtt{pwd},\mathtt{pwd}'):=$ “wrong guess.

Now we describe the weaker notion of test password as follows:

- **TestPwd**: On (TestPwd,sid,$\mathsf{Party}_i$,$\mathtt{pwd}^*$) from the adversary $\mathcal{A}$: If there is a `fresh` record ($\mathsf{Party}_i$,$\mathtt{pwd}^{\mathsf{Party}_i}$), then set $\ell'\leftarrow\mathtt{Ham}(\mathtt{pwd}_{\mathsf{Party}_i},\mathtt{pwd}^*)$ and do:
  - If $\ell'\le\ell$, mark the record `compromised` and reply to $\mathcal{A}$ with the leakage $L_c(\mathtt{pwd}^{\mathsf{Party}_i},\mathtt{pwd}^*)$;
  - If $\ell<\ell'\le\gamma$, mark the record `compromised` and reply $L_m(\mathtt{pwd}^{\mathsf{Party}_i},\mathtt{pwd}^*)$.
  - $\gamma<\ell'$, mark the record `interrupted` and replay $\mathcal{A}$ with $L_f(\mathtt{pwd}^{\mathsf{Party}_i},\mathtt{pwd}^*)$.

Here $\ell$ denotes the Hamming distance threshold, and $n-\gamma$ represents the robustness parameter. In prior work on robust secret sharing [5,16] with perfect privacy, there exists a gap between the reconstruction threshold $t=n-\ell$ and the robustness bound $n-\gamma$. This gap corresponds to a strictly weaker security notion: when the Hamming distance between two passwords lies in the interval $(\ell,\gamma]$, the ideal functionality may still reveal the positions of matching characters, even though the key-exchange will be unsuccessful. In contrast, our RRSS construction eliminates this gap by ensuring that the reconstruction and robustness thresholds coincide. Consequently, we adopt the original TestPwd oracle, which does not include any auxiliary leakage functionality.

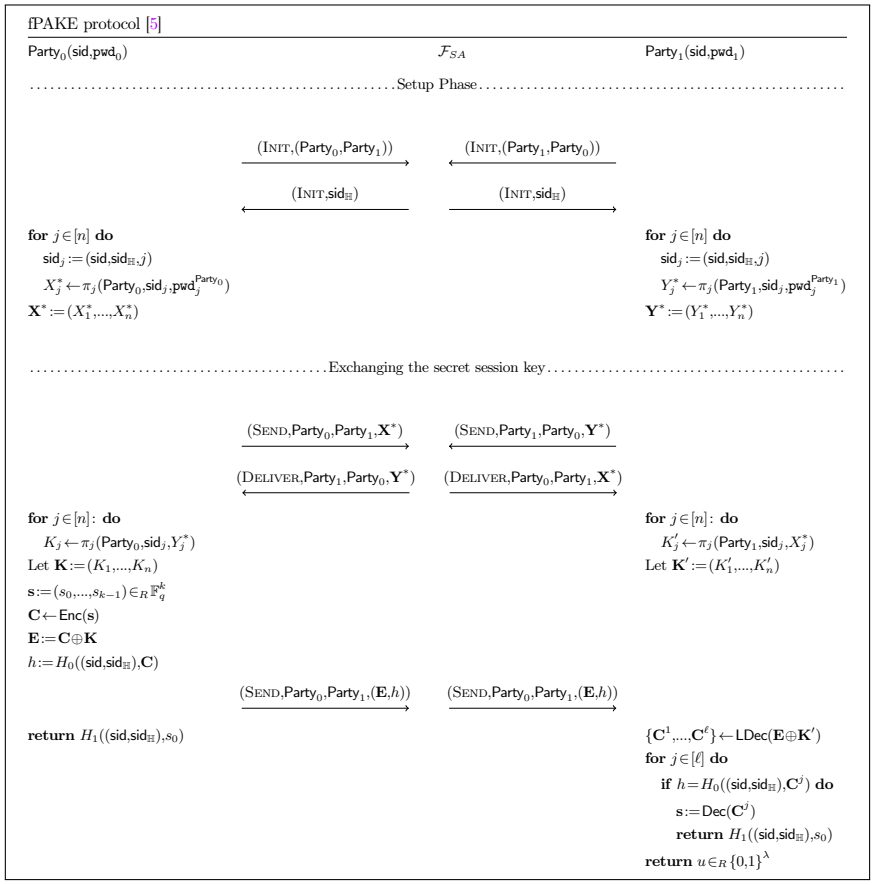



Figure 11: Description of $\pi_{\mathsf{fPAKE}}$ protocol from [5] that uses $n$ executions of the protocol $\pi_j$ (realizing $\mathcal{F}_{\mathsf{iPAKE}}$ in Figure 6) in the $\mathcal{F}_{\mathsf{SA}}$-hybrid model and random encoding scheme with list decoding $\Pi_{\text{RandEncode}} = (\mathsf{Enc}, \mathsf{Dec}, \mathsf{LDec})$. The protocol $\pi_{\mathsf{fPAKE}}$ only achieves a relaxed notion of fPAKE security.

**Construction 6**

$(sk,pk) \leftarrow \mathsf{KeyGen}(1^\lambda;r)$:

1. run $\left(\mathtt{P}.sk,\mathtt{P}.pk=(N)\right) \leftarrow \mathtt{P}.\mathsf{KeyGen}(1^\lambda;r)$.
   - If $\min\{p,q\}<|\Sigma|$ or $\gcd(N,\phi(N))\neq 1$ repeat step 1. //run $P$.KeyGen again.
3. set $\mathtt{SS}=\Pi_{SSS}.(\mathsf{ShareGen},\mathsf{SecretRecover})$ over finite field $\mathbb{F}_{2^\lambda}$.
2. set $sk=\mathtt{P}.sk$ and $pk=(\mathtt{P}.pk,\lambda)$.

$c \leftarrow \mathsf{Enc}_{pk}(m;r)$:

1. parse $m=(m[1],...,m[n]), r=(r_1,...,r_n) \in \left(\mathbb{Z}_N^*\right)^n$.
2. $\forall 1\leq i\leq n$:
   - compute $\hat{m}[i]=\mathsf{ToInt}(m[i])$
   - compute $c_i=(1+N)^{\hat{m}[i]}{r_i}^N \mod N^2$
3. output $c=(b=0,c_1,...,c_n)$.

$c \leftarrow \mathsf{CEnc}_{pk}(c_m,m_2,m_3;r)$:

1. parse $m_2=(m_2[1],...,m_2[n]) \in \Sigma^n$, $r=(r_1,...,r_n) \in \left(\mathbb{Z}_N^*\right)^n$, $c_m=(b,c_1,\cdots,c_n)$, $pk=(\mathtt{P}.pk,\lambda)$;
2. if $b=1$, output $\perp$.
4. $\forall i\in[n]$ : compute $\hat{m}_2[i]=\mathsf{ToInt}(m_2[i])$;
5. Pick $K\in_R\{0,1\}^\lambda$, compute $c_{AE}=\mathsf{Auth.Enc}_K(m_3)$;
6. $\left([[s]]_1,...,[[s]]_n\right) \leftarrow \mathtt{SS}.\mathsf{ShareGen}_\lambda(n,n-l,K)$;
7. $\forall j\in[n]$, compute $[[\hat{s}]]_j=[[s]]_j+2^\lambda k_j$ s.t. $k_j\in_R \left[\lfloor\frac{N}{2^\lambda}\rfloor\right]$;
8. $\forall j\in[n]$, compute $\tilde{c}[j]={c_j}^{-R_j}(N+1)^{R_j m_2[j]+[[\hat{s}]]_j}$
9. output $\tilde{c}=(b=1,\tilde{c}[1],\cdots,\tilde{c}[n],c_{AE})$;

$\{m,\perp\}=\mathsf{CDec}_{sk}\left((b,\tilde{c})\right)$:

2.1 if $b=1$ // The ciphertext is extracted from $\mathsf{CEnc}(\cdot)$
   1. parse $\tilde{c}=(\tilde{c}[1],\cdots,\tilde{c}[n],c_{AE})$, AND set $m=\perp$
   2. $\forall j\in[n]$ compute $[[\hat{s}']]_j=\mathtt{P}.\mathsf{Dec}_{sk}(c[j])$ and compute $[[s']]_j=[[\hat{s}']]_j \mod 2^\lambda$.
   3. For all $\binom{n}{n-\ell}$ possible $S\subseteq[n]$ with $|S|=n-\ell$
      - compute $K_S=\mathtt{SS}.\mathsf{SecretRecover}\left(\{[[s']]_j\}_{\forall j\in S}\right)$
      - compute $\left(v,\hat{m}'\right)=\mathsf{Auth.Dec}_{K_S}(c_{AE})$, if $v=1$, return $m=\hat{m}'$
   4. If for all $S\subseteq[n], v=0$, then return $m=\perp$.

2.2 if $b=0$ // The ciphertext is extracted from $\mathsf{Enc}(\cdot)$
   - parse $\tilde{c}=(\tilde{c}_1,\cdots,\tilde{c}_n)$, AND set $m=\perp$
   - for $1\leq i\leq n$, compute $\hat{m}_i=\mathtt{P}.\mathsf{Dec}_{sk}(\tilde{c}_i)$
   - return $m=\mathsf{ToInt}^{-1}(\hat{m}_1)\|...\|\mathsf{ToInt}^{-1}(\hat{m}_n)$

Figure 12: Ameri and Blocki's [1] concrete construction of conditional encryption over Hamming distance at most $\ell$, i.e., $P_{\ell,\mathtt{Ham}}$. Note that the running time of step 3 in CDec is $\Omega\left(\binom{n}{\ell}\right)$ which intractable for non-constant $\ell=\omega(1)$.